

fda_a2_25061944

April 2, 2024

1 Assessment Task 2: Data exploration and preparation

1.1 Author: Nicolas Huber

1.2 Student ID: 25061944

2 Scenario

A global space astrometry mission, Gaia is building the largest, most precise three-dimensional map of our Galaxy by surveying nearly two billion objects. Throughout the course of its mission, Gaia monitors each of its target stars about 14 times per year. It is precisely charting their positions, distances, movements, and changes in brightness. It is expected to discover hundreds of thousands of new celestial objects, such as extra-solar planets and brown dwarfs, and observe hundreds of thousands of asteroids within our own Solar System. The mission is also studying more than 1 million distant quasars and providing stringent new tests of Albert Einstein's General Theory of Relativity.

In this task, the Head of the Analytics Unit asks you to use Gaia observation to do 2-class (A,B) stellar classification to help understand the Galaxy. As you will see, this dataset is highly complicated and includes a lot of features that make this problem more challenging.

Your tasks include:

- understanding the specifics of the dataset;
- extracting information about each of the attributes, possible associations between them, and any other specifics of the dataset.
- The tasks in the assignment are specified below.

3 0. Preparation

```
[1]: import os
import math
import shutil

import numpy as np
import pandas as pd

from IPython.display import display, Markdown
```

```

import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.colors import Normalize
import matplotlib.cm as cm

import seaborn as sns

from scipy.stats import zscore, kurtosis, skew, norm
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler, \
    LabelBinarizer, RobustScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering

import umap.umap_ as umap
from sklearn.manifold import TSNE

from ydata_profiling import ProfileReport

```

```

[2]: # Suppress warnings
import warnings
warnings.filterwarnings('ignore')

```

```

[3]: import matplotlib
print('matplotlib: {}'.format(matplotlib.__version__)) # downgraded due to \
    heatmap bug

```

matplotlib: 3.7.3

4 Constants

```

[4]: # Set a random seed
SEED = 31011997

```

```

[5]: SAMPLE_RATIO = 0.1

```

```

[6]: # PARAMETERS TO BE ADJUSTED
SKEWNESS_THRESHOLD = 0.5
CORRELATION_THRESHOLD = 0.7
OUTLIER_COUNT_THRESHOLD = 0
MAX_CLUSTER_LEVEL = 6

```

```

[7]: sns.color_palette()

```

```

[7]: [(0.12156862745098039, 0.4666666666666667, 0.7058823529411765),
      (1.0, 0.4980392156862745, 0.054901960784313725),

```

```
(0.17254901960784313, 0.6274509803921569, 0.17254901960784313),
(0.8392156862745098, 0.15294117647058825, 0.1568627450980392),
(0.5803921568627451, 0.403921568627451, 0.7411764705882353),
(0.5490196078431373, 0.33725490196078434, 0.29411764705882354),
(0.8901960784313725, 0.4666666666666667, 0.7607843137254902),
(0.4980392156862745, 0.4980392156862745, 0.4980392156862745),
(0.7372549019607844, 0.7411764705882353, 0.13333333333333333),
(0.09019607843137255, 0.7450980392156863, 0.8117647058823529)]
```

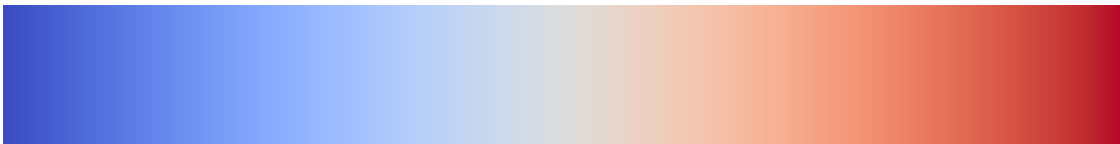
```
[8]: sns.color_palette("viridis", as_cmap=True)
```

```
[8]:
```



```
[9]: sns.color_palette("coolwarm", as_cmap=True)
```

```
[9]:
```



```
[10]: PALETTE = 'coolwarm'
```

5 Data Loading

```
[11]: # Data Loading
df = pd.read_csv(filepath_or_buffer="../data/32130_AT2_25061944.csv")
header = pd.read_excel(io="../data/head_description.xlsx")
```

```
[12]: #df = df.sample(frac=SAMPLE_RATIO, random_state=SEED) # To speed up
```

```
[13]: df.shape
```

```
[13]: (150, 29)
```

6 Setup

```
[14]: # Prepare output directory
def prepare_output_dir(directory):
    # Check if directory exists
    if os.path.exists(directory):
        # Remove directory and its contents
        for root, dirs, files in os.walk(directory, topdown=False):
            for file in files:
                os.remove(os.path.join(root, file))
            for dir in dirs:
                os.rmdir(os.path.join(root, dir))
        os.rmdir(directory)

    # Create new directory
    os.makedirs(directory)

[15]: def setup_folders(main_folder, subfolders):
    """
    Ensures a folder structure with multiple (potentially nested) empty
    ↪ subfolders.
    - main_folder: The path to the main folder.
    - subfolders: A list of subfolder paths to be created within the main
    ↪ folder. Nested folders can be specified using '/'.
    """

    print(f'Current working directory: {os.getcwd()}')
    # Ensure the main folder exists
    if not os.path.exists(main_folder):
        os.makedirs(main_folder)

    # Iterate through each subfolder path
    for subfolder_path in subfolders:
        full_path = os.path.join(main_folder, subfolder_path)

        # Check if the subfolder path exists
        if os.path.exists(full_path):
            # Clear the subfolder if it exists (remove then recreate)
            shutil.rmtree(full_path)

        # Create the subfolder path, including any nested directories
        os.makedirs(full_path)

[16]: main_folder = './output'
subfolders = ['exploration/statistics', 'exploration/features', 'exploration/
↪ reports', 'pk', 'excel', 'exploration/features/joint-plots', 'exploration/
↪ dimred/pca', 'exploration/dimred/hclustering']
```

```
setup_folders(main_folder, subfolders)
```

Current working directory: /Users/nicolas/Code/Personal/ml-hub/data_lab

7 1A. Initial data exploration

Identify the attribute type of each attribute in your dataset. If it's not clear, you may need to justify why you chose the type. For each attribute, conduct below studies for each of them: Identify the values of the summarising properties for the attributes, including frequency, location and spread (e.g. value ranges of the attributes, frequency of values, distributions, medians, means, variances, percentiles, etc. - the statistics that have been covered in the lectures and materials given). Note that not all of these summary statistics will make sense for all the attribute types, so use your judgement! Where necessary, use proper visualisations for the corresponding statistics.

Using KNIME or Python, explore multiple attributes relationship of your dataset, and identify any outliers, clusters of similar instances, “interesting” attributes and specific values of those attributes. Note that you may need to ‘temporarily’ recode attributes to numeric or from numeric to nominal. The report should include the corresponding snapshots from the tools and an explanation of what has been identified there.

```
[17]: df.describe()
```

```
[17]:
```

	Unnamed: 0.1	Unnamed: 0	RA_ICRS	DE_ICRS	Source	\
count	150.000000	150.000000	150.000000	150.000000	1.500000e+02	
mean	91061.646667	45061.646667	130.588906	43.272701	1.114598e+18	
std	55035.230123	27322.303333	119.641917	20.864169	9.727927e+17	
min	1401.000000	342.000000	0.541176	-22.027796	1.417396e+17	
25%	45104.750000	20138.750000	40.838805	36.492539	3.531856e+17	
50%	94970.000000	45841.000000	74.242906	48.399212	5.108945e+17	
75%	135780.250000	64808.500000	285.836696	58.363765	2.003751e+18	
max	184974.000000	98593.000000	358.405123	74.215014	3.122184e+18	

	Plx	PM	pmRA	pmDE	Gmag	...	\
count	150.000000	150.000000	150.000000	150.000000	150.000000	...	
mean	0.831655	4.016053	-0.942907	-1.310913	13.247697	...	
std	0.767924	5.931726	5.911947	3.721082	2.131076	...	
min	-1.272000	0.200000	-56.257000	-13.307000	7.977370	...	
25%	0.307850	1.289500	-2.192500	-2.719750	11.474332	...	
50%	0.598250	2.572500	-0.776000	-1.130500	13.519697	...	
75%	1.175100	4.372250	0.482250	-0.136250	14.830429	...	
max	4.791400	59.532000	27.722000	19.474000	17.421974	...	

	BP-G	G-RP	pscol	Teff	Dist	\
count	150.000000	150.000000	6.000000	150.000000	150.000000	
mean	0.303709	0.458808	1.546400	9720.521333	2244.057394	
std	0.202940	0.231987	0.123345	2494.776065	1768.430393	
min	-0.032467	-0.063511	1.382600	6808.600000	210.976500	

25%	0.157669	0.298389	1.478900	7852.175000	821.470600
50%	0.254522	0.441148	1.534750	9441.800000	1826.018600
75%	0.367696	0.580098	1.606200	10753.750000	3251.098850
max	0.907987	0.982482	1.736000	24443.700000	9546.603000

	Rad	Lum-Flame	Mass-Flame	Age-Flame	z-Flame
count	150.000000	148.000000	129.000000	97.000000	148.000000
mean	2.740126	113.563014	2.474047	0.655536	0.545032
std	1.318502	350.435020	0.885738	0.294695	0.183143
min	1.111500	5.821184	1.498000	0.201000	0.224140
25%	1.802750	16.660178	1.871000	0.398000	0.421489
50%	2.413700	31.080989	2.259000	0.587000	0.499100
75%	3.318500	75.207080	2.800000	0.899000	0.622282
max	10.394200	3275.650400	7.017000	1.238000	1.274817

[8 rows x 28 columns]

```
[18]: df.dtypes
```

```
[18]: Unnamed: 0.1      int64
      Unnamed: 0      int64
      RA_ICRS        float64
      DE_ICRS        float64
      Source         int64
      Plx            float64
      PM            float64
      pmRA          float64
      pmDE          float64
      Gmag           float64
      e_Gmag         float64
      BPmag          float64
      e_BPmag        float64
      RPyag          float64
      e_RPyag        float64
      GRVSmag        float64
      e_GRVSmag      float64
      BP-RP          float64
      BP-G           float64
      G-RP           float64
      pscol          float64
      Teff           float64
      Dist           float64
      Rad            float64
      Lum-Flame      float64
      Mass-Flame     float64
      Age-Flame      float64
      z-Flame        float64
```

```
SpType-ELS      object
dtype: object
```

```
[19]: # Initial Profiling report
#profile_raw = ProfileReport(df, title="Profiling Report of raw Data")
#profile_raw.to_file("./output/exploration/reports/profiling_report_raw.html")
```

```
[20]: # Exploratory statistics

def generate_statistics(df, print_col_stats=False):

    numeric_stats = []
    non_numeric_stats = []

    for col_num, column in enumerate(df.columns):
        data = df[column]

        # Display column name
        if print_col_stats:
            display(Markdown(f'## {column}'))

        # Determine if column is numeric or categorical
        if pd.api.types.is_numeric_dtype(data):

            # Calculating Z-score outliers
            outliers_count = (np.abs(zscore(data.dropna())) > 3).sum()
            # Safely calculating Z-Score Outliers Percentage
            z_score_outliers_percentage = (outliers_count / data.count()) * 100
            if outliers_count > 0 else 0

        # Descriptive Statistics for numeric data
        stats = {
            # General stats
            'Count': data.count(),
            'Mean': data.mean(),
            'Median': data.median(),
            'Mode': data.mode().values[0] if not data.mode().empty else np.
            nan,
            'Variance': data.var(),
            'Standard Deviation': data.std(),

            # Range
            'Range': data.max() - data.min(),
            'Minimum': data.min(),
            'Maximum': data.max(),
```

```

# Quantiles and Outliers
'IQR': data.quantile(0.75) - data.quantile(0.25),
'25%': data.quantile(0.25),
'50%': data.quantile(0.50),
'75%': data.quantile(0.75),
'Z-Score Outliers': outliers_count,
'Z-Score Outliers Ratio': z_score_outliers_percentage,
# Distribution
'Skewness': skew(data.dropna()),
'Kurtosis': kurtosis(data.dropna()),
#'IQR Outliers': 'See Box Plot',

# Values
'Missing Values': data.isna().sum(),
'Missing Values Ratio': data.isnull().mean() * 100,
'Unique Values': data.nunique(),
'Positive Values Count': (data > 0).sum(),
'Zero Values Count': (data == 0).sum(),
'Negative Values Count': (data < 0).sum()
}
numeric_stats.append(stats)

# Plotting
fig, ax = plt.subplots(3, 3, figsize=(18, 10))

# First row of plots (existing plots)
sns.boxplot(x=data, ax=ax[0, 0])
ax[0, 0].set_title('Box Plot')

sns.histplot(data, kde=True, ax=ax[0, 1])
ax[0, 1].set_title('Histogram & Density Plot')

sns.violinplot(x=data, ax=ax[0, 2])
ax[0, 2].set_title('Violin Plot')

# Second row for the new histogram plot
sns.boxplot(data=df, x=data, y="SpType-ELS",
            ax=ax[1, 0],
            palette=PALETTE,
            )
ax[1, 0].set_title('Boxplot by SpType-ELS')

sns.histplot(

```

```

        df,
        x=data, hue="SpType-ELS",
        multiple="stack",
        log_scale=False,
        ax=ax[1, 1],
        palette=PALETTE,
    )
    ax[1, 1].set_title('Stacked Histogram by SpType-ELS')

    sns.violinplot(data=df, x=data, y="SpType-ELS",
                   split=True, inner="quart", fill=False,
                   ax=ax[1, 2],
                   palette='coolwarm',
                   )
    ax[1, 2].set_title('Split Violinplot by SpType-ELS')

    sns.boxenplot(
        df, x=data, y="SpType-ELS",
        ax=ax[2, 0],
        palette=PALETTE,
    )
    ax[2, 0].set_title('Boxenplot by SpType-ELS')

    ax[2, 1].axis('off')
    ax[2, 2].axis('off')

    # Overall figure title and layout adjustments
    fig.suptitle(f'Descriptive Statistics for Numerical Attribute_{column}')
    plt.tight_layout()

    # Saving the figure
    plt.savefig(f'./output/exploration/statistics/{stats_{col_num}_{column}_numerical.png}')

    if print_col_stats:
        # Print statistics
        for key, value in stats.items():
            display(Markdown(f'- **{key}:** {value}'))
        plt.show()
    plt.close()

else:
    # Descriptive Statistics for categorical data
    stats = {

```

```

        'Count of Occurrences': data.value_counts(),
        'Missing Ratio': data.isnull().mean() * 100,
        'Unique Values': data.nunique()
    }
    non_numeric_stats.append(stats)

    # Visualization for categorical data
    plt.figure(figsize=(10, 5))
    sns.countplot(y=data, palette=PALETTE)
    plt.title(label=f'Frequency Count for Categorical Attribute_{column}')
    plt.savefig(f'./output/exploration/statistics/
stats_{col_num}_{column}_categorical.png')
    if print_col_stats:
        # Descriptive Statistics for categorical data
        display(Markdown(f'- **Count of Occurrences:**\n{data.
value_counts().to_frame()}'))
        display(Markdown(f'- **Missing Ratio:** {data.isnull().mean() *
100}%'))
        display(Markdown(f'- **Unique Values:** {data.nunique()}'))

    plt.show()
    plt.close()

    # Convert stats lists to DataFrames
    # numeric_df = pd.DataFrame(numeric_stats).transpose()
    # Convert stats lists to DataFrames with column names as indices for
numeric stats
    numeric_df = pd.DataFrame(numeric_stats, index=df.columns[df.apply(pd.api.
types.is_numeric_dtype)]).transpose()
    non_numeric_df = pd.concat({key: pd.DataFrame(value).transpose() for key,
value in enumerate(non_numeric_stats)}).unstack(0)

    # Print tables as Markdown
    print("### Numeric Columns Statistics\n")
    display(Markdown(numeric_df.transpose().to_markdown()))
    numeric_df.transpose().to_excel("./output/excel/stats_numeric.xlsx")
    print("\n### Non-Numeric Columns Statistics\n")
    display(Markdown(non_numeric_df.transpose().to_markdown()))
    non_numeric_df.to_excel("./output/excel/stats_non_numeric.xlsx")

```

[21]: generate_statistics(df=df, print_col_stats=False)

Numeric Columns Statistics

	Standard										Z-Score		Missing		Positives				
	De-	vi-	a-	Count	Mean	Median	Mode	Variance	Min	Max	Q1	Q2	Q3	Val-	Missing	Uniq-	Val-	Zero	Negative
	vi-	a-	Count	Mean	Median	Mode	Variance	Min	Max	Q1	Q2	Q3	Val-	Missing	Uniq-	Val-	Zero	Negative	
Unknown0.1	500	100	100	100	0.02558348357401184970675510487035780	0	0.03455501	0	150	150	0	0	1.25209	0	150	150	0	0	
Unknown0	500	100	100	100	7.4630782298251429859466208385841480805	0	0.1608450	0	150	150	0	0	1.04705	0	150	150	0	0	
RA_150139.5820291436412.657.86543358204.9083882289.837	500	139.5820291436412.657.86543358204.9083882289.837	139.5820291436412.657.86543358204.9083882289.837	139.5820291436412.657.86543358204.9083882289.837	1.0128	0	0.7817070	0	150	150	0	0	1.10128	0	150	150	0	0	
DE_150352783992435.3048062242874.225.8762492539823638	500	352783992435.3048062242874.225.8762492539823638	352783992435.3048062242874.225.8762492539823638	352783992435.3048062242874.225.8762492539823638	1.26954	0.6666671.03255	0	150	136	0	14	1.26954	0	150	136	0	14		
Source1501.114608981974169272.29504473742216505735818291039757	500	1.114608981974169272.29504473742216505735818291039757	1.114608981974169272.29504473742216505735818291039757	1.114608981974169272.29504473742216505735818291039757	0.882656	0	0.7488650	0	150	150	0	0	0.882656	0	150	150	0	0	
Plx1500.830658226065897767904344.79048602307859825751	500	0.830658226065897767904344.79048602307859825751	0.830658226065897767904344.79048602307859825751	0.830658226065897767904344.79048602307859825751	1.272	0.6666670.112874	0	149	149	0	1	1.272	0	149	149	0	1		
PM1504.01057258635.18593593392259.5320827289572537225	500	4.01057258635.18593593392259.5320827289572537225	4.01057258635.18593593392259.5320827289572537225	4.01057258635.18593593392259.5320827289572537225	1.3333329249616	0	1.3333329249616	0	149	150	0	0	1.3333329249616	0	149	150	0	0	
pmRA0 - - - 34.9519189597927.7267475 - 0.48225	500	0 - - - 34.9519189597927.7267475 - 0.48225	0 - - - 34.9519189597927.7267475 - 0.48225	0 - - - 34.9519189597927.7267475 - 0.48225	1.33333352.9004	0	1.33333352.9004	0	149	55	0	95	1.33333352.9004	0	149	55	0	95	
pmDE0 - - - 13.8367232878119.4245835 - - 32	500	0 - - - 13.8367232878119.4245835 - - 32	0 - - - 13.8367232878119.4245835 - - 32	0 - - - 13.8367232878119.4245835 - - 32	1.298763054	0	1.298763054	0	148	33	0	117	1.298763054	0	148	33	0	117	
Gmag15013.2437579774374248108447097737232561.47351978304	500	13.2437579774374248108447097737232561.47351978304	13.2437579774374248108447097737232561.47351978304	13.2437579774374248108447097737232561.47351978304	0.062833704	0	0.062833704	0	150	150	0	0	0.062833704	0	150	150	0	0	
e_Ch5000002880270876827304508659027008327600027002798753333.12367888	500	0002880270876827304508659027008327600027002798753333.12367888	0002880270876827304508659027008327600027002798753333.12367888	0002880270876827304508659027008327600027002798753333.12367888	0.0705	0	0.0705	0	69	150	0	0	0.0705	0	69	150	0	0	
BPmag15013.553878957629272604097985870830357618182507	500	13.553878957629272604097985870830357618182507	13.553878957629272604097985870830357618182507	13.553878957629272604097985870830357618182507	0.037651849	0	0.037651849	0	150	150	0	0	0.037651849	0	150	150	0	0	
e_BP500000380000082845092027028200270201000530230002900338873	500	0003800000082845092027028200270201000530230002900338873	0003800000082845092027028200270201000530230002900338873	0003800000082845092027028200270201000530230002900338873	5.531729063	0	5.531729063	0	131	150	0	0	5.531729063	0	131	150	0	0	
RPmag15012.78890789294899672390728204233039721980182383	500	12.78890789294899672390728204233039721980182383	12.78890789294899672390728204233039721980182383	12.78890789294899672390728204233039721980182383	0.0602651855	0	0.0602651855	0	150	150	0	0	0.0602651855	0	150	150	0	0	
e_RP50000044808008790951029828600372240018008008032871.3333345578855	500	000448080008790951029828600372240018008008032871.3333345578855	000448080008790951029828600372240018008008032871.3333345578855	000448080008790951029828600372240018008008032871.3333345578855	0.06	0	0.06	0	103	150	0	0	0.06	0	103	150	0	0	
GRVSmag1501261719015971570461679901349285328329741236626	500	1261719015971570461679901349285328329741236626	1261719015971570461679901349285328329741236626	1261719015971570461679901349285328329741236626	0.062476591	0	0.062476591	0	84	84	0	0	0.062476591	0	84	84	0	0	
e_GRV503691709700106078222220479422560404222228797505	500	3691709700106078222220479422560404222228797505	3691709700106078222220479422560404222228797505	3691709700106078222220479422560404222228797505	12.380220513893844	0	12.380220513893844	0	84	84	0	0	12.380220513893844	0	84	84	0	0	
BP-1500.7625062670.18664308939391.860448032607302953074	500	0.7625062670.18664308939391.860448032607302953074	0.7625062670.18664308939391.860448032607302953074	0.7625062670.18664308939391.860448032607302953074	0.6335780	0	0.6335780	0	150	148	0	2	0.6335780	0	150	148	0	2	
RP0.0959780.095978	500	0.0959780.095978	0.0959780.095978	0.0959780.095978	0.159386	0	0.159386	0	150	148	0	2	0.159386	0	150	148	0	2	
BP-1500.308729522.040120009404540.900927002506590327696	500	308729522.040120009404540.900927002506590327696	308729522.040120009404540.900927002506590327696	308729522.040120009404540.900927002506590327696	0.9912633990	0	0.9912633990	0	150	148	0	2	0.9912633990	0	150	148	0	2	
G0.0324670.032467	500	0.0324670.032467	0.0324670.032467	0.0324670.032467	0.41287	0	0.41287	0	150	148	0	2	0.41287	0	150	148	0	2	
G-1500.4588081148.0582319875990.980428072883801580098	500	4588081148.0582319875990.980428072883801580098	4588081148.0582319875990.980428072883801580098	4588081148.0582319875990.980428072883801580098	0.845494	0	0.845494	0	150	148	0	2	0.845494	0	150	148	0	2	
RP0.0635110.063511	500	0.0635110.063511	0.0635110.063511	0.0635110.063511	0.41287	0	0.41287	0	150	148	0	2	0.41287	0	150	148	0	2	
pscd1.54653473820162124035343826736.1273478953476062	500	1.54653473820162124035343826736.1273478953476062	1.54653473820162124035343826736.1273478953476062	1.54653473820162124035343826736.1273478953476062	0.845494	0	0.845494	0	150	148	0	2	0.845494	0	150	148	0	2	
Teff150972042188246.223914786358082644290178529481187528	500	972042188246.223914786358082644290178529481187528	972042188246.223914786358082644290178529481187528	972042188246.223914786358082644290178529481187528	1.3333333881037	0	1.3333333881037	0	148	150	0	0	1.3333333881037	0	148	150	0	0	

	Standard														Z-Score			Missing				Zero			Negative								
	De-	vi-	a-	Count	Mean	Median	Mode	Variance	StdDev	Range	Min	Max	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	liers	Out-	Out-Ra-	Val-	Missing	Unique	Val-	Val-	Val-	Count	Count	Count
Dist1	502	244.1806	200.9702	175	89435.268	9.5662	429803.48	163031.4	2.6666	6.7123	9.555	0	150	150	0	0																	
Rad1	502	740.4137	1115.3845	852	8271110.3942	1157802761373182	1.3333	3007.9802	9	0	150	150	0	0																			
Lum1	481	113.3638	182128	850.4326	95823275	6554696	602085.2071	1.3513	1.9564	287	1.3333	338	148	0	0																		
Flame																																	
Mask	292	472025	2.1063	7805825	5389.498	0.010929	9.8712	259.8	3	2.3253	1.2746	214	125	129	0	0																	
Flame																																	
Age	97	0.65587	0.483	0.8294	6937.201	238.500	398.587	0.8990	0	0.2143	673	35.3333	3	97	0	0																	
Flame																																	
z-	1480	546039	1220.0851	831030	6241270	82007420489	16222822	0.2703	106468	53	1.3333	338	148	0	0																		
Flame																																	

Non-Numeric Columns Statistics

	Count of Occurrences	Missing Ratio	Unique Values
('A', 0)	81	0	2
('B', 0)	69	0	2

```
[22]: def gen_missing_values_barplot(df, suffix, sorted=False):
    total_number_of_rows = len(df.index)
    total_missing_values = df.isnull().sum()
    percentage_missing = (total_missing_values / total_number_of_rows) * 100

    data = pd.DataFrame({
        'Columns': df.columns,
        'Missing Values %': percentage_missing
    })

    if sorted:
        data = data.sort_values(by='Missing Values %', ascending=True) # Sort_
        ascending for vertical bars

    plt.figure(figsize=(max(6, 0.45 * len(df.columns)), 5)) # Adjusted figsize_
        for vertical orientation

    # Plot for Missing Values % with vertical bars
    bars = sns.barplot(y='Missing Values %', x='Columns', data=data,
        color='orange')
    plt.title('Relative Percentage of Missing Values')
```

```

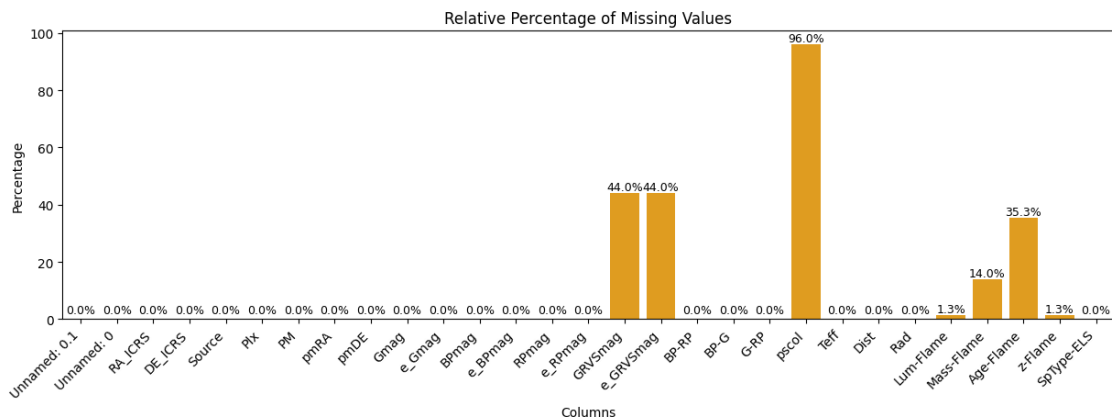
plt.ylabel('Percentage')
plt.xlabel('Columns')
plt.xticks(rotation=45, ha="right") # Rotate column names for better
↳readability

# Adjusting the position of percentage labels to avoid collision
for bar in bars.patches:
    height = bar.get_height()
    label_x_pos = bar.get_x() + bar.get_width() / 2
    label_y_pos = height*1 if height > 0 else 1
    plt.text(label_x_pos, label_y_pos, f'{height:.1f}%',
             ha='center', va='bottom', color='black', fontsize=9,
↳rotation=0)

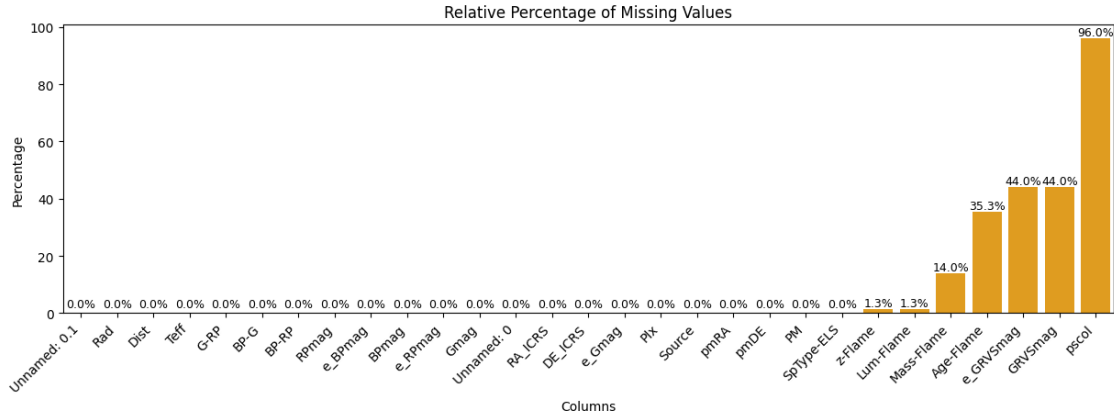
plt.tight_layout(rect=[0, 0, 0.95, 0.95]) # Adjust layout to fit everything
if sorted:
    plt.savefig(f'./output/exploration/missing_values_{suffix}.png',
↳bbox_inches='tight')
else:
    plt.savefig(f'./output/exploration/missing_values_sorted_{suffix}.png',
↳bbox_inches='tight')
plt.show()

```

[23]: `gen_missing_values_barplot(df=df, suffix='raw', sorted=False)`



[24]: `gen_missing_values_barplot(df=df, suffix='raw', sorted=True)`



8 Initial Data Exploration Insights

Following our preliminary examination of the dataset, we identified the following patterns regarding missing data:

- A substantial portion of data, **97.0%**, is absent from the **pscol** column, indicating a critical lack of information within this field.
- The **GRVSmag** column is missing **41.6%** of its data, highlighting a significant gap in our dataset.
- Similarly, **41.6%** of the data in the **e_GRVSmag** column is missing, paralleling the situation in the **GRVSmag** column.
- The **Age-Flame** column has **25.2%** missing data, suggesting a moderate amount of information is unavailable.
- A smaller, yet noteworthy, **8.1%** of data is missing from the **Mass-Flame** column.
- Only a minimal **2.4%** of data is missing in both the **z-Flame** and **Lum-Flame** columns, indicating these fields are largely complete.
- All other columns are fully populated with data, showing no gaps in information.

These observations provide a critical foundation for subsequent data cleaning and imputation strategies to ensure comprehensive data analysis.

9 Enhanced Approaches for Handling Missing Data

In addressing the challenge of missing data, we will undertake specific actions tailored to the unique characteristics of each column as outlined below:

Column	% Miss-Count	Recommended Action	Rationale
pscol	97.0%	Very High Drop column	The column is missing nearly all its data, rendering it impractical for any meaningful analysis or recovery.

Column	% Miss-Missing	Count	Recommended Action	Rationale
GRVSmag	41.6%	High	Impute with Median	Given the boxplot's significant value drop between 12 to 13, despite a steady increase from 6 to 11, imputing with the median is advised to maintain robustness. Additional domain knowledge is needed to assess normal distribution characteristics for precise imputation.
e_GRVSmag	31.6%	High	Impute with Mean	The error distribution primarily clusters around 0, then declines sharply with a long, flat tail, suggesting an exponential distribution. Averaging is preferred for imputation in this context.
Age-Flame	25.0%	Moderate	Impute with Median	With a notable drop in values between 0.3 to 0.7 and a continuous rise thereafter, median imputation is recommended to "fill the gap" and aim for a normal-like distribution, pending further domain insight.
Mass-Flame	8.1%	Low	Delete missing data rows	The relatively minor proportion of missing data allows for direct removal without significant impact on the dataset's integrity.
z-Flame	2.1%	Minimal	Delete missing data rows	Given the negligible amount of missing data, row deletion is a straightforward and minimally disruptive solution.
Lum-Flame	2.4%	Minimal	Delete missing data rows	Similar to z-Flame , the sparse missing data warrants simple row removal for data cleanliness without losing substantial information.

The above actions are proposed to strategically address missing data issues across various columns, ensuring that data integrity and analytical validity are maintained throughout the dataset processing phase.

9.1 Correlations

```
[25]: #plt.figure(figsize=(16, 16))
#sns.pairplot(df, hue='SpType-ELS', palette=PALETTE)
#plt.savefig('./output/exploration/features/pairplot_raw.pdf')
```

```
[26]: #pd.plotting.scatter_matrix(df, alpha=0.2)
#plt.savefig('./output/exploration/correlation-map.pdf')
```

```
[27]: # Select only numeric columns
df_only_numeric_columns = df.select_dtypes(include=['float64', 'int64'])
# Compute correlation matrix
```

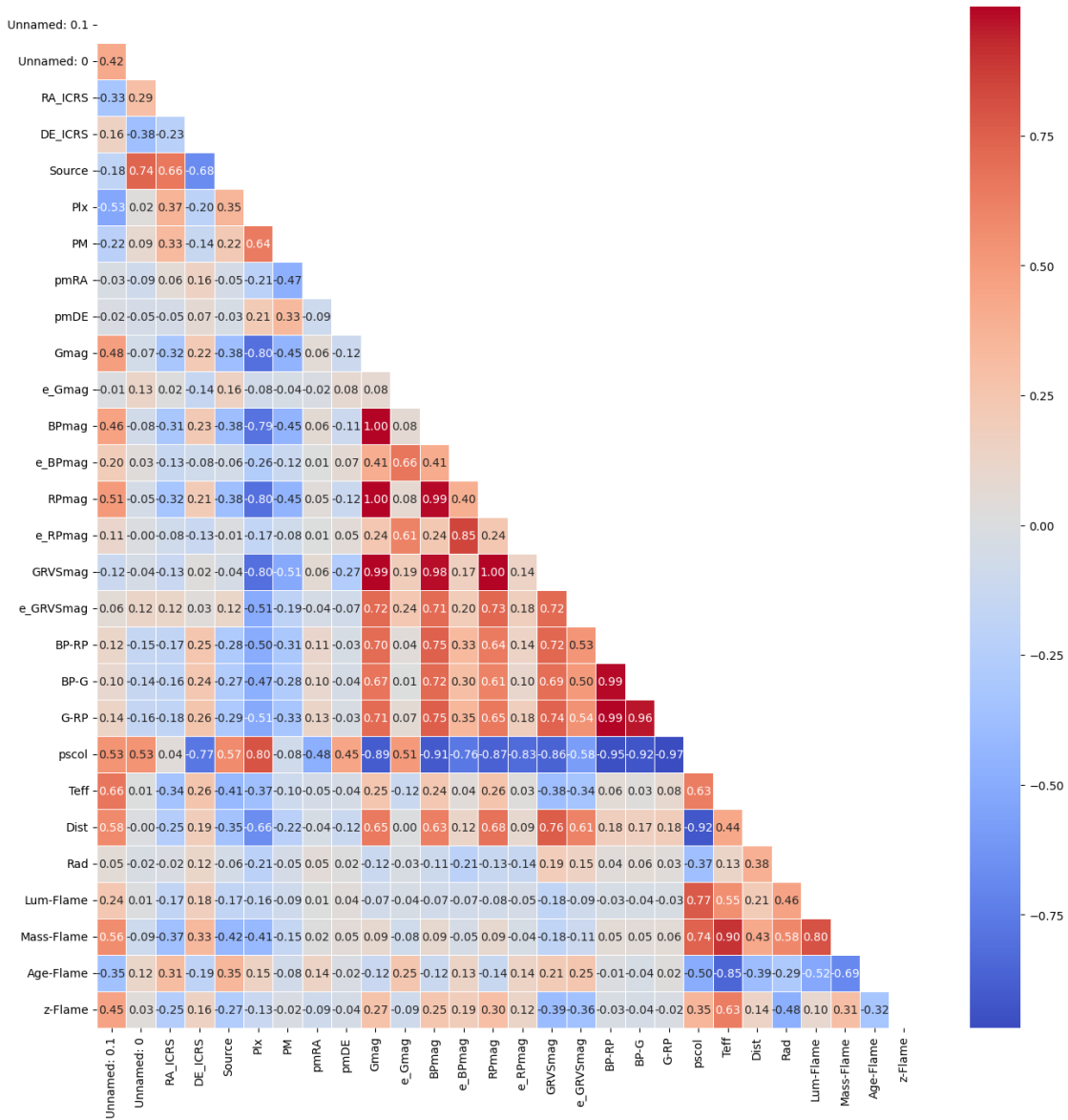
```

corr_matrix = df_only_numeric_columns.corr()

# Create a mask for the upper triangle
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Visualize correlation matrix
plt.figure(figsize=(16, 16))
ax = sns.heatmap(data=corr_matrix, annot=True, cmap=PALETTE, mask=mask, fmt=".
↪2f", linewidths=0.5)
plt.savefig('./output/exploration/features/correlation_heatmap_raw.pdf')

```



```
[28]: # Stack the correlation matrix to get pairs of features and their correlations
correlation_pairs = corr_matrix.stack().reset_index()
correlation_pairs.columns = ['Feature 1', 'Feature 2', 'Correlation']

# Sort by correlation values in descending order
sorted_correlation_pairs = correlation_pairs.sort_values(by='Correlation',
    ↪ascending=False)

print(sorted_correlation_pairs)
```

	Feature 1	Feature 2	Correlation
0	Unnamed: 0.1	Unnamed: 0.1	1.000000
406	e_RPmag	e_RPmag	1.000000
87	DE_ICRS	DE_ICRS	1.000000
116	Source	Source	1.000000
145	Plx	Plx	1.000000
..	...	...	...
524	BP-G	pscol	-0.923912
577	pscol	BP-RP	-0.952474
496	BP-RP	pscol	-0.952474
579	pscol	G-RP	-0.968597
552	G-RP	pscol	-0.968597

[784 rows x 3 columns]

```
[29]: # Filter correlation pairs above the threshold, excluding those involving the
    ↪same column and duplicates
high_correlation_pairs = sorted_correlation_pairs[
    (sorted_correlation_pairs['Correlation'] > CORRELATION_THRESHOLD) &
    (sorted_correlation_pairs['Feature 1'] < sorted_correlation_pairs['Feature_
    ↪2']) # Exclude duplicates
]

# Plotting the bar chart
plt.figure(figsize=(10, 6))
bars = plt.barh(high_correlation_pairs['Feature 1'] + ' - ' +
    ↪high_correlation_pairs['Feature 2'],
                high_correlation_pairs['Correlation'], color='skyblue')
plt.xlabel('Correlation')
plt.ylabel('Feature Pair')
plt.title(f'Correlated Feature Pairs with Correlation > {CORRELATION_THRESHOLD}
    ↪(excluding same feature and duplicates)')
plt.gca().invert_yaxis() # Invert y-axis for better readability

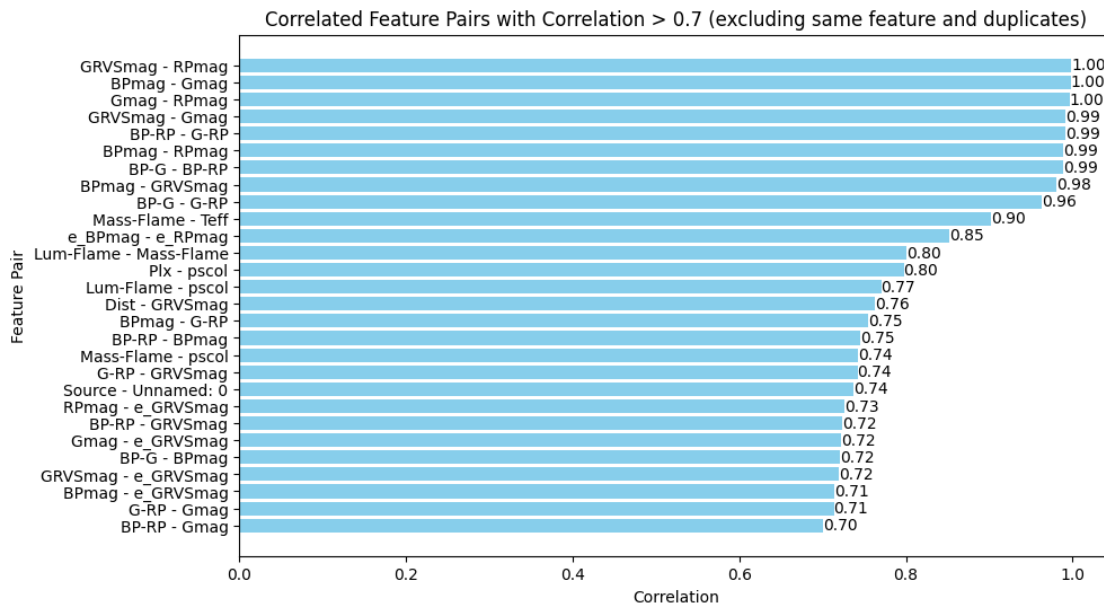
# Annotate bars with correlation values
for bar, correlation in zip(bars, high_correlation_pairs['Correlation']):
```

```

plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
        f'{correlation:.2f}',
        va='center', ha='left')

plt.show()
plt.savefig('./output/exploration/features/correlated_features_hierarchy_raw.
pdf')

```



<Figure size 640x480 with 0 Axes>

```

[30]: # This functions creates single files
def
    scatter_plot_between_attributes_based_on_corr_subplots(df_including_class_col,
    ignore_same_col_corr=True, corr_threshold=0.7, above_correlation=False):
        df = df_including_class_col.select_dtypes(include=['float64', 'int64'])

        unique_classes = df_including_class_col['SpType-ELS'].unique()
        colors = plt.cm.coolwarm(np.linspace(0, 1, len(unique_classes)))
        markers = ['o', 'x', 's', 'D', '^', 'v', '<', '>', 'p', '*', '+']

        class_colors = {class_name: color for class_name, color in
        zip(unique_classes, colors)}
        class_markers = {class_name: marker for class_name, marker in
        zip(unique_classes, markers[:len(unique_classes)])}

        for n_index, column in enumerate(df.columns):
            for m_index, column_iterator in enumerate(df.columns):

```

```

        if ignore_same_col_corr and column == column_iterator:
            continue

        correlation = df.iloc[:, n_index].corr(df.iloc[:, m_index])
        abs_corr = abs(correlation)
        corr_condition = abs_corr >= corr_threshold if above_correlation
    else abs_corr <= corr_threshold

    if corr_condition:
        x = df.iloc[:, n_index]
        y = df.iloc[:, m_index]
        class_labels = df_including_class_col['SpType-ELS']

        try:
            slope, intercept = np.polyfit(x, y, 1)
            line = slope * x + intercept
            plot_regression_line = True
        except np.linalg.LinAlgError:
            plot_regression_line = False

        plt.figure(figsize=(5, 5))

        for class_value in unique_classes:
            idx = df_including_class_col['SpType-ELS'] == class_value
            plt.scatter(x[idx], y[idx], c=[class_colors[class_value]],
    marker=class_markers[class_value], label=class_value)

            if plot_regression_line:
                plt.plot(x, line, color='black', linestyle='-.')

        plt.xlabel(column)
        plt.ylabel(column_iterator)
        plt.title(f'Feature {column} vs. Feature {column_iterator} -
    (Correlation: {correlation:.2f})')
        plt.legend()

        filename = f'{column}-{column_iterator}-scatter.pdf'.replace('/',
    ', -')

        corr_dir = 'corr_above' if above_correlation else 'corr_below'
        dir_path = f'./output/exploration/features/
    {corr_dir}_scatterplot_{corr_threshold}'

        os.makedirs(dir_path, exist_ok=True)
        plt.savefig(f'{dir_path}/{filename}')
        print(n_index, column, m_index, column_iterator)
        plt.close()

```

```

[31]: def scatter_plot_between_attributes_based_on_corr(df_including_class_col,
↳ ignore_same_col_corr=True, corr_threshold=0.7, above_correlation=False):
    df = df_including_class_col.select_dtypes(include=['float64', 'int64'])

    # Preparing class colors and markers
    unique_classes = df_including_class_col['SpType-ELS'].unique()
    colors = plt.cm.coolwarm(np.linspace(0, 1, len(unique_classes)))
    markers = ['o', 'x', 's', 'D', '^', 'v', '<', '>', 'p', '*', '+']
    class_colors = {class_name: color for class_name, color in
↳ zip(unique_classes, colors)}
    class_markers = {class_name: marker for class_name, marker in
↳ zip(unique_classes, markers[:len(unique_classes)])}

    # Calculate the total number of plots more explicitly
    total_plots = 0
    for n_index, column in enumerate(df.columns):
        for m_index, column_iterator in enumerate(df.columns):
            if ignore_same_col_corr and column == column_iterator:
                continue

            correlation = df.iloc[:, n_index].corr(df.iloc[:, m_index])
            abs_corr = abs(correlation)
            corr_condition = (abs_corr >= corr_threshold) if above_correlation
↳ else (abs_corr <= corr_threshold)

            if corr_condition:
                total_plots += 1

    # Determine grid size for subplots
    grid_size = math.ceil(math.sqrt(total_plots))

    # Create a single figure for all subplots
    fig, axs = plt.subplots(grid_size, grid_size, figsize=(5*grid_size,
↳ 5*grid_size))
    fig.subplots_adjust(hspace=0.4, wspace=0.4) # Adjust spacing between plots

    # Flatten the axes array for easy indexing
    axs = axs.flatten()
    plot_count = 0

    for n_index, column in enumerate(df.columns):
        for m_index, column_iterator in enumerate(df.columns):
            if ignore_same_col_corr and column == column_iterator:
                continue

            correlation = df.iloc[:, n_index].corr(df.iloc[:, m_index])
            abs_corr = abs(correlation)

```

```

        corr_condition = (abs_corr >= corr_threshold) if above_correlation_
↪else (abs_corr <= corr_threshold)

        if corr_condition:
            x = df.iloc[:, n_index]
            y = df.iloc[:, m_index]

            # Attempt to fit a linear regression line
            try:
                slope, intercept = np.polyfit(x, y, 1)
                line = slope * x + intercept
                plot_regression_line = True
            except np.linalg.LinAlgError:
                plot_regression_line = False

            # Plot in the next subplot position
            ax = axs[plot_count]
            for class_value in unique_classes:
                idx = df_including_class_col['SpType-ELS'] == class_value
                ax.scatter(x[idx], y[idx], c=[class_colors[class_value]],
↪marker=class_markers[class_value], label=class_value)

            if plot_regression_line:
                ax.plot(x, line, color='black', linestyle='-.')

            ax.set_xlabel(column)
            ax.set_ylabel(column_iterator)
            ax.set_title(f'{column} vs. {column_iterator} - (Correlation:
↪{correlation:.2f})')
            #if plot_count == 0: # Add legend only to the first plot to
↪save space
                #ax.legend()
            ax.legend()

            print(n_index, column, m_index, column_iterator)
            plot_count += 1

        # Hide any unused subplots
        for idx in range(plot_count, len(axs)):
            axs[idx].axis('off')

        # Save the entire figure with all subplots
        filename = f'./output/exploration/correlation-scatter.pdf'
        plt.savefig(filename)
        if above_correlation:

```

```

plt.title(f'Correlated Feature Pairs with an absolute Correlation value_
↳of > {corr_threshold}')
else:
plt.title(f'Correlated Feature Pairs with an absolute Correlation value_
↳of < {corr_threshold}')
plt.show()

```

```

[32]: scatter_plot_between_attributes_based_on_corr_subplots(df_including_class_col=df,
↳ignore_same_col_corr=True, corr_threshold=CORRELATION_THRESHOLD,
↳above_correlation=True)

```

```

1 Unnamed: 0 4 Source
3 DE_ICRS 20 pscol
4 Source 1 Unnamed: 0
5 Plx 9 Gmag
5 Plx 11 BPmag
5 Plx 13 RPmag
5 Plx 15 GRVSmag
5 Plx 20 pscol
9 Gmag 5 Plx
9 Gmag 11 BPmag
9 Gmag 13 RPmag
9 Gmag 15 GRVSmag
9 Gmag 16 e_GRVSmag
9 Gmag 17 BP-RP
9 Gmag 19 G-RP
9 Gmag 20 pscol
11 BPmag 5 Plx
11 BPmag 9 Gmag
11 BPmag 13 RPmag
11 BPmag 15 GRVSmag
11 BPmag 16 e_GRVSmag
11 BPmag 17 BP-RP
11 BPmag 18 BP-G
11 BPmag 19 G-RP
11 BPmag 20 pscol
12 e_BPmag 14 e_RPmag
12 e_BPmag 20 pscol
13 RPmag 5 Plx
13 RPmag 9 Gmag
13 RPmag 11 BPmag
13 RPmag 15 GRVSmag
13 RPmag 16 e_GRVSmag
13 RPmag 20 pscol
14 e_RPmag 12 e_BPmag
14 e_RPmag 20 pscol
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value

```



```

** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
15 GRVSmag 20 pscol
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
15 GRVSmag 22 Dist
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 9 Gmag
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 11 BPmag
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 13 Rpmag
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 15 GRVSmag
17 BP-RP 9 Gmag
17 BP-RP 11 BPmag
17 BP-RP 15 GRVSmag
17 BP-RP 18 BP-G
17 BP-RP 19 G-RP
17 BP-RP 20 pscol
18 BP-G 11 BPmag

```

```

18 BP-G 17 BP-RP
18 BP-G 19 G-RP
18 BP-G 20 pscol
19 G-RP 9 Gmag
19 G-RP 11 BPmag
19 G-RP 15 GRVSmag
19 G-RP 17 BP-RP
19 G-RP 18 BP-G
19 G-RP 20 pscol
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 3 DE_ICRS
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 5 Plx
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 9 Gmag
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 11 BPmag
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 12 e_BPmag
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value

```



```

** On entry to DLASCLS parameter number 4 had an illegal value
25 Mass-Flame 24 Lum-Flame
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
** On entry to DLASCLS parameter number 5 had an illegal value
** On entry to DLASCLS parameter number 4 had an illegal value
26 Age-Flame 21 Teff

```

```

[33]: scatter_plot_between_attributes_based_on_corr(df_including_class_col=df,
↳ ignore_same_col_corr=True, corr_threshold=CORRELATION_THRESHOLD,
↳ above_correlation=True)

```

```

1 Unnamed: 0 4 Source
3 DE_ICRS 20 pscol
4 Source 1 Unnamed: 0
5 Plx 9 Gmag
5 Plx 11 BPmag
5 Plx 13 RPmag
5 Plx 15 GRVSmag
5 Plx 20 pscol
9 Gmag 5 Plx
9 Gmag 11 BPmag
9 Gmag 13 RPmag
9 Gmag 15 GRVSmag
9 Gmag 16 e_GRVSmag
9 Gmag 17 BP-RP
9 Gmag 19 G-RP
9 Gmag 20 pscol
11 BPmag 5 Plx
11 BPmag 9 Gmag
11 BPmag 13 RPmag
11 BPmag 15 GRVSmag
11 BPmag 16 e_GRVSmag
11 BPmag 17 BP-RP
11 BPmag 18 BP-G
11 BPmag 19 G-RP
11 BPmag 20 pscol
12 e_BPmag 14 e_RPmag
12 e_BPmag 20 pscol
13 RPmag 5 Plx
13 RPmag 9 Gmag
13 RPmag 11 BPmag
13 RPmag 15 GRVSmag
13 RPmag 16 e_GRVSmag
13 RPmag 20 pscol
14 e_RPmag 12 e_BPmag

```



```

    ** On entry to DLASCLS parameter number 4 had an illegal value
15 GRVSmag 19 G-RP
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
15 GRVSmag 20 pscol
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
15 GRVSmag 22 Dist
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 9 Gmag
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 11 BPmag
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 13 Rpmag
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
    ** On entry to DLASCLS parameter number 5 had an illegal value
    ** On entry to DLASCLS parameter number 4 had an illegal value
16 e_GRVSmag 15 GRVSmag
17 BP-RP 9 Gmag
17 BP-RP 11 BPmag
17 BP-RP 15 GRVSmag
17 BP-RP 18 BP-G

```

```

17 BP-RP 19 G-RP
17 BP-RP 20 pscol
18 BP-G 11 BPmag
18 BP-G 17 BP-RP
18 BP-G 19 G-RP
18 BP-G 20 pscol
19 G-RP 9 Gmag
19 G-RP 11 BPmag
19 G-RP 15 GRVSmag
19 G-RP 17 BP-RP
19 G-RP 18 BP-G
19 G-RP 20 pscol
20 pscol 3 DE_ICRS
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 5 Plx
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 9 Gmag
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 11 BPmag
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
  ** On entry to DLASCLS parameter number 5 had an illegal value
  ** On entry to DLASCLS parameter number 4 had an illegal value
20 pscol 12 e_BPmag
  ** On entry to DLASCLS parameter number 4 had an illegal value

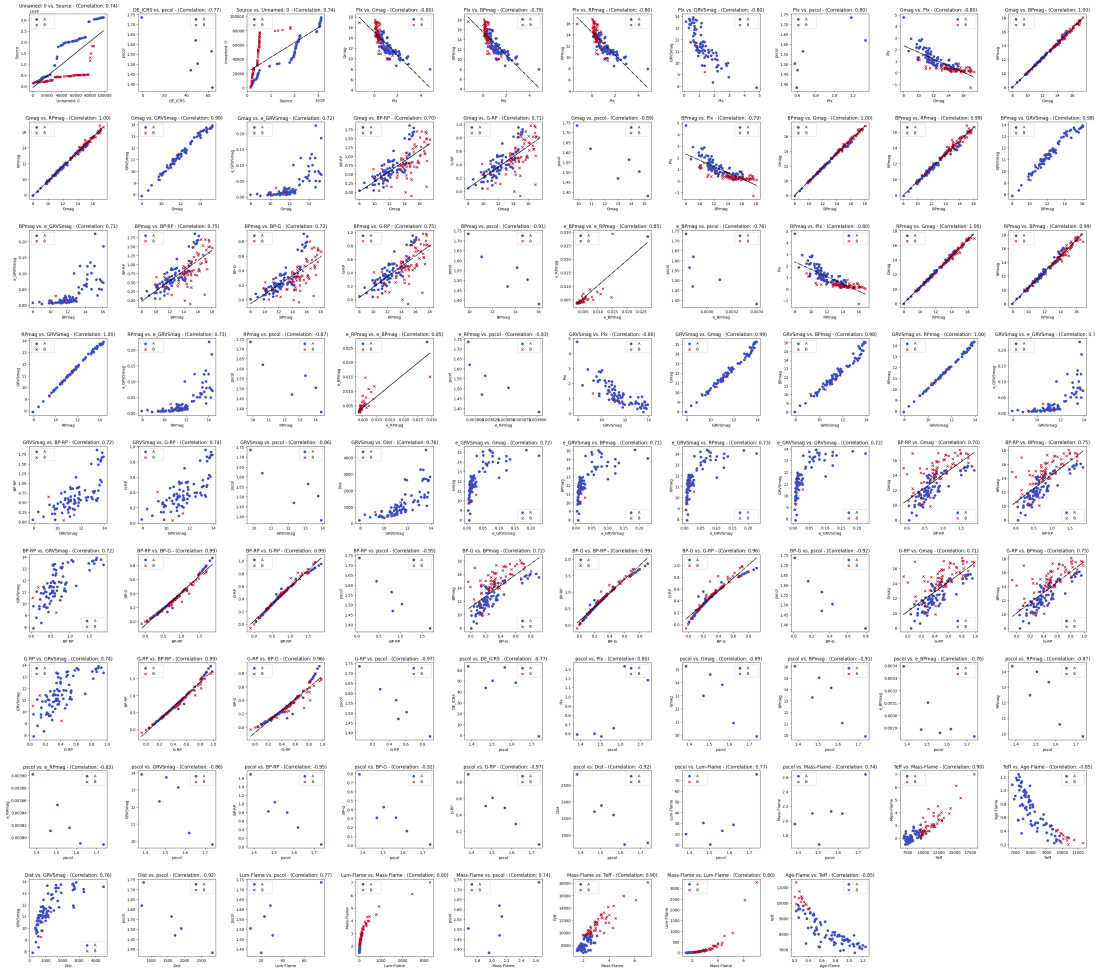
```


** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 5 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value

25 Mass-Flame 24 Lum-Flame

** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value
 ** On entry to DLASCLS parameter number 5 had an illegal value
 ** On entry to DLASCLS parameter number 4 had an illegal value

26 Age-Flame 21 Teff



Correlated Feature Pairs with an absolute Correlation value of > 0.7

```
[34]: #scatter_plot_between_attributes_based_on_corr(df_including_class_col = df,
      ↪ ignore_same_col_corr=True, corr_threshold=0.2, above_correlation=False) #
      ↪ takes ages
```

```
[35]: def
      ↪ distribution_plot_between_attributes_based_on_corr_subplots(df_including_class_col,
      ↪ ignore_same_col_corr=True, corr_threshold=0.7, above_correlation=False):
      df = df_including_class_col.select_dtypes(include=['float64', 'int64'])

      # Loop through each pair of columns to check for correlation
      for n_index, column in enumerate(df.columns):
          for m_index, column_iterator in enumerate(df.columns):
              if ignore_same_col_corr and column == column_iterator:
                  continue

              correlation = df[column].corr(df[column_iterator])
              abs_corr = abs(correlation)
              corr_condition = abs_corr >= corr_threshold if above_correlation
      ↪ else abs_corr <= corr_threshold

              if corr_condition:
                  # Prepare data with correct column names for plotting
                  data = pd.concat([df[column], df[column_iterator]],
      ↪ df_including_class_col['SpType-ELS']], axis=1)
                  data.columns = [column, column_iterator, 'Class']

                  # Create a figure to contain the custom layout
                  plt.figure(figsize=(10, 15))

                  # First subplot: regression plot
                  plt.subplot(3, 1, 1)
                  sns.regplot(data=data, x=column, y=column_iterator)
                  plt.xlabel(column) # Set x-axis label
                  plt.ylabel(column_iterator) # Set y-axis label
                  plt.title('Regression Plot')

                  # Second subplot: scatter plot with hue
                  plt.subplot(3, 1, 2)
                  sns.scatterplot(data=data, x=column, y=column_iterator,
      ↪ hue='Class')

                  plt.xlabel(column) # Set x-axis label
                  plt.ylabel(column_iterator) # Set y-axis label
                  plt.title('Scatter Plot with HUE')

                  # Third subplot: KDE plot
                  plt.subplot(3, 1, 3)
                  sns.kdeplot(data=data, x=column, y=column_iterator, fill=True)
```

```

plt.xlabel(column) # Set x-axis label
plt.ylabel(column_iterator) # Set y-axis label
plt.title('KDE Plot')

# Finalize and save the figure
plt.suptitle(f'Distribution Plots for {column} vs. {column_iterator} (Correlation: {correlation:.2f})')
plt.tight_layout()
plt.subplots_adjust(top=0.85) # Adjust top space to fit
suptitle

# Save figure
corr_dir = 'corr_above' if above_correlation else 'corr_below'
dir_path = f'./output/exploration/features/'
{corr_dir}_jointplot_{corr_threshold}'
os.makedirs(dir_path, exist_ok=True)
filename = f'{column}-{column_iterator}-joint.pdf'.replace('/', '-')

plt.savefig(f'{dir_path}/{filename}')
plt.close()

```

[36]: `distribution_plot_between_attributes_based_on_corr_subplots(df_including_class_col=df, ignore_same_col_corr=True, corr_threshold=CORRELATION_THRESHOLD, above_correlation=True)`

[37]: `def jointplots_between_attributes_based_on_corr_subplots(df_including_class_col, ignore_same_col_corr=True, corr_threshold=0.7, above_correlation=False):`

```

    df = df_including_class_col.select_dtypes(include=['float64', 'int64'])

    # Loop through each pair of columns
    for n_index, column in enumerate(df.columns):
        for m_index, column_iterator in enumerate(df.columns):
            if ignore_same_col_corr and column == column_iterator:
                continue

            correlation = df[column].corr(df[column_iterator])
            abs_corr = abs(correlation)
            corr_condition = abs_corr >= corr_threshold if above_correlation
            else abs_corr <= corr_threshold

            if corr_condition:
                # Data for jointplot
                data = pd.concat([df[column], df[column_iterator]],
df_including_class_col['SpType-ELS'], axis=1)
                data.columns = [column, column_iterator, 'SpType-ELS']

```

```

        # Directory setup
        corr_dir = 'corr_above' if above_correlation else 'corr_below'
        dir_path = f'./output/exploration/features/jointplots/
↳{corr_dir}_jointplot_{corr_threshold}'
        os.makedirs(dir_path, exist_ok=True)

        # Regression plot
        g_reg = sns.jointplot(data=data, x=column, y=column_iterator,
↳kind='reg')
        g_reg.fig.suptitle(f'Regression: {column} vs.
↳{column_iterator}')
        reg_filename = f'{column}-{column_iterator}-regression.png'.
↳replace('/', '-')
        g_reg.savefig(f'{dir_path}/{reg_filename}')
        plt.close(g_reg.fig)

        # Scatter plot with hue
        g_scatter = sns.jointplot(data=data, x=column,
↳y=column_iterator, hue='SpType-ELS')
        g_scatter.fig.suptitle(f'Scatter with Hue: {column} vs.
↳{column_iterator}')
        scatter_filename = f'{column}-{column_iterator}-scatter.png'.
↳replace('/', '-')
        g_scatter.savefig(f'{dir_path}/{scatter_filename}')
        plt.close(g_scatter.fig)

        # KDE plot
        g_kde = sns.jointplot(data=data, x=column, y=column_iterator,
↳kind='kde')
        g_kde.fig.suptitle(f'KDE: {column} vs. {column_iterator}')
        kde_filename = f'{column}-{column_iterator}-kde.png'.replace('/',
↳', '-')
        g_kde.savefig(f'{dir_path}/{kde_filename}')
        plt.close(g_kde.fig)

```

```

[38]: jointplots_between_attributes_based_on_corr_subplots(df_including_class_col=df,
↳ignore_same_col_corr=True, corr_threshold=CORRELATION_THRESHOLD,
↳above_correlation=True)

```

KeyboardInterrupt

Traceback (most recent call last)

Cell In[38], line 1

----> 1

↳jointplots_between_attributes_based_on_corr_subplots(df_including_class_col=df, ignore_sam

```

Cell In[37], line 32, in
    ↪ jointplots_between_attributes_based_on_corr_subplots(df_including_class_col,
    ↪ ignore_same_col_corr, corr_threshold, above_correlation)
        29 plt.close(g_reg.fig)
        31 # Scatter plot with hue
--> 32 g_scatter =
    ↪ sns.jointplot(data=data, x=column, y=column_iterator, hue='SpType-ELS')
        33 g_scatter.fig.suptitle(f'Scatter with Hue: {column} vs.
    ↪ {column_iterator}')
        34 scatter_filename = f'{column}-{column_iterator}-scatter.png'.replace('/',
    ↪ ', '-')

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
    ↪ site-packages/seaborn/axisgrid.py:2241, in jointplot(data, x, y, hue, kind,
    ↪ height, ratio, space, dropna, xlim, ylim, color, palette, hue_order, hue_norm,
    ↪ marginal_ticks, joint_kws, marginal_kws, **kwargs)
        2238 dropna = True
        2240 # Initialize the JointGrid object
-> 2241 grid = JointGrid(
        2242     data=data, x=x, y=y, hue=hue,
        2243     palette=palette, hue_order=hue_order, hue_norm=hue_norm,
        2244     dropna=dropna, height=height, ratio=ratio, space=space,
        2245     xlim=xlim, ylim=ylim, marginal_ticks=marginal_ticks,
        2246 )
        2248 if grid.hue is not None:
        2249     marginal_kws.setdefault("legend", False)

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
    ↪ site-packages/seaborn/axisgrid.py:1703, in JointGrid.__init__(self, data, x,
    ↪ y, hue, height, ratio, space, palette, hue_order, hue_norm, dropna, xlim,
    ↪ ylim, marginal_ticks)
        1700 self.ax_marg_y = ax_marg_y
        1702 # Turn off tick visibility for the measure axis on the marginal plots
-> 1703 plt.setp(ax_marg_x.get_xticklabels(), visible=False)
        1704 plt.setp(ax_marg_y.get_yticklabels(), visible=False)
        1705 plt.setp(ax_marg_x.get_xticklabels(minor=True), visible=False)

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
    ↪ site-packages/matplotlib/axes/_base.py:74, in _axis_method_wrapper.
    ↪ __set_name__.<locals>.wrapper(self, *args, **kwargs)
        73 def wrapper(self, *args, **kwargs):
--> 74     return get_method(self)(*args, **kwargs)

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
    ↪ site-packages/matplotlib/axis.py:1468, in Axis.get_ticklabels(self, minor,
    ↪ which)
        1466 if minor:
        1467     return self.get_minorticklabels()
-> 1468 return self.get_majorticklabels()

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/axis.py:1425, in Axis.get_majorticklabels(self)
    1423 def get_majorticklabels(self):
    1424     """Return this Axis' major tick labels, as a list of `~.text.Text`.
    ↳"""
-> 1425     self._update_ticks()
    1426     ticks = self.get_major_ticks()
    1427     labels1 = [tick.label1 for tick in ticks if tick.label1.
↳get_visible()]

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/axis.py:1282, in Axis._update_ticks(self)
    1280     tick.set_label1(label)
    1281     tick.set_label2(label)
-> 1282 minor_locs = self.get_minorticklocs()
    1283 minor_labels = self.minor.formatter.format_ticks(minor_locs)
    1284 minor_ticks = self.get_minor_ticks(len(minor_locs))

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/axis.py:1503, in Axis.get_minorticklocs(self)
    1501 minor_locs = np.asarray(self.minor.locator())
    1502 if self.remove_overlapping_locs:
-> 1503     major_locs = self.major.locator()
    1504     transform = self._scale.get_transform()
    1505     tr_minor_locs = transform.transform(minor_locs)

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/ticker.py:2136, in MaxNLocator.__call__(self)
    2134 def __call__(self):
    2135     vmin, vmax = self.axis.get_view_interval()
-> 2136     return self.tick_values(vmin, vmax)

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/ticker.py:2144, in MaxNLocator.tick_values(self,
↳vmin, vmax)
    2141     vmin = -vmax
    2142     vmin, vmax = mtransforms.nonsingular(
    2143         vmin, vmax, expander=1e-13, tiny=1e-14)
-> 2144     locs = self._raw_ticks(vmin, vmax)
    2146     prune = self._prune
    2147     if prune == 'lower':

```

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11 /
↳site-packages/matplotlib/ticker.py:2083, in MaxNLocator._raw_ticks(self, vmin
↳vmax)
    2081 if self._nbins == 'auto':
    2082     if self.axis is not None:

```

```

-> 2083         nbins = np.clip(self.axis.get_tick_space(),
2084                             max(1, self._min_n_ticks - 1), 9)
2085     else:
2086         nbins = 9

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11/site-packages/numpy/core/fromnumeric.py:2169, in clip(a, a_min, a_max, out, **kwargs)

```

2100 @array_function_dispatch(_clip_dispatcher)
2101 def clip(a, a_min, a_max, out=None, **kwargs):
2102     """
2103     Clip (limit) the values in an array.
2104     (...)
2167
2168     """
-> 2169     return _wrapfunc(a, 'clip', a_min, a_max, out=out, **kwargs)

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11/site-packages/numpy/core/fromnumeric.py:56, in _wrapfunc(obj, method, *args, **kwds)

```

54 bound = getattr(obj, method, None)
55 if bound is None:
---> 56     return _wrapit(obj, method, *args, **kwds)
57 try:
58     return bound(*args, **kwds)

```

File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11/site-packages/numpy/core/fromnumeric.py:45, in _wrapit(obj, method, *args, **kwds)

```

43 except AttributeError:
44     wrap = None
---> 45 result = getattr(asarray(obj), method)(*args, **kwds)
46 if wrap:
47     if not isinstance(result, mu.ndarray):

```

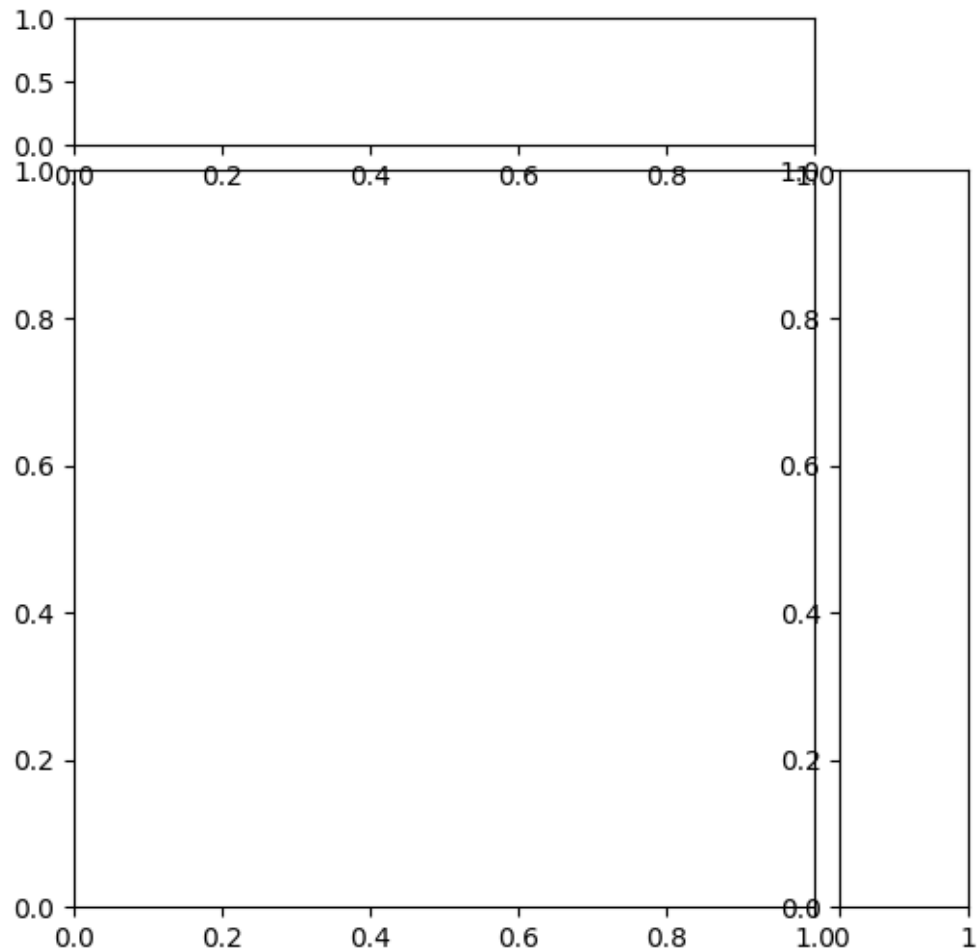
File ~/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11/site-packages/numpy/core/_methods.py:99, in _clip(a, min, max, out, **kwargs)

```

97     return um.maximum(a, min, out=out, **kwargs)
98 else:
---> 99     return um.clip(a, min, max, out=out, **kwargs)

```

KeyboardInterrupt:



```
[ ]: df.to_pickle("../output/pk/raw_data.pkl")
```

10 Handle missing data / values

```
[ ]: # DEVELOPMENT ONLY
#df = pd.read_csv(filepath_or_buffer="../data/32130_AT2_25061944.csv")
#df = df.sample(frac=SAMPLE_RATIO, random_state=SEED)
```

```
[ ]: df_orig = df.copy(deep=True) # deep copy
# orig will be converted to Excel file at the end, works as protocol
# original dataframe will be prepared to actual doing data analysis
```

```
[ ]: # Drop pscol since most of the data is missing
# This will be done later
# df.drop(['pscol'], axis=1, inplace=True)
```

```
[ ]: def impute_values(column, df, df_orig, method='median'):
    # Fill missing values in column with values
    mask = df[column].isnull()
    if method == 'median':
        fill_value = df[column].median()
    elif method == 'mean':
        fill_value = df[column].mean()
    else:
        raise ValueError("Invalid method. Please choose 'median' or 'mean'.")

    df[column] = df[column].fillna(fill_value)

    filled_values = df[column].fillna(df['GRVSmag'].median())
    df_orig.loc[mask, f'{column}_imputed_median'] = filled_values[mask] #
    ↪copies only imputed values
    return df, df_orig
```

```
[ ]: df, df_orig = impute_values('GRVSmag', df, df_orig, 'median')
```

```
[ ]: df, df_orig = impute_values('e_GRVSmag', df, df_orig, 'median')
```

```
[ ]: df, df_orig = impute_values('Age-Flame', df, df_orig, 'median')
```

```
[ ]: def delete_rows(column, df, df_orig):
    # Delete rows containing nan-value in column
    print(f'Number of rows before dropping: {df.shape[0]}')
    deleted_rows_mask = df[column].isnull() # Mask to identify rows to be
    ↪deleted
    deleted_rows_index = df[deleted_rows_mask].index # Index of rows to be
    ↪deleted

    df = df.dropna(subset=[column]) # Dropping rows with NaN in column
    print(f'Number of rows after dropping: {df.shape[0]}')

    # Mark the corresponding rows in df_orig where values were deleted
    df_orig[f'{column}_deleted_rows'] = '' # New column and initialize with
    ↪empty string
    df_orig.loc[deleted_rows_index, f'{column}_deleted_rows'] = f'DELETED ROW
    ↪due to missing {column} value'
    return df, df_orig
```

```
[ ]: df, df_orig = delete_rows('Mass-Flame', df, df_orig)
```

Number of rows before dropping: 3000

Number of rows after dropping: 2756

```
[ ]: df, df_orig = delete_rows('z-Flame', df, df_orig)
```

Number of rows before dropping: 2756
Number of rows after dropping: 2756

```
[ ]: df, df_orig = delete_rows('Lum-Flame', df, df_orig)
```

Number of rows before dropping: 2756
Number of rows after dropping: 2756

```
[ ]: df.to_pickle("./output/pk/cleaned_missing_data.pkl")
df_orig.to_pickle("./output/pk/cleaned_missing_data_protocol.pkl")
df_orig.to_excel("./output/excel/cleaned_missing_data_protocol.xlsx")
```

11 1B. Data preprocessing

Perform each of the following data preparation tasks (each task applies to the original data) using your choice of tool:

11.1 Task 1

Use the following binning techniques to smooth the values of the following two attributes: - RA_ICRS - DE_ICRS For each attribute, you must apply:

Equi-width binning Equi-depth binning In the assignment report, for each of these techniques, you need to illustrate your steps. In your Excel workbook file place the results in separate columns in the corresponding spreadsheet. Use your judgement in choosing the appropriate number of bins - and justify this in the report.

```
[ ]: # Equi-width binning
def equi_width_binning(series, n_bins):
    min_value, max_value = series.min(), series.max()
    bin_edges = np.linspace(min_value, max_value, n_bins + 1)

    # Use pd.cut to handle the binning based on calculated edges
    bins = pd.cut(series, bins=bin_edges, labels=False, include_lowest=True)

    return bins
```

```
[ ]: # Equi-depth binning
def equi_depth_binning(series, n_bins):
    sorted_series = series.sort_values()
    values_per_bin = len(series) // n_bins

    bins = pd.Series(index=series.index, dtype=int)
    for i, (index, value) in enumerate(sorted_series.items()):
        bins.at[index] = i // values_per_bin

    bins = bins.clip(upper=n_bins-1)

    return bins
```

```
[ ]: # Define number of bins based on common best-practices; might be adjusted later
n_bins = round(math.sqrt(df.shape[0]))
```

```
[ ]: # Equi-width binning on RA_ICRS
df[f'RA_ICRS_equip_width_{n_bins}'] = equip_width_binning(df['RA_ICRS'],
    ↪n_bins=n_bins)
df_orig[f'RA_ICRS_added_equip_width_{n_bins}'] =
    ↪equip_width_binning(df['RA_ICRS'], n_bins=n_bins)
```

```
[ ]: # Equi-depth binning on RA_ICRS
df[f'RA_ICRS_equip_depth_{n_bins}'] = equip_depth_binning(df['RA_ICRS'],
    ↪n_bins=n_bins)
df_orig[f'RA_ICRS_added_depth_width_{n_bins}'] =
    ↪equip_depth_binning(df['RA_ICRS'], n_bins=n_bins)
```

```
[ ]: # Equi-width binning on DE_ICRS
df[f'DE_ICRS_equip_width_{n_bins}'] = equip_width_binning(df['DE_ICRS'],
    ↪n_bins=n_bins)
df_orig[f'DE_ICRS_added_equip_width_{n_bins}'] =
    ↪equip_width_binning(df['DE_ICRS'], n_bins=n_bins)
```

```
[ ]: # Equi-depth binning on DE_ICRS
df[f'DE_ICRS_equip_depth_{n_bins}'] = equip_depth_binning(df['DE_ICRS'],
    ↪n_bins=n_bins)
df_orig[f'DE_ICRS_added_depth_width_{n_bins}'] =
    ↪equip_depth_binning(df['DE_ICRS'], n_bins=n_bins)
```

11.2 Task 2

Use the following techniques to normalise the following attribute: - Age-Flame For this attribute, you must apply:

min-max normalization to transform the values onto the range [0.0-1.0]. z-score normalization to transform the values. The assignment report provides an explanation of each of the applied techniques. In your Excel workbook file place the results in separate columns in the corresponding spreadsheet.

```
[ ]: def scale_column(column, df, df_orig, method='minmax'):
    """
    Scales a column in 'df' using the specified 'method' and appends the scaled
    ↪values to 'df_orig'.

    Parameters:
    - column: the column name in 'df' to be scaled.
    - df: the original DataFrame containing the column to be scaled.
    - df_orig: the DataFrame to which the scaled values will be appended.
    - method: the scaling method to use ('minmax', 'robust', 'softmax', or
    ↪'standard').
```

```

Returns:
- df: the DataFrame with the scaled column added.
- df_orig: the DataFrame with the scaled column values appended.
"""

# Select the scaler based on the 'method' parameter
if method == 'minmax':
    scaler = MinMaxScaler()
elif method == 'robust':
    scaler = RobustScaler()
elif method == 'standard':
    scaler = StandardScaler()
else:
    raise ValueError(f"Unsupported scaling method ({method}). Choose
↳ 'minmax', 'robust', or 'standard'.")

# Reshape the data to (-1, 1) because it's a single feature
values = df[column].values.reshape(-1, 1)

# Apply the selected scaler
if method == 'softmax':
    # Custom softmax scaling
    exp_values = np.exp(values - np.max(values))
    scaled_values = exp_values / np.sum(exp_values)
else:
    # Scikit-learn scaling
    scaled_values = scaler.fit_transform(values)

# Assign scaled values back to df with a new column name
df[f'{column}_scaled_{method}'] = scaled_values.flatten()

# Create a temporary Series for the scaled values with the same index as df
scaled_series = pd.Series(scaled_values.flatten(), index=df.index)

# Mask to identify rows with non-null values that were scaled
mask = ~df[column].isnull()

# Assign scaled values to df_orig based on the mask
df_orig.loc[df.index[mask], f'{column}_applied_scaled_{method}'] =
↳ scaled_series[mask].values

return df, df_orig

```

```
[ ]: df, df_orig = scale_column('Age-Flame', df, df_orig, 'minmax')
```

```
[ ]: df, df_orig = scale_column('Age-Flame', df, df_orig, 'standard')
```

11.3 Task 3

Discretise the Mass-Flame attribute into the following categories: Small (0 - 2) Medium (2 — 4) Large (4 — 8) Provide the frequency of each category in your dataset. Your assignment report should provide an explanation of each of the applied techniques. In your Excel workbook file place the results in a separate column in the corresponding spreadsheet.

```
[ ]: def discretize_column(column, df, df_orig, bins, categories):  
    """  
    Discretizes the specified column into categories based on bins, encodes the  
    categories numerically,  
    and appends the numerically encoded values to another DataFrame.  
  
    Parameters:  
    - column: the column name in 'df' to be discretized and encoded.  
    - df: the DataFrame containing the column to be processed.  
    - df_orig: the DataFrame to which the numerically encoded values will be  
    appended.  
    - bins: the bins used for discretization.  
    - categories: the labels for the discretized categories.  
  
    Returns:  
    - df: the DataFrame with the added categorized and numerically encoded  
    columns.  
    - df_orig: the DataFrame with the numerically encoded column values  
    appended.  
    """  
  
    # Discretize the column based on the provided bins and categories  
    categorized_column_name = f'{column}_categorized'  
    df[categorized_column_name] = pd.cut(df[column], bins=bins,  
    labels=categories, right=False)  
  
    # Initialize the LabelEncoder  
    label_encoder = LabelEncoder()  
  
    # Fit and transform the categories to numeric values  
    numeric_column_name = f'{column}_numeric'  
    df[numeric_column_name] = label_encoder.  
    fit_transform(df[categorized_column_name])  
  
    # Mask to identify rows with non-null values in the categorized column  
    mask = ~df[categorized_column_name].isnull()  
  
    # Append the numerically encoded values to df_orig based on the mask  
    df_orig.loc[df.index[mask], f'{column}_applied_category'] = df.loc[mask,  
    categorized_column_name].values
```

```

df_orig.loc[df.index[mask], f'{column}_applied_numeric'] = df.loc[mask,
↳numeric_column_name].values

return df, df_orig

```

```

[ ]: # Introduce categories and the corresponding quantities
mass_flame_categories = ['Small', 'Medium', 'Large']
mass_flame_bins = [0, 2, 4, 8]

df, df_orig = discretize_column('Mass-Flame', df, df_orig, mass_flame_bins,
↳mass_flame_categories)

```

```

[ ]: # Display the frequency of the categories
frequencies = df['Mass-Flame_categorized'].value_counts()
frequencies

```

```

[ ]: Mass-Flame_categorized
Medium    1659
Small      956
Large      141
Name: count, dtype: int64

```

11.4 Task 4

Binarise the SpType-ELS variable [with values “0” or “1”]. Your assignment report should provide an explanation of the applied binarisation technique. In your Excel workbook file place the results in separate columns in the corresponding spreadsheet.

```

[ ]: # Trim trailing spaces in 'SpType-ELS' column
df['SpType-ELS'] = df['SpType-ELS'].str.strip()

```

```

[ ]: spytype_els_classes = df['SpType-ELS'].unique()
spytype_els_classes[1]

```

```

[ ]: 'A'

```

```

[ ]: def binarize_column(column, df, df_orig, positive_class):
    """
    Binarizes a specified column based on a positive class or classes and
    ↳appends
    the binarized values to another DataFrame. It treats occurrences of the
    ↳positive
    class(es) as 1 and all others as 0.

    Parameters:
    - column: the column name in 'df' to be binarized.
    - df: the DataFrame containing the column to be processed.
    - df_orig: the DataFrame to which the binarized values will be appended.
    """

```

```

- positive_class: a list of unique values to be treated as the positive_
↳class (1).

Returns:
- df: the DataFrame with the added binarized column.
- df_orig: the DataFrame with the binarized column values appended.
"""

# Directly create a binarized column based on whether the value is in_
↳positive_classes
df[f'{column}_binarized'] = df[column].apply(lambda x: 1 if x in_
↳positive_classes else 0)

# Mask to identify rows with non-null values in the original column
mask = ~df[column].isnull()

# Append the binarized values to df_orig based on the mask
df_orig.loc[df.index[mask], f'{column}_applied_binarized'] = df.loc[mask,
↳f'{column}_binarized'].values

return df, df_orig

```

```

[ ]: positive_classes = spytype_els_classes[0] # define A as positive class
df, df_orig = binarize_column('SpType-ELS', df, df_orig, positive_classes)

```

12 Further Preprocessing

12.1 Index

```

[ ]: # THIS DOES MESS UP THE PROTOCOL DF, if we need to do it, do it at the very end!
# Sort by Source
# df = df.sort_values(by=['Source'])

```

```

[ ]: # Apply insert new index column
# df.insert(len(df.columns), 'Source cleaned', range(0, 0 + len(df)))
# df = df.reset_index(drop=True)
# df

```

12.2 Unnamed column wrangling

```

[ ]: # Check Unnamed: 0.1 and Unnamed: 0
# Action 1: compare and reduce to one column since we have duplicate entries_
↳with some prefixes in Unnamed: 0.1 col

def normalize_value(value1, value2):
    # Convert values to strings to handle them uniformly
    str_value1, str_value2 = str(value1), str(value2)

```

```

# Remove leading numbers if they make the rest of the string identical
for i in range(0, min(len(str_value1), len(str_value2))):
    if str_value1.endswith(str_value2[i:]):
        return int(str_value2[i:])
    if str_value2.endswith(str_value1[i:]):
        return int(str_value1[i:])

# Default case: if both values are identical or no prefix logic is found
return int(str_value1) if len(str_value1) < len(str_value2) else
↪int(str_value2)

```

```

[ ]: # Apply the normalization to each row and create a new column with cleaned
↪values
df['Unnamed: 0 cleaned'] = df.apply(lambda row: normalize_value(row['Unnamed: 0.
↪1'], row['Unnamed: 0']), axis=1)
df

```

```

[ ]:      Unnamed: 0.1  Unnamed: 0    RA_ICRS    DE_ICRS      Source \
0          143167      43167  56.507386  57.312075  448865558939244928
1          113941      13941  69.700204  43.031842  204328228682288896
3          152440      52440  43.015008  62.001698  466496189234060544
4           33765      33765  216.635374  29.119972  1283977178899988096
5          139337      39337   4.221593  63.110068  431387064971676288
...
2995         107931         7931  82.391573  40.720309  193917876501880576
2996         10156        10156  68.233401  58.437177  278632532995302912
2997         114166        14166  71.352793  43.494834  204586648276738304
2998          27391        27391  21.729208  84.735422  573698916534023168
2999          28219        28219  124.608943  22.220359  676498997419535360

```

```

      Plx      PM    pmRA    pmDE      Gmag ... RA_ICRS_equip_width_52 \
0    0.1799  0.296 -0.256  0.149  16.675682 ...                      8
1    0.1760  0.400  0.150  0.370  15.894940 ...                      10
3    0.2920  0.865 -0.864 -0.014  15.164660 ...                      6
4    2.3660  28.159 -24.834  13.273   9.990480 ...                     31
5    0.1891  1.905 -1.824 -0.547  14.590605 ...                      0
...
2995  0.2806  0.851 -0.114 -0.844  13.510389 ...                      11
2996  1.0242  5.222  0.500 -5.198  12.453662 ...                      9
2997  0.4130  2.260  0.294 -2.240  15.135862 ...                     10
2998  2.8879  3.882 -3.879 -0.159   9.268933 ...                      3
2999  4.9570  35.504 -31.952 -15.478   8.620995 ...                     18

```

```

      RA_ICRS_equip_depth_52  DE_ICRS_equip_width_52  DE_ICRS_equip_depth_52 \
0                        17.0                        38                        36.0
1                        21.0                        32                        21.0

```

3	14.0	40	43.0
4	37.0	26	11.0
5	1.0	41	45.0
...	...	...	...
2995	26.0	31	19.0
2996	20.0	39	38.0
2997	22.0	32	21.0
2998	6.0	50	51.0
2999	36.0	24	10.0

	Age-Flame_scaled_minmax	Age-Flame_scaled_standard \
0	0.092942	-0.742142
1	0.165507	-0.267789
3	0.038767	-1.096283
4	0.334493	0.836870
5	0.165507	-0.267789
...	...	...
2995	0.165507	-0.267789
2996	0.262425	0.365765
2997	0.003976	-1.323712
2998	0.058648	-0.966323
2999	0.508449	1.974018

	Mass-Flame_categorized	Mass-Flame_numeric	SpType-ELS_binarized \
0	Medium	1	1
1	Medium	1	1
3	Medium	1	1
4	Small	2	0
5	Large	0	1
...	...	...	...
2995	Medium	1	1
2996	Small	2	0
2997	Medium	1	1
2998	Medium	1	0
2999	Small	2	0

	Unnamed: 0 cleaned
0	43167
1	13941
3	52440
4	33765
5	39337
...	...
2995	7931
2996	10156
2997	14166
2998	27391

2999

28219

[2756 rows x 39 columns]

13 Drop columns

```
[ ]: df.columns
```

```
[ ]: Index(['Unnamed: 0.1', 'Unnamed: 0', 'RA_ICRS', 'DE_ICRS', 'Source', 'Plx',
          'PM', 'pmRA', 'pmDE', 'Gmag', 'e_Gmag', 'BPmag', 'e_BPmag', 'RPmag',
          'e_RPmag', 'GRVSmag', 'e_GRVSmag', 'BP-RP', 'BP-G', 'G-RP', 'pscol',
          'Teff', 'Dist', 'Rad', 'Lum-Flame', 'Mass-Flame', 'Age-Flame',
          'z-Flame', 'SpType-ELS', 'RA_ICRS_equip_width_52',
          'RA_ICRS_equip_depth_52', 'DE_ICRS_equip_width_52',
          'DE_ICRS_equip_depth_52', 'Age-Flame_scaled_minmax',
          'Age-Flame_scaled_standard', 'Mass-Flame_categorized',
          'Mass-Flame_numeric', 'SpType-ELS_binarized', 'Unnamed: 0 cleaned'],
          dtype='object')
```

```
[ ]: drop_reasons = [
    {'pscol': 'More than 97% of this column is missing.'},
    {'Unnamed: 0.1': 'No information about the content of this column;
    ↪probably the result of a loading and saving the data in pandas.'},
    {'Unnamed: 0': 'No information about the content of this column; probably
    ↪the result of a loading and saving the data in pandas.'},
    {'Unnamed: 0 cleaned': 'No information about the content of this column;
    ↪probably the result of a loading and saving the data in pandas.'},
    {'Source': 'Similar to an ID, which does not provide additional
    ↪information to the actual data.'},
    {'Source cleaned': 'Similar to an ID, which does not provide additional
    ↪information to the actual data.'},
    {'RA_ICRS': 'Original row, that was preprocessed using binning.'},
    {'RA_ICRS_equip_depth_{n_bins}': 'Row that was preprocessed using binning,
    ↪does not add more information to the data.'},
    {'DE_ICRS': 'Original row, that was preprocessed using binning.'},
    {'DE_ICRS_equip_depth_{n_bins}': 'Row that was preprocessed using binning,
    ↪does not add more information to the data.'},
    {'Age-Flame': 'Original row, that was preprocessed using minmax
    ↪standardization.'},
    {'Age-Flame_scaled_minmax': 'Row that was preprocessed using minmax
    ↪standardization, does not add more information to the data.'},
    {'Mass-Flame': 'Original row, that was preprocessed using discretization.
    ↪'},
    {'Mass-Flame_categorized': 'Row that was preprocessed using discretization,
    ↪does not add more information to the data.'},
    {'SpType-ELS': 'Original row, that was preprocessed using binarization.'},
```

```
]
```

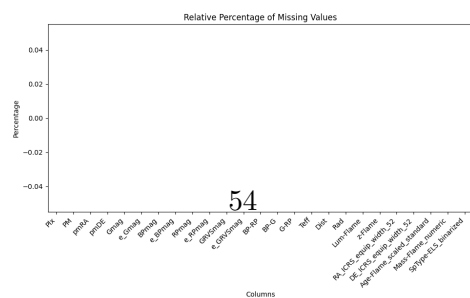
```
[ ]: for drop_reason in drop_reasons:
    for col, reason in drop_reason.items():
        if col in df.columns:
            print(f"{col} is dropped because: {reason}")
            df_orig[f"{col}_deleted_col"] = df[col]
            df.drop([col], axis=1, inplace=True)
            df_orig[f"{col}_deleted_col"] = f"DELETED ROW due to: {reason}"
        else:
            print(f"{col} is not present in df.")
```

pscol is dropped because: More than 97% of this column is missing.
Unnamed: 0.1 is dropped because: No information about the content of this column; probably the result of a loading and saving the data in pandas.
Unnamed: 0 is dropped because: No information about the content of this column; probably the result of a loading and saving the data in pandas.
Unnamed: 0 cleaned is dropped because: No information about the content of this column; probably the result of a loading and saving the data in pandas.
Source is dropped because: Similar to an ID, which does not provide additional information to the actual data.
Source cleaned is not present in df.
RA_ICRS is dropped because: Original row, that was preprocessed using binning.
RA_ICRS_equip_depth_52 is dropped because: Row that was preprocessed using binning, does not add more information to the data.
DE_ICRS is dropped because: Original row, that was preprocessed using binning.
DE_ICRS_equip_depth_52 is dropped because: Row that was preprocessed using binning, does not add more information to the data.
Age-Flame is dropped because: Original row, that was preprocessed using minmax standardization.
Age-Flame_scaled_minmax is dropped because: Row that was preprocessed using minmax standardization, does not add more information to the data.
Mass-Flame is dropped because: Original row, that was preprocessed using discretization.
Mass-Flame_categorized is dropped because: Row that was preprocessed using discretization, does not add more information to the data.
SpType-ELS is dropped because: Original row, that was preprocessed using binarization.

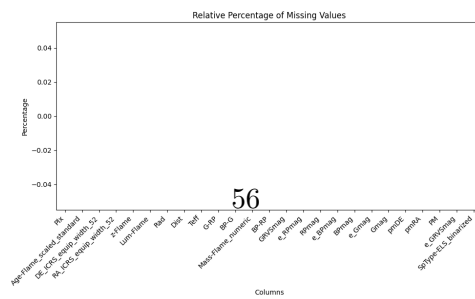
```
[ ]: df.columns
```

```
[ ]: Index(['Plx', 'PM', 'pmRA', 'pmDE', 'Gmag', 'e_Gmag', 'BPmag', 'e_BPmag',
          'RPMag', 'e_RPMag', 'GRVSmag', 'e_GRVSmag', 'BP-RP', 'BP-G', 'G-RP',
          'Teff', 'Dist', 'Rad', 'Lum-Flame', 'z-Flame', 'RA_ICRS_equip_width_52',
          'DE_ICRS_equip_width_52', 'Age-Flame_scaled_standard',
          'Mass-Flame_numeric', 'SpType-ELS_binarized'],
          dtype='object')
```

```
[ ]: gen_missing_values_barplot(df=df, suffix='cleaned', sorted=False)
```

[illegible]

```
[ ]: gen_missing_values_barplot(df=df, suffix='cleaned', sorted=True)
```

[illegible]

```
[ ]: # Profiling report after preprocessing
#profile_cleaned = ProfileReport(df, title="Profiling Report of clean Data")
#profile_cleaned.to_file("./output/exploration/reports/profiling_report_cleaned.
↳html")

[ ]: df.to_pickle("./output/pk/cleaned_data.pkl")
df_orig.to_pickle("./output/pk/cleaned_data_protocol.pkl")
df_orig.to_excel("./output/excel/cleaned_data_protocol.xlsx")
```

13.1 Standardizing

- MinMaxScaler if your data has a clear minimum and maximum and you want to scale the features to a specific range.
- StandardScaler if your data is normally distributed and you want to scale the features to have a mean of 0 and a standard deviation of 1.
- RobustScaler if your data contains outliers and you want to scale the features while preserving the relative differences between data points.

```
[ ]: df.columns

[ ]: Index(['Plx', 'PM', 'pmRA', 'pmDE', 'Gmag', 'e_Gmag', 'BPmag', 'e_BPmag',
          'RPMag', 'e_RPMag', 'GRVSmag', 'e_GRVSmag', 'BP-RP', 'BP-G', 'G-RP',
          'Teff', 'Dist', 'Rad', 'Lum-Flame', 'z-Flame', 'RA_ICRS_equip_width_52',
          'DE_ICRS_equip_width_52', 'Age-Flame_scaled_standard',
          'Mass-Flame_numeric', 'SpType-ELS_binarized'],
          dtype='object')

[ ]: already_processed = ['RA_ICRS_equip_width_16', 'DE_ICRS_equip_width_16',
↳'Age-Flame_scaled_minmax', 'Age-Flame_scaled_standard',
↳'Mass-Flame_numeric', 'SpType-ELS_binarized'] # TODO: Refactor

[ ]: def suggest_scaler_for_df(df, already_processed=[]):
    columns_dict = {}
    for column in df.columns:
        if column not in already_processed:
            if pd.api.types.is_numeric_dtype(df[column]):
                data_skewness = skew(df[column].dropna())
                Q1, Q3 = df[column].quantile([0.25, 0.75])
                IQR = Q3 - Q1
                outlier_count = ((df[column] < (Q1 - 1.5 * IQR)) | (df[column]
↳ (Q3 + 1.5 * IQR))).sum()

                # Initialize dictionary for the column
                columns_dict[column] = {'scaler': None, 'reason': ''}
```

```

        # Suggesting scaler based on skewness and outliers
        if abs(data_skewness) > SKEWNESS_THRESHOLD:
            if outlier_count > OUTLIER_COUNT_THRESHOLD:
                columns_dict[column]['scaler'] = 'robust'
                columns_dict[column]['reason'] = 'High skewness and
↳outliers present'
            else:
                columns_dict[column]['scaler'] = 'minmax'
                columns_dict[column]['reason'] = 'High skewness'
        else:
            if outlier_count > OUTLIER_COUNT_THRESHOLD:
                columns_dict[column]['scaler'] = 'robust'
                columns_dict[column]['reason'] = 'Outliers present'
            else:
                columns_dict[column]['scaler'] = 'standard'
                columns_dict[column]['reason'] = 'Data is fairly normal'
    else:
        # Non-numeric column
        columns_dict[column] = {'scaler': None, 'reason': 'Not numeric'}
return columns_dict

```

```
[ ]: normalization_reasons = suggest_scaler_for_df(df, already_processed)
```

```
[ ]: def pretty_print_normalization_dict(normalization_dict):
    for key, value in normalization_dict.items():
        print(f'{key}:')
        print(f"    Scaler: {value['scaler']}")
        print(f"    Reason: {value['reason']}\n")

```

```
[ ]: pretty_print_normalization_dict(normalization_reasons)
```

Plx:

```

    Scaler: robust
    Reason: High skewness and outliers present

```

PM:

```

    Scaler: robust
    Reason: High skewness and outliers present

```

pmRA:

```

    Scaler: robust
    Reason: Outliers present

```

pmDE:

```

    Scaler: robust
    Reason: High skewness and outliers present

```

Gmag:

Scaler: robust
Reason: Outliers present

e_Gmag:
Scaler: robust
Reason: High skewness and outliers present

BPmag:
Scaler: robust
Reason: Outliers present

e_BPmag:
Scaler: robust
Reason: High skewness and outliers present

RPmag:
Scaler: robust
Reason: Outliers present

e_RPmag:
Scaler: robust
Reason: High skewness and outliers present

GRVSmag:
Scaler: robust
Reason: Outliers present

e_GRVSmag:
Scaler: robust
Reason: High skewness and outliers present

BP-RP:
Scaler: robust
Reason: High skewness and outliers present

BP-G:
Scaler: robust
Reason: High skewness and outliers present

G-RP:
Scaler: robust
Reason: Outliers present

Teff:
Scaler: robust
Reason: High skewness and outliers present

Dist:

Scaler: robust
Reason: High skewness and outliers present

Rad:
Scaler: robust
Reason: High skewness and outliers present

Lum-Flame:
Scaler: robust
Reason: High skewness and outliers present

z-Flame:
Scaler: robust
Reason: High skewness and outliers present

RA_ICRS_equip_width_52:
Scaler: minmax
Reason: High skewness

DE_ICRS_equip_width_52:
Scaler: robust
Reason: High skewness and outliers present

```
[ ]: for column, info in normalization_reasons.items():
    if info['scaler']:
        print(f"Scaling {column} using {info['scaler']}")
        df, df_orig = scale_column(column, df, df_orig, method=info['scaler'])

        # we drop it afterward and append the reason to the dict
        print(f"{column} is dropped because: Original row, that was
↳preprocessed using {info['scaler']} standardization.")
        df_orig[f"{col}_deleted_col"] = df[column]
        df.drop([column], axis=1, inplace=True)
        df_orig[f"{col}_deleted_col"] = f"DELETED ROW due to: {info['scaler']}
↳standardization"
```

Scaling Plx using robust

Plx is dropped because: Original row, that was preprocessed using robust standardization.

Scaling PM using robust

PM is dropped because: Original row, that was preprocessed using robust standardization.

Scaling pmRA using robust

pmRA is dropped because: Original row, that was preprocessed using robust standardization.

Scaling pmDE using robust

pmDE is dropped because: Original row, that was preprocessed using robust

standardization.
Scaling Gmag using robust
Gmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling e_Gmag using robust
e_Gmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling BPmag using robust
BPmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling e_BPmag using robust
e_BPmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling RPmag using robust
RPmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling e_RPmag using robust
e_RPmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling GRVSmag using robust
GRVSmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling e_GRVSmag using robust
e_GRVSmag is dropped because: Original row, that was preprocessed using robust standardization.
Scaling BP-RP using robust
BP-RP is dropped because: Original row, that was preprocessed using robust standardization.
Scaling BP-G using robust
BP-G is dropped because: Original row, that was preprocessed using robust standardization.
Scaling G-RP using robust
G-RP is dropped because: Original row, that was preprocessed using robust standardization.
Scaling Teff using robust
Teff is dropped because: Original row, that was preprocessed using robust standardization.
Scaling Dist using robust
Dist is dropped because: Original row, that was preprocessed using robust standardization.
Scaling Rad using robust
Rad is dropped because: Original row, that was preprocessed using robust standardization.
Scaling Lum-Flame using robust
Lum-Flame is dropped because: Original row, that was preprocessed using robust standardization.
Scaling z-Flame using robust
z-Flame is dropped because: Original row, that was preprocessed using robust

standardization.

Scaling RA_ICRS_equip_width_52 using minmax

RA_ICRS_equip_width_52 is dropped because: Original row, that was preprocessed using minmax standardization.

Scaling DE_ICRS_equip_width_52 using robust

DE_ICRS_equip_width_52 is dropped because: Original row, that was preprocessed using robust standardization.

```
[ ]: df.to_pickle("./output/pk/preprocessed_data.pkl")
df_orig.to_pickle("./output/pk/preprocessed_data_protocol.pkl")
df_orig.to_excel("./output/excel/preprocessed_data_protocol.xlsx")
```

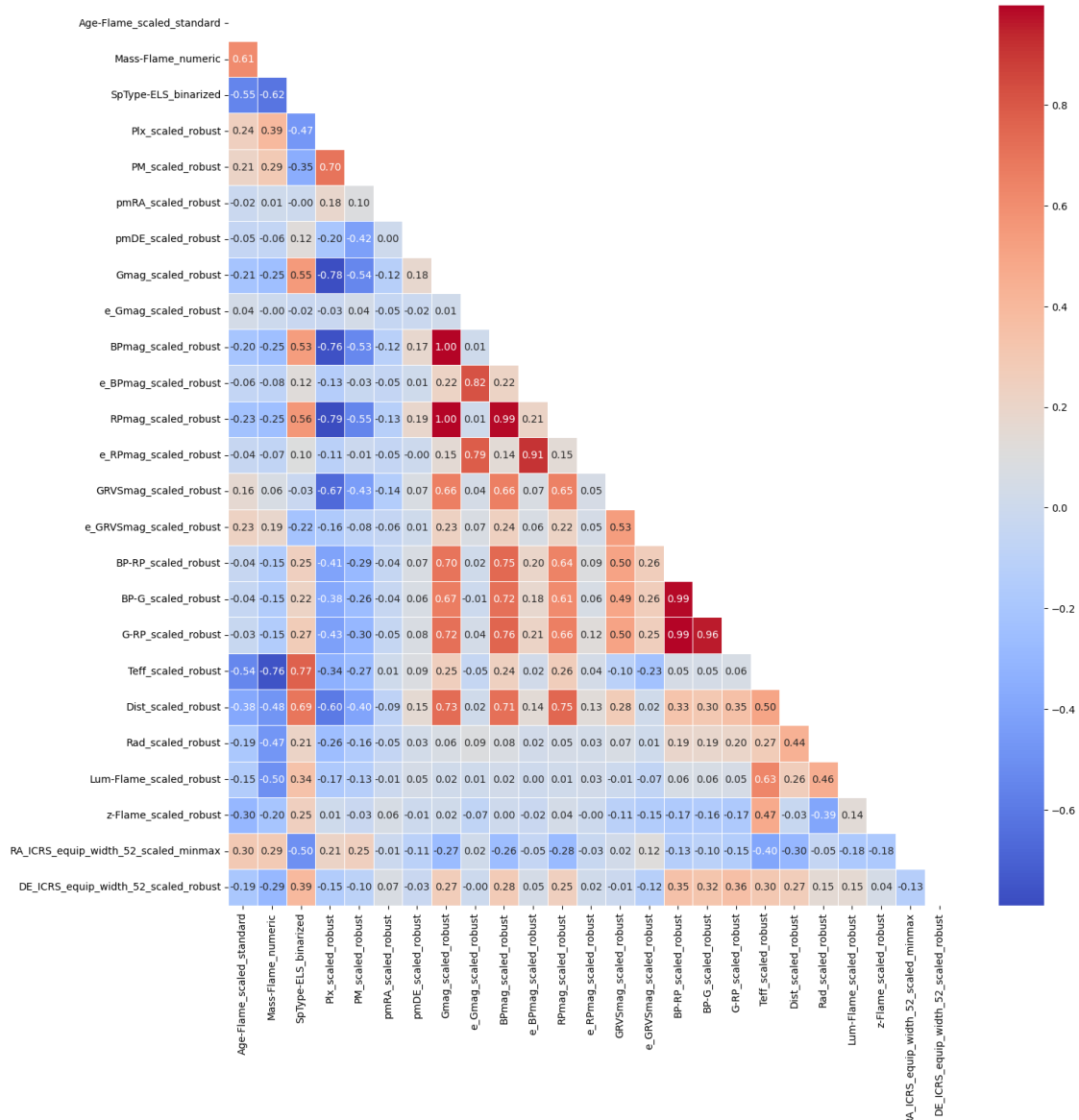
14 Correlation

```
[ ]: #TODO: Refactor, since we already have this code
```

```
[ ]: # Select only numeric columns
df_only_numeric_columns = df.select_dtypes(include=['float64', 'int64'])
# Compute correlation matrix
corr_matrix = df_only_numeric_columns.corr()

# Create a mask for the upper triangle
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Visualize correlation matrix
plt.figure(figsize=(16, 16))
ax = sns.heatmap(data=corr_matrix, annot=True, cmap=PALETTE, mask=mask, fmt=".
↪2f", linewidths=0.5)
plt.savefig('./output/exploration/features/correlation_heatmap_preprocessed.
↪pdf')
```



```
[ ]: # Stack the correlation matrix to get pairs of features and their correlations
correlation_pairs = corr_matrix.stack().reset_index()
correlation_pairs.columns = ['Feature 1', 'Feature 2', 'Correlation']
```

```
[ ]: # Sort by correlation values in descending order
sorted_correlation_pairs = correlation_pairs.sort_values(by='Correlation',
↪ascending=False)

print(sorted_correlation_pairs)
```

	Feature 1	Feature 2	Correlation
0	Age-Flame_scaled_standard	Age-Flame_scaled_standard	1.000000

338	GRVSmag_scaled_robust	GRVSmag_scaled_robust	1.000000
78	Plx_scaled_robust	Plx_scaled_robust	1.000000
104	PM_scaled_robust	PM_scaled_robust	1.000000
130	pmRA_scaled_robust	pmRA_scaled_robust	1.000000
..	...	...	...
228	BPmag_scaled_robust	Plx_scaled_robust	-0.763276
82	Plx_scaled_robust	Gmag_scaled_robust	-0.777089
178	Gmag_scaled_robust	Plx_scaled_robust	-0.777089
86	Plx_scaled_robust	RPmag_scaled_robust	-0.789667
278	RPmag_scaled_robust	Plx_scaled_robust	-0.789667

[625 rows x 3 columns]

```
[ ]: # Filter correlation pairs above the threshold, excluding those involving the
      ↪ same column and duplicates
high_correlation_pairs = sorted_correlation_pairs[
    (sorted_correlation_pairs['Correlation'] > CORRELATION_THRESHOLD) &
    (sorted_correlation_pairs['Feature 1'] < sorted_correlation_pairs['Feature_
      ↪ 2']) # Exclude duplicates
]

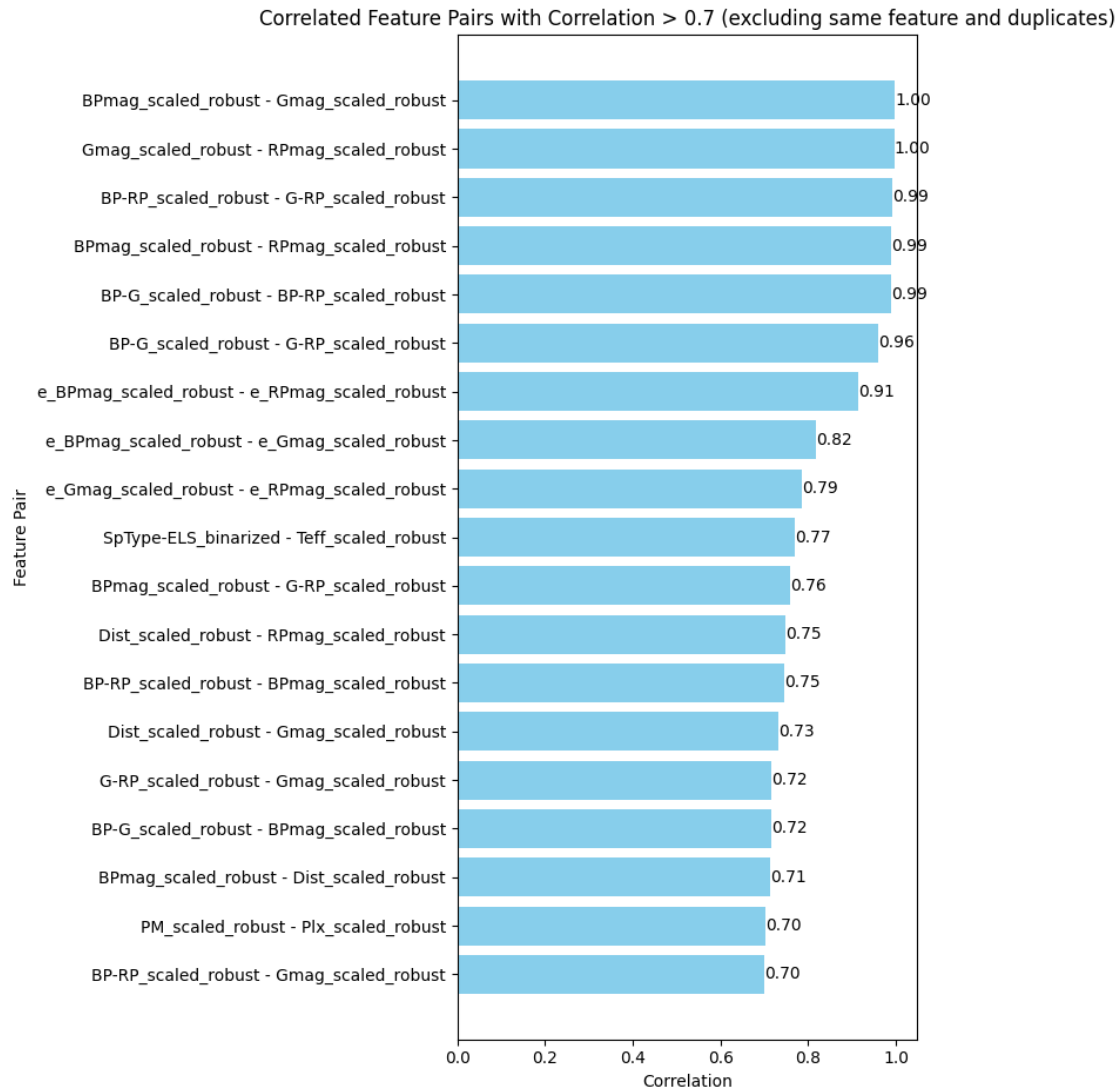
# Adjusting figure size and layout for clarity and visibility of names
plt.figure(figsize=(10, 0.5 * len(high_correlation_pairs))) # Adjust height
      ↪ dynamically based on number of pairs
bars = plt.barh(high_correlation_pairs['Feature 1'] + ' - ' +
      ↪ high_correlation_pairs['Feature 2'],
                high_correlation_pairs['Correlation'], color='skyblue')
plt.xlabel('Correlation')
plt.ylabel('Feature Pair')
plt.title(f'Correlated Feature Pairs with Correlation > {CORRELATION_THRESHOLD}
      ↪ (excluding same feature and duplicates)')
plt.gca().invert_yaxis() # Invert y-axis for better readability

# Annotate bars with correlation values
for bar, correlation in zip(bars, high_correlation_pairs['Correlation']):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
      ↪ f'{correlation:.2f}',
            va='center', ha='left')

plt.tight_layout() # Adjust layout to make room for label

# Before saving, adjust the layout to prevent cutting off label texts
plt.savefig('./output/exploration/features/
      ↪ correlated_features_hierarchy_preprocessed.pdf', bbox_inches='tight')

plt.show()
```



```
[94]: high_correlation_pairs
```

```
[94]:
```

	Feature 1	Feature 2	Correlation
232	BPmag_scaled_robust	Gmag_scaled_robust	0.997783
186	Gmag_scaled_robust	RPmag_scaled_robust	0.996836
392	BP-RP_scaled_robust	G-RP_scaled_robust	0.991117
236	BPmag_scaled_robust	RPmag_scaled_robust	0.989714
415	BP-G_scaled_robust	BP-RP_scaled_robust	0.989071
417	BP-G_scaled_robust	G-RP_scaled_robust	0.960676
262	e_BPmag_scaled_robust	e_RPmag_scaled_robust	0.913511
258	e_BPmag_scaled_robust	e_Gmag_scaled_robust	0.817018
212	e_Gmag_scaled_robust	e_RPmag_scaled_robust	0.786148
68	SpType-ELS_binarized	Teff_scaled_robust	0.768813

242	BPmag_scaled_robust	G-RP_scaled_robust	0.758235
486	Dist_scaled_robust	RPmag_scaled_robust	0.749327
384	BP-RP_scaled_robust	BPmag_scaled_robust	0.745566
482	Dist_scaled_robust	Gmag_scaled_robust	0.731922
432	G-RP_scaled_robust	Gmag_scaled_robust	0.716730
409	BP-G_scaled_robust	BPmag_scaled_robust	0.716028
244	BPmag_scaled_robust	Dist_scaled_robust	0.713496
103	PM_scaled_robust	Plx_scaled_robust	0.702756
382	BP-RP_scaled_robust	Gmag_scaled_robust	0.700517

```
[95]: corr_matrix = df.corr().abs()

# Select upper triangle of correlation matrix
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).
    ↳astype(bool))

# Find features with correlation greater than 0.8
high_correlation_columns = [column for column in upper_tri.columns if
    ↳any(upper_tri[column] > CORRELATION_THRESHOLD)]
high_correlation_columns
```

```
[95]: ['PM_scaled_robust',
      'Gmag_scaled_robust',
      'BPmag_scaled_robust',
      'e_BPmag_scaled_robust',
      'RPmag_scaled_robust',
      'e_RPmag_scaled_robust',
      'BP-RP_scaled_robust',
      'BP-G_scaled_robust',
      'G-RP_scaled_robust',
      'Teff_scaled_robust',
      'Dist_scaled_robust']
```

```
[96]: feature_to_be_dropped = [ # 'Gmag_scaled_robust',
                                'BPmag_scaled_robust',
                                'e_BPmag_scaled_robust',
                                'RPmag_scaled_robust',
                                'e_RPmag_scaled_robust',
                                'BP-RP_scaled_robust',
                                'BP-G_scaled_robust',
                                'G-RP_scaled_robust',
                                # 'Teff_scaled_robust',
                                'Dist_scaled_robust'
                                ]
```

```
[97]: df.columns
```

```
[97]: Index(['Age-Flame_scaled_standard', 'Mass-Flame_numeric',
          'SpType-ELS_binarized', 'Plx_scaled_robust', 'PM_scaled_robust',
          'pmRA_scaled_robust', 'pmDE_scaled_robust', 'Gmag_scaled_robust',
          'e_Gmag_scaled_robust', 'BPmag_scaled_robust', 'e_BPmag_scaled_robust',
          'RPMag_scaled_robust', 'e_RPMag_scaled_robust', 'GRVSmag_scaled_robust',
          'e_GRVSmag_scaled_robust', 'BP-RP_scaled_robust', 'BP-G_scaled_robust',
          'G-RP_scaled_robust', 'Teff_scaled_robust', 'Dist_scaled_robust',
          'Rad_scaled_robust', 'Lum-Flame_scaled_robust', 'z-Flame_scaled_robust',
          'RA_ICRS_equip_width_52_scaled_minmax',
          'DE_ICRS_equip_width_52_scaled_robust'],
          dtype='object')
```

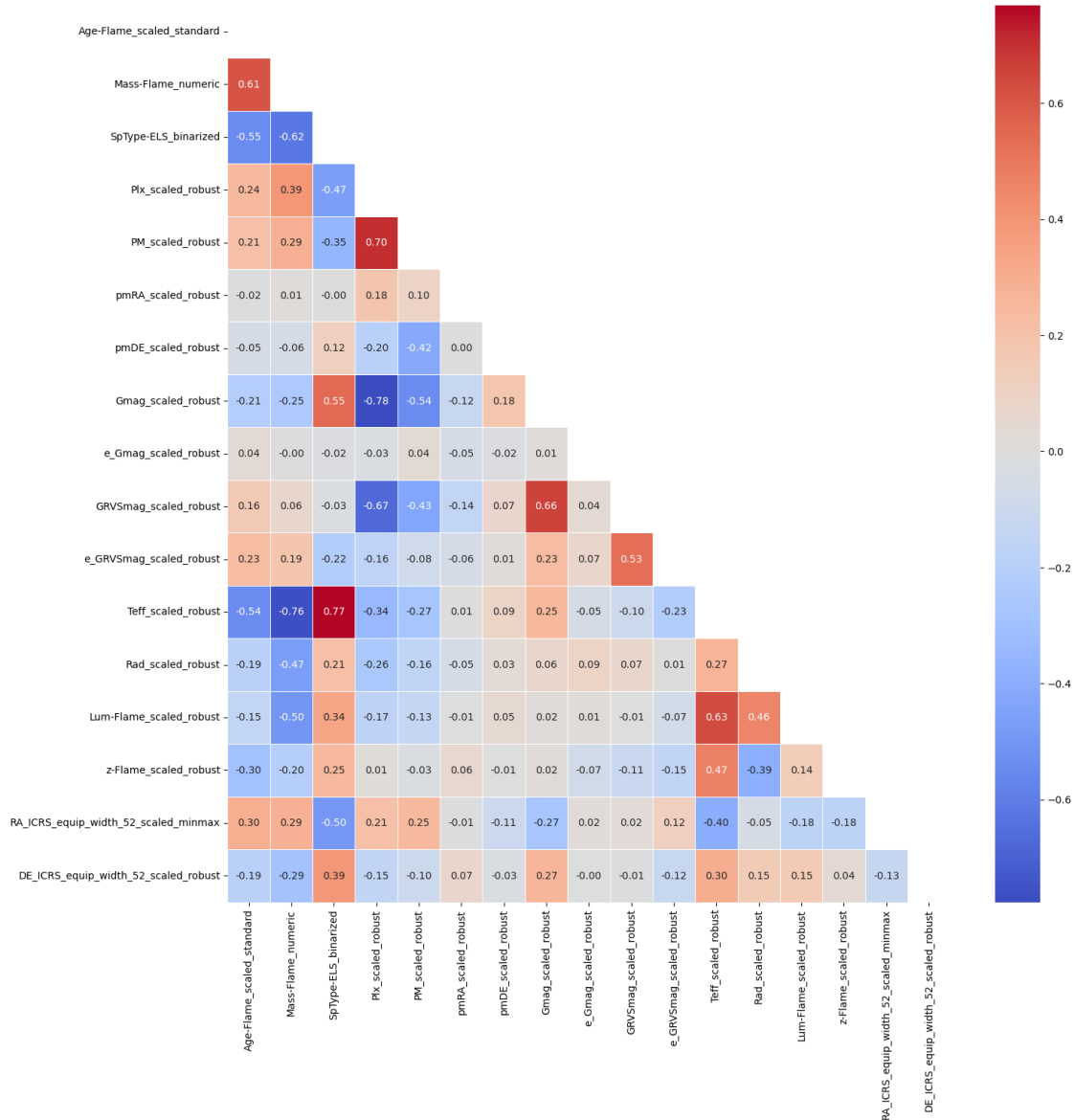
```
[98]: for col in feature_to_be_dropped:
        print(f"{col} is dropped to minimize overall correlation.")
        # Log the column data before dropping
        df_orig[f"{col}_deleted_col"] = df[col]
        # Drop the column from the original DataFrame
        df.drop(col, axis=1, inplace=True)
        # Mark the column as deleted in the log DataFrame
        df_orig[f"{col}_deleted_col_reason"] = "DELETED due to: minimize overall_
        correlation"
```

BPmag_scaled_robust is dropped to minimize overall correlation.
e_BPmag_scaled_robust is dropped to minimize overall correlation.
RPMag_scaled_robust is dropped to minimize overall correlation.
e_RPMag_scaled_robust is dropped to minimize overall correlation.
BP-RP_scaled_robust is dropped to minimize overall correlation.
BP-G_scaled_robust is dropped to minimize overall correlation.
G-RP_scaled_robust is dropped to minimize overall correlation.
Dist_scaled_robust is dropped to minimize overall correlation.

```
[99]: # Select only numeric columns
df_only_numeric_columns = df.select_dtypes(include=['float64', 'int64'])
# Compute correlation matrix
corr_matrix = df_only_numeric_columns.corr()

# Create a mask for the upper triangle
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Visualize correlation matrix
plt.figure(figsize=(16, 16))
ax = sns.heatmap(data=corr_matrix, annot=True, cmap=PALETTE, mask=mask, fmt="%.
    2f", linewidths=0.5)
plt.savefig('./output/exploration/correlation_heatmap_final.pdf')
```



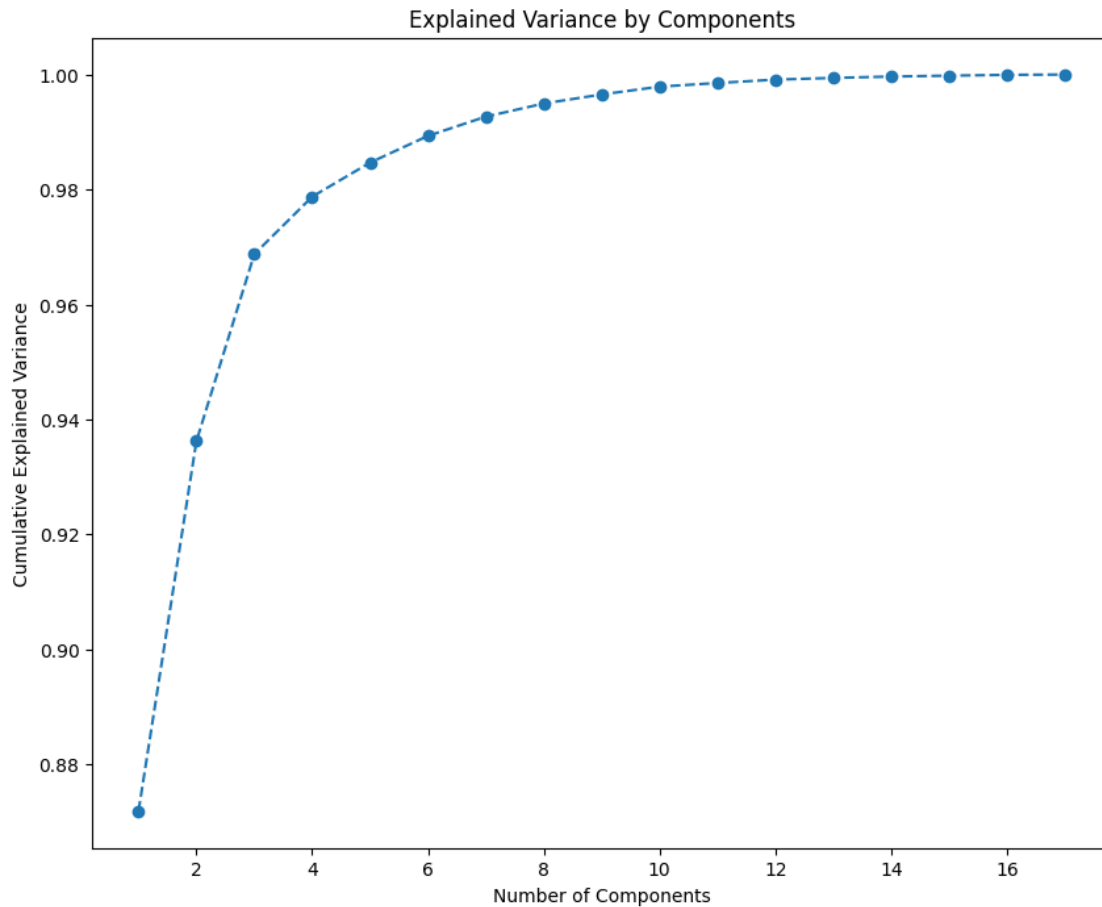
15 Dimred

15.1 PCA

```
[100]: pca = PCA()
pca.fit(df)
var_ratio = pca.explained_variance_ratio_
```

```
[101]: fig = plt.figure(figsize=(10,8))
plt.plot(range(1, len(var_ratio)+1), var_ratio.cumsum(), marker='o',
linestyle='--')
```

```
plt.title('Explained Variance by Components')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
fig.savefig('./output/exploration/dimred/pca_variance_by_principal_components.
pdf')
```



```
[102]: PCA_COMPONENTS = 2
```

```
[103]: pca = PCA(n_components=PCA_COMPONENTS)
pca.fit(df)
scores_pca = pca.transform(df)
print(scores_pca)
```

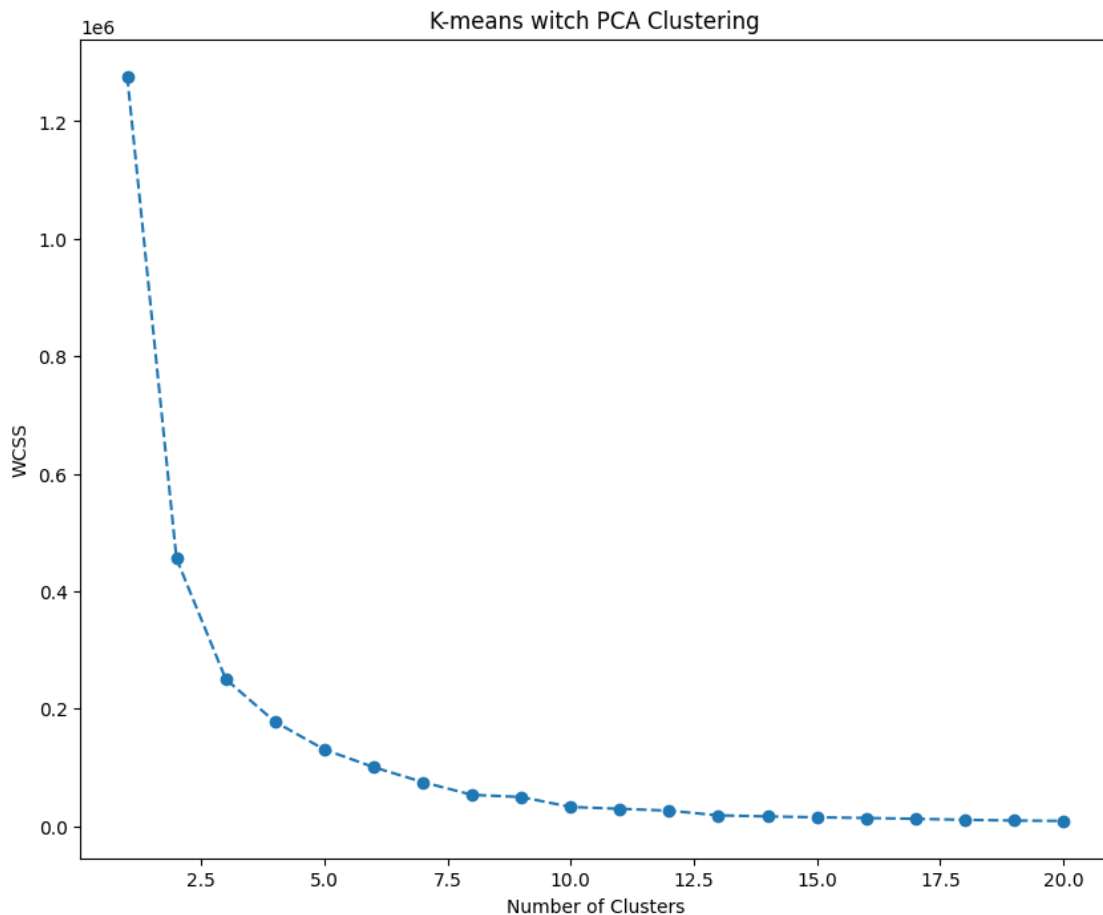
```
[[-0.87608602 -1.34432356]
 [-2.02040883 -1.30136457]
 [-2.51011103 -1.33837776]
 ...
 [-2.42427707 -1.33159604]
 [-2.54611959 -2.97231361]]
```

```
[-1.84153019 -3.0167047 ]]
```

16 Clustering

```
[104]: wcss = []  
       for i in range(1, 21):  
           kmeans_pca = KMeans(n_clusters=i, init='k-means++', random_state=SEED)  
           kmeans_pca.fit(scores_pca)  
           wcss.append(kmeans_pca.inertia_)
```

```
[105]: fig = plt.figure(figsize = (10,8))  
       plt.plot(range(1, 21), wcss, marker='o', linestyle='--')  
       plt.title('K-means witch PCA Clustering')  
       plt.xlabel('Number of Clusters')  
       plt.ylabel('WCSS')  
       fig.savefig('./output/exploration/dimred/pca_no_clusters.pdf')
```



```
[106]: OPTIMAL_N_CLUSTER = 2
```

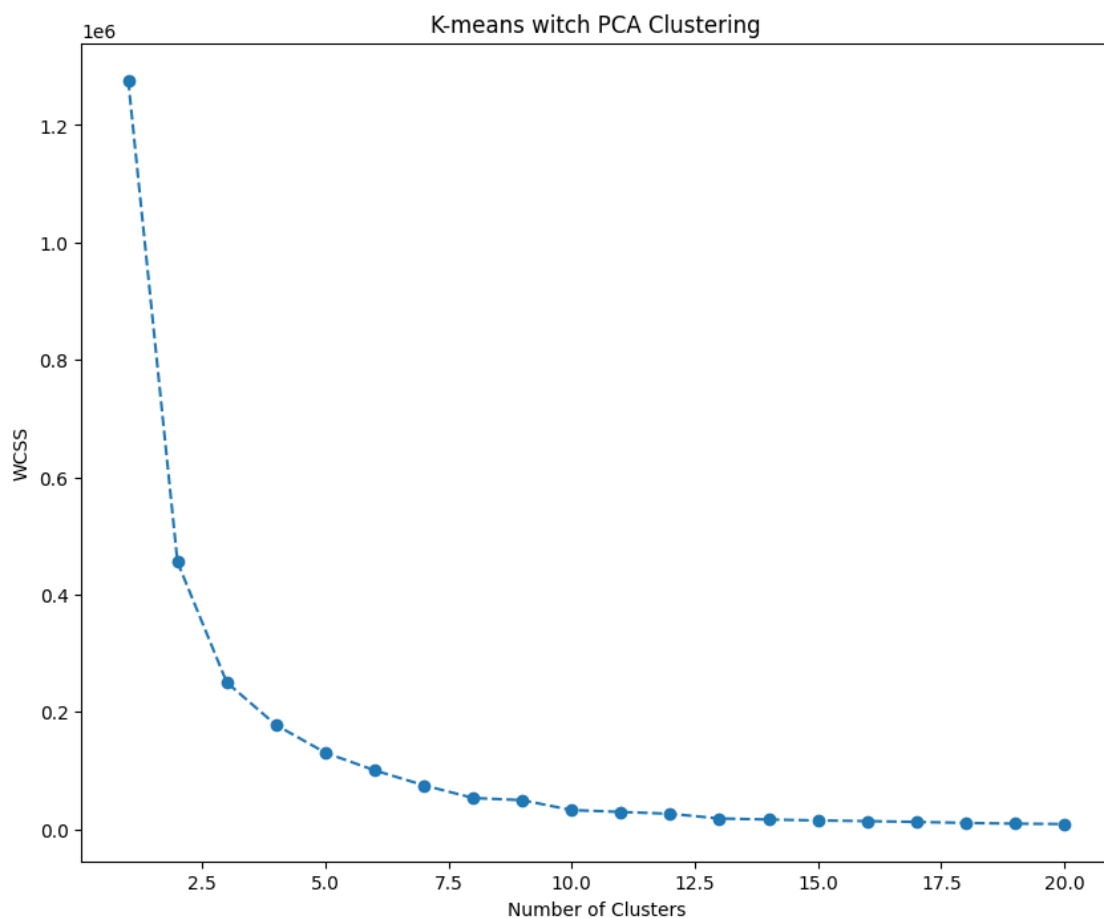
```
[107]: kmeans_pca = KMeans(n_clusters=OPTIMAL_N_CLUSTER, init='k-means++',
    ↪random_state=SEED)
kmeans_pca.fit(scores_pca)
```

```
[107]: KMeans(n_clusters=2, random_state=31011997)
```

```
[108]: df_seg_pca_kmeans = pd.concat([df.reset_index(drop=True), pd.
    ↪DataFrame(scores_pca)], axis=1)
df_seg_pca_kmeans.columns.values[-PCA_COMPONENTS:] = ['PComp 1', 'PComp 2']

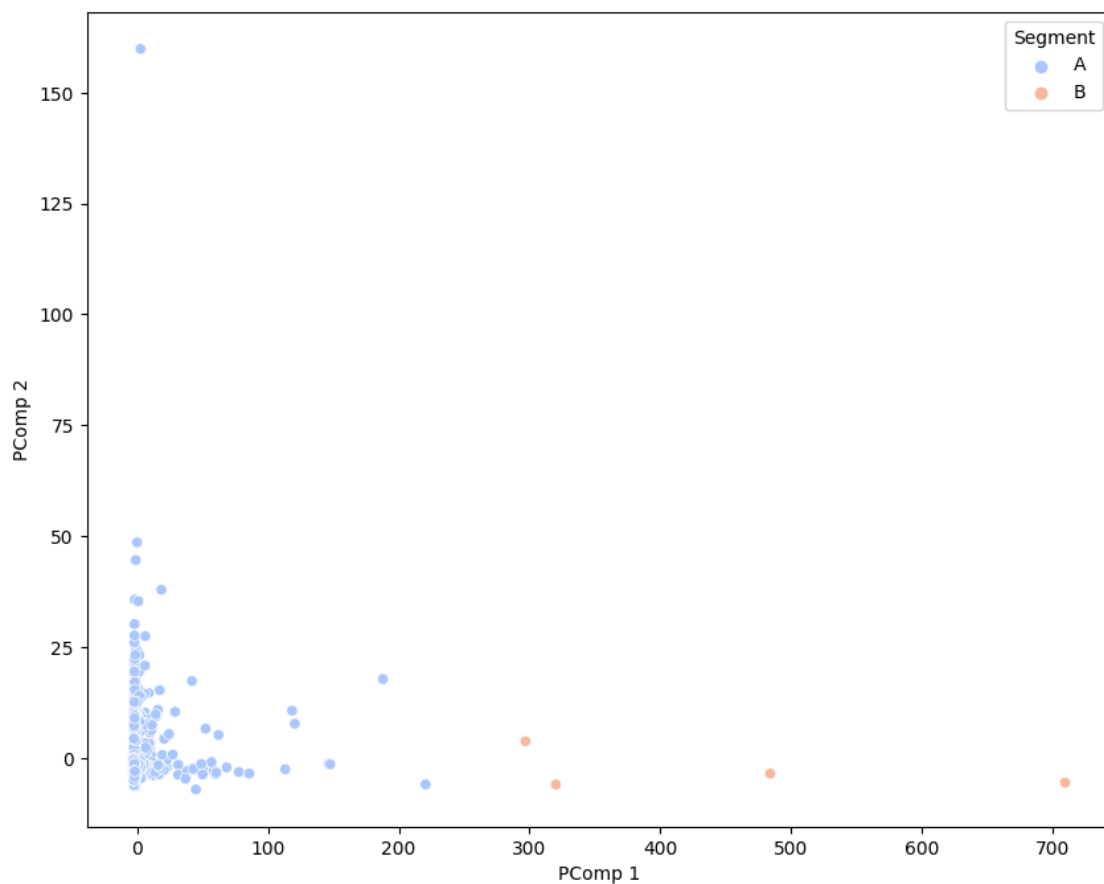
df_seg_pca_kmeans['Segment K-means PCA'] = kmeans_pca.labels_
```

```
[109]: fig = plt.figure(figsize = (10,8))
plt.plot(range(1, 21), wcss, marker='o', linestyle='--')
plt.title('K-means witch PCA Clustering')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
fig.savefig('./output/exploration/dimred/pca_no_clusters_wcss.pdf')
```



```
[110]: df_segm_pca_kmeans['Segment'] = df_segm_pca_kmeans['Segment K-means PCA'].
        ↪map({0: 'A',
        ↪1: 'B',
        ↪})
```

```
[111]: x_axis = df_segm_pca_kmeans['PComp 1']
        y_axis = df_segm_pca_kmeans['PComp 2']
        plt.figure(figsize=(10,8))
        sns.scatterplot(x=x_axis, y=y_axis, hue = df_segm_pca_kmeans['Segment'],
        ↪palette=PALETTE)
        plt.show()
```



```
[112]: df_cluster = df_segm_pca_kmeans.iloc[:, -4:] # 2 components + hue + cat
        df_cluster.head
```

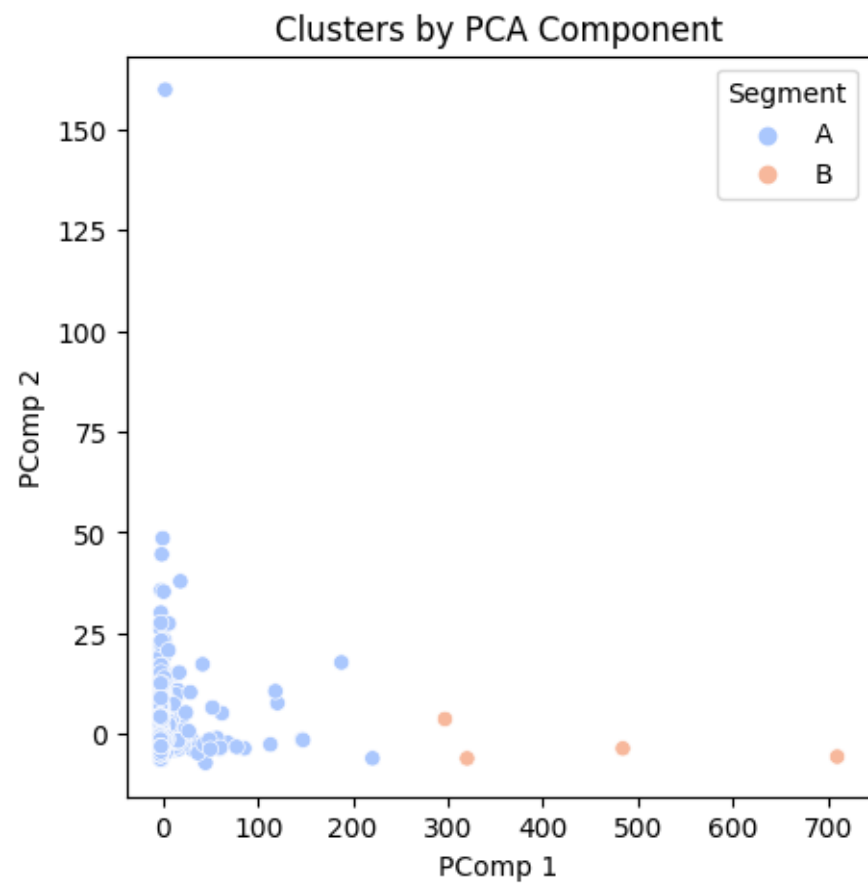
```
[112]: <bound method NDFrame.head of          PComp 1  PComp 2  Segment K-means PCA
Segment
```

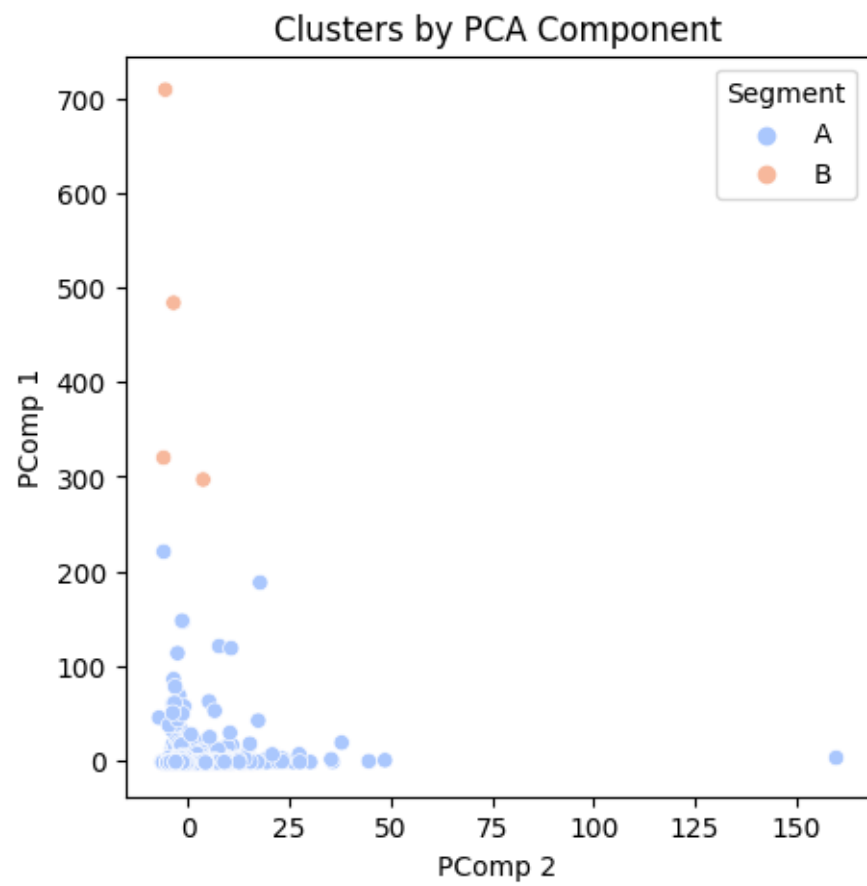
0	-0.876086	-1.344324	0	A
1	-2.020409	-1.301365	0	A
2	-2.510111	-1.338378	0	A
3	-2.594749	-3.122191	0	A
4	-2.084107	-1.930079	0	A
...	...	...	...	...
2751	-1.689417	-1.626774	0	A
2752	-1.459519	-0.951990	0	A
2753	-2.424277	-1.331596	0	A
2754	-2.546120	-2.972314	0	A
2755	-1.841530	-3.016705	0	A

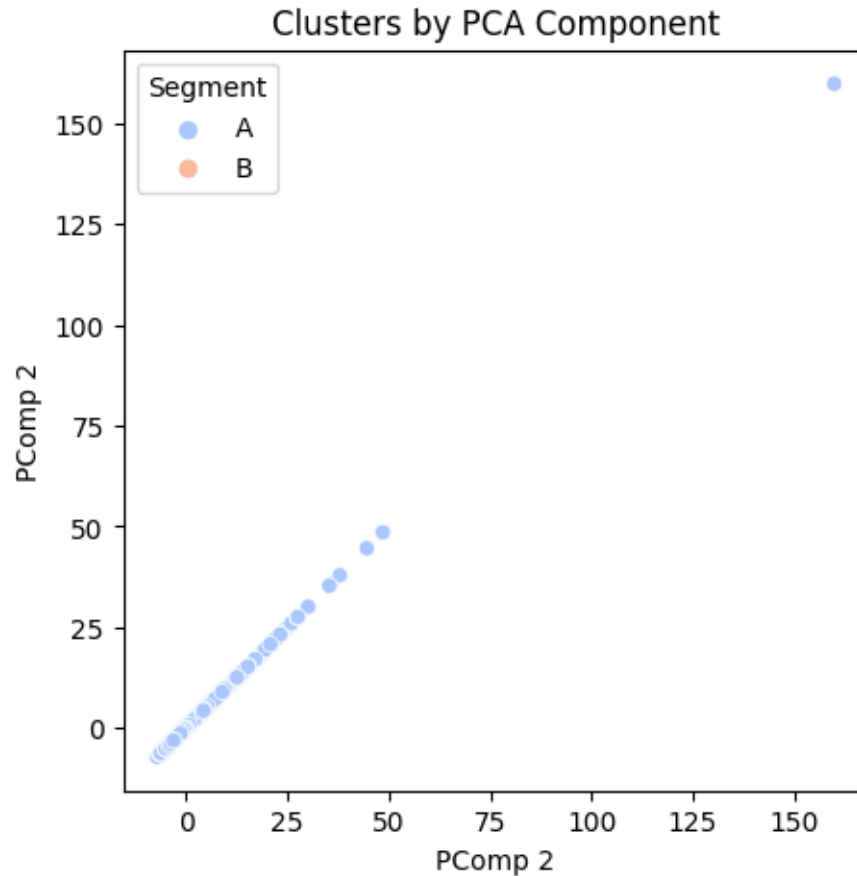
[2756 rows x 4 columns]>

```
[113]: for n_index, column in enumerate(df_cluster.columns[:PCA_COMPONENTS]):
        for m_index, column_iterator in enumerate(df_cluster.columns[:
        PCA_COMPONENTS]):
            #print(n_index, column, m_index, column_iterator)
            fig = plt.figure(figsize=(5, 5))
            sns.scatterplot(x=df_cluster[column], y=df_cluster[column_iterator],
            hue = df_segm_pca_kmeans['Segment'], palette=PALETTE)
            plt.xlabel(column)
            plt.ylabel(column_iterator)
            plt.title('Clusters by PCA Component')
            fig.savefig('./output/exploration/dimred/pca/PCA {} vs {}.pdf'.
            format(column, column_iterator))
            #plt.show()
```









```
[114]: # PCA
pca = PCA(n_components=PCA_COMPONENTS)
pca_transformed = pca.fit_transform(df)
```

```
[115]: # UMAP
umap_reducer = umap.UMAP(n_components=PCA_COMPONENTS, random_state=SEED)
umap_transformed = umap_reducer.fit_transform(df)
```

OMP: Info #276: omp_set_nested routine deprecated, please use
omp_set_max_active_levels instead.

```
[116]: # t-SNE
tsne = TSNE(n_components=PCA_COMPONENTS, random_state=SEED)
tsne_transformed = tsne.fit_transform(df)
```

```
[117]: # Cluster labels with KMeans for each DR technique
kmeans_pca = KMeans(n_clusters=OPTIMAL_N_CLUSTER, random_state=SEED).
    ↪ fit(pca_transformed)
labels_pca = kmeans_pca.labels_
```

```
[118]: kmeans_umap = KMeans(n_clusters=OPTIMAL_N_CLUSTER, random_state=SEED).
        ↪fit(umap_transformed)
        labels_umap = kmeans_umap.labels_
```

```
[119]: kmeans_tsne = KMeans(n_clusters=OPTIMAL_N_CLUSTER, random_state=SEED).
        ↪fit(tsne_transformed)
        labels_tsne = kmeans_tsne.labels_
```

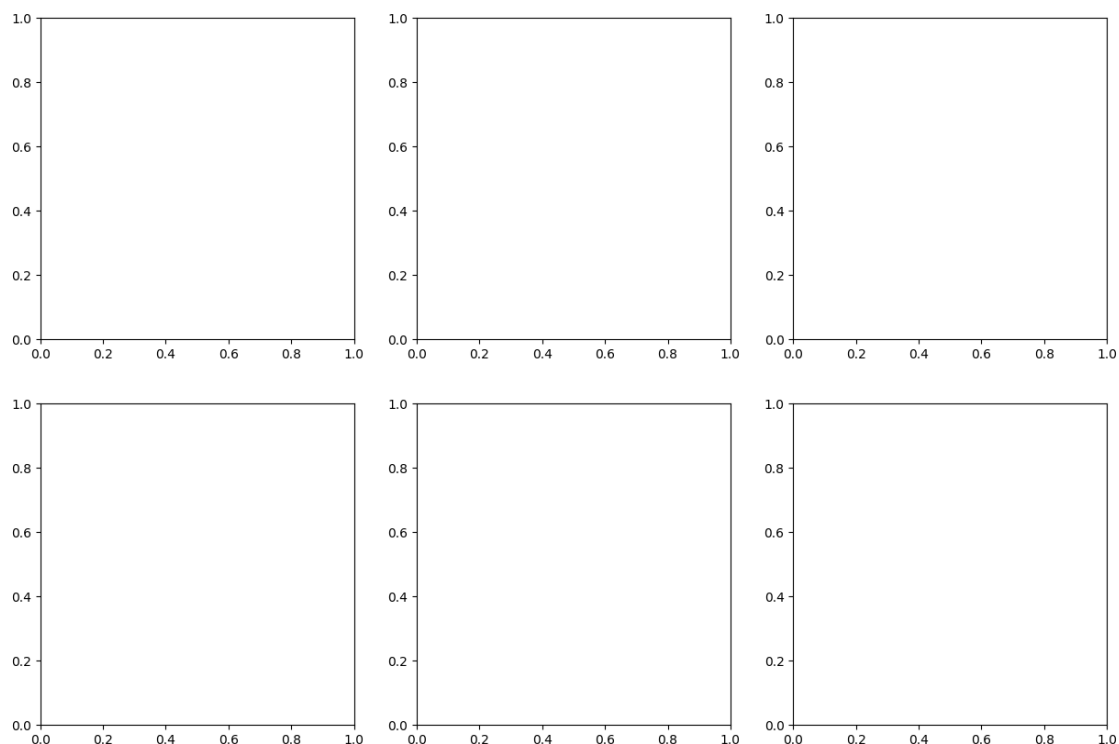
```
[120]: datasets = {
        'PCA': (scores_pca, df_segm_pca_kmeans['Segment']),
        'UMAP': (umap_transformed, labels_umap),
        't-SNE': (tsne_transformed, labels_tsne)
    }

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    # Assuming 'Class' column in your DataFrame contains the two classes
    class_labels = df['SpType-ELS_binarized'].values

    # Titles for plots
    titles = ['PCA', 'UMAP', 't-SNE']

    # Data for each column
    data_transformations = [pca_transformed, umap_transformed, tsne_transformed]
    cluster_labels = [labels_pca, labels_umap, labels_tsne]
```



```
[121]: # Access the 'viridis' colormap
viridis_cmap = plt.get_cmap('viridis')

# Select four distinct colors from 'viridis'
colors = [viridis_cmap(0.1), # Lighter color for Cluster 1
          viridis_cmap(0.35), # Another color for Cluster 2
          viridis_cmap(0.6), # Darker color for Class A
          viridis_cmap(0.85)] # Another distinct color for Class B

# Apply selected colors for clusters and classes
PALETTE_CLUSTERS = {0: colors[0], 1: colors[1]}
PALETTE_CLASSES = {0: colors[2], 1: colors[3]}

[122]: # Set a larger figsize here
fig, axes = plt.subplots(2, 3, figsize=(18, 10)) # Adjust width and height as
↪needed

for i, title in enumerate(titles):
    # First row - colored by clusters
    sns.scatterplot(x=data_transformations[i][:, 0], y=data_transformations[i][:, 1],
↪, 1],
                    hue=cluster_labels[i], palette=PALETTE_CLUSTERS, ax=axes[0,
↪i], legend=None)
    axes[0, i].set_title(f'Clusters by {title} Components')
    axes[0, i].set_xlabel('Component 1') # X-axis label
    axes[0, i].set_ylabel('Component 2') # Y-axis label

    # Second row - colored by class labels
    sns.scatterplot(x=data_transformations[i][:, 0], y=data_transformations[i][:, 1],
↪, 1],
                    hue=class_labels, palette=PALETTE_CLASSES, ax=axes[1, i],
↪legend=None)
    axes[1, i].set_title(f'{title} Components Colored by Class')
    axes[1, i].set_xlabel('Component 1') # X-axis label
    axes[1, i].set_ylabel('Component 2') # Y-axis label

# Create custom legends from the selected colors
legend_elements_clusters = [Line2D([0], [0], marker='o', color='w',
↪markerfacecolor=colors[0], markersize=10, label='Cluster 1'),
                             Line2D([0], [0], marker='o', color='w',
↪markerfacecolor=colors[1], markersize=10, label='Cluster 2')]

legend_elements_classes = [Line2D([0], [0], marker='o', color='w',
↪markerfacecolor=colors[2], markersize=10, label='Class A'),
```

```

Line2D([0], [0], marker='o', color='w',
↪markerfacecolor=colors[3], markersize=10, label='Class B'))

# Place one legend for clusters at the top right
fig.legend(handles=legend_elements_clusters, loc='upper right',
↪bbox_to_anchor=(0.95, 0.8), title="Cluster")

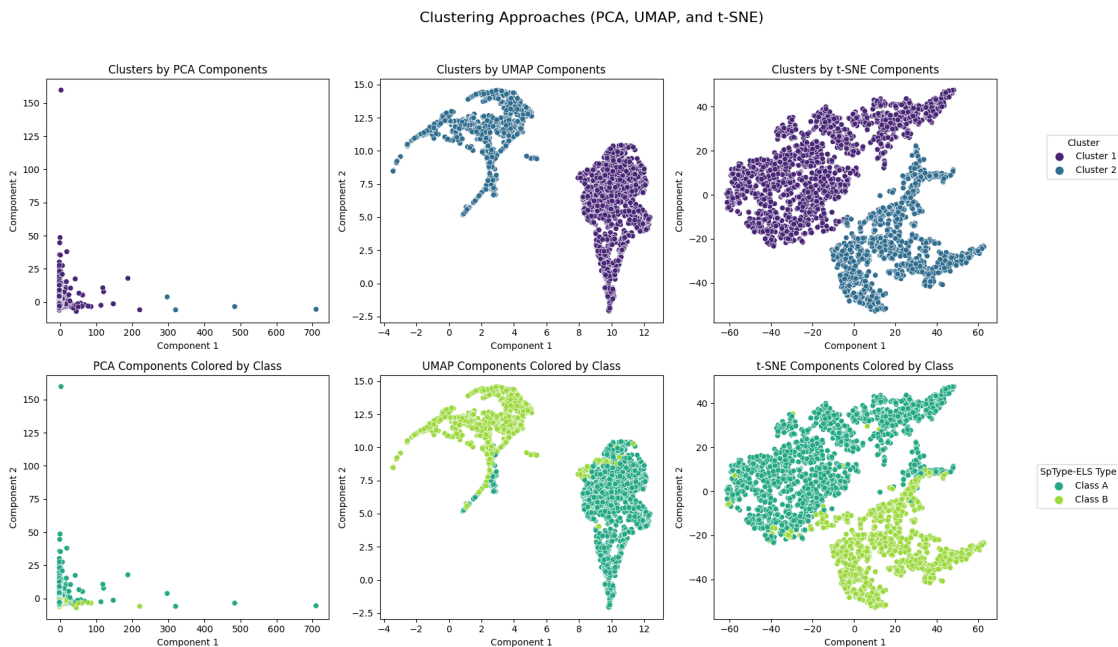
# Place another legend for classes below the clusters legend
fig.legend(handles=legend_elements_classes, loc='upper right',
↪bbox_to_anchor=(0.95, 0.3), title="SpType-ELS Type")

# Add a global title at the top of the figure
fig.suptitle('Clustering Approaches (PCA, UMAP, and t-SNE)', fontsize=16)

# Adjust layout to make room for the global title and legends outside the plots
plt.tight_layout(rect=[0, 0, 0.85, 0.95])

fig.savefig('./output/exploration/clustering.png')
plt.show()

```



17 Hierarchical Clustering

```
[123]: def plot_dendrogram_with_methods(data, dr_technique, levels=3):
    methods = ['ward', 'complete', 'average', 'single']
    num_methods = len(methods)

    for i in range(1, levels + 1):
        fig = plt.figure(figsize=(24, 6 * num_methods))

        for j, method in enumerate(methods):
            linked = linkage(data, method=method)
            plt.subplot(num_methods, 3, j * 3 + 1)
            dendrogram(linked, truncate_mode='level', p=i)
            plt.title(f'Level {i} Dendrogram ({method.capitalize()} Linkage)')

            model = AgglomerativeClustering(n_clusters=i, linkage=method)
            clusters = model.fit_predict(data)

            ax_middle = plt.subplot(num_methods, 3, j * 3 + 2)
            scatter_middle = plt.scatter(data[:, 0], data[:, 1], c=clusters,
            ↪ cmap='viridis', s=100, alpha=0.5)
            plt.title(f'Level {i} Scatter Plot ({method.capitalize()} Linkage)')
            plt.colorbar(scatter_middle, label='Cluster')

            ax_right = plt.subplot(num_methods, 3, j * 3 + 3)
            # Handle the case for level 1 specifically, because matplotlib is a
            ↪ bit weird?
            if i == 1:
                # Ridiculous workaround that cost me like 4-6 hours : 'D
                # For level 1, use the color of the scatter plot directly
                # For a single cluster, map its value to the midpoint of the
            ↪ colormap's range
                norm = Normalize(vmin=0, vmax=1) # The range of values in your
            ↪ clusters array
                mapper = cm.ScalarMappable(norm=norm, cmap='viridis')
                color = mapper.to_rgba(0.5) # Use the midpoint value for a
            ↪ single cluster

                center = data.mean(axis=0)
                size = len(data)
                plt.scatter([center[0]], [center[1]], s=size/len(data)*10000,
            ↪ color=color, alpha=0.5)
                plt.text(center[0], center[1], f'{100:.1f}%', ha='center',
            ↪ va='center', fontsize=8, color='white')
            else:
                # For levels greater than 1, proceed as normal
```

```

        centers = np.array([data[clusters == k].mean(axis=0) for k in
↪range(i)])
        sizes = np.array([len(data[clusters == k]) for k in range(i)])
        rel_sizes = sizes / sizes.sum() * 10000 # Normalize and scale
↪sizes for visibility
        plt.scatter(centers[:, 0], centers[:, 1], s=rel_sizes,
↪c=range(i), cmap='viridis', alpha=0.5)
        for center, size in zip(centers, sizes):
            plt.text(center[0], center[1], f'{{(size / sizes.sum() *
↪100):.1f}}%', ha='center', va='center', fontsize=8, color='white')

        plt.title(f'Level {i} Cluster Sizes ({method.capitalize()}
↪Linkage)')
        plt.xlim(ax_middle.get_xlim())
        plt.ylim(ax_middle.get_ylim())

        plt.tight_layout()
        plt.show()
        fig.savefig(f'./output/exploration/dimred/hclustering/
↪{dr_technique}_level_{i}_hclustering.png')

```

```

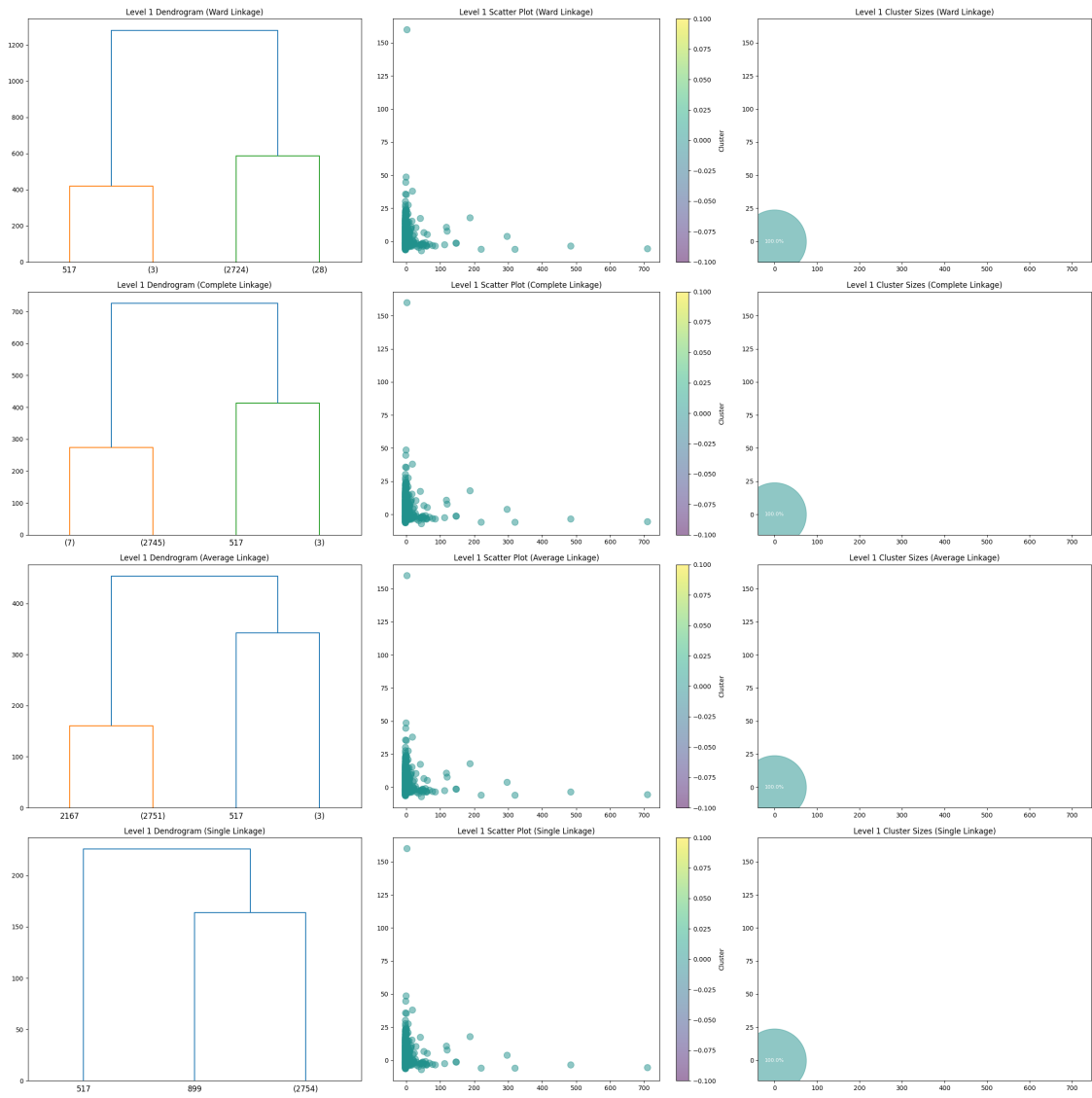
[124]: df['dim_red_pca_x'] = pca_transformed[:, 0]
        df['dim_red_pca_y'] = pca_transformed[:, 1]
        pca_results = df[['dim_red_pca_x', 'dim_red_pca_y']]

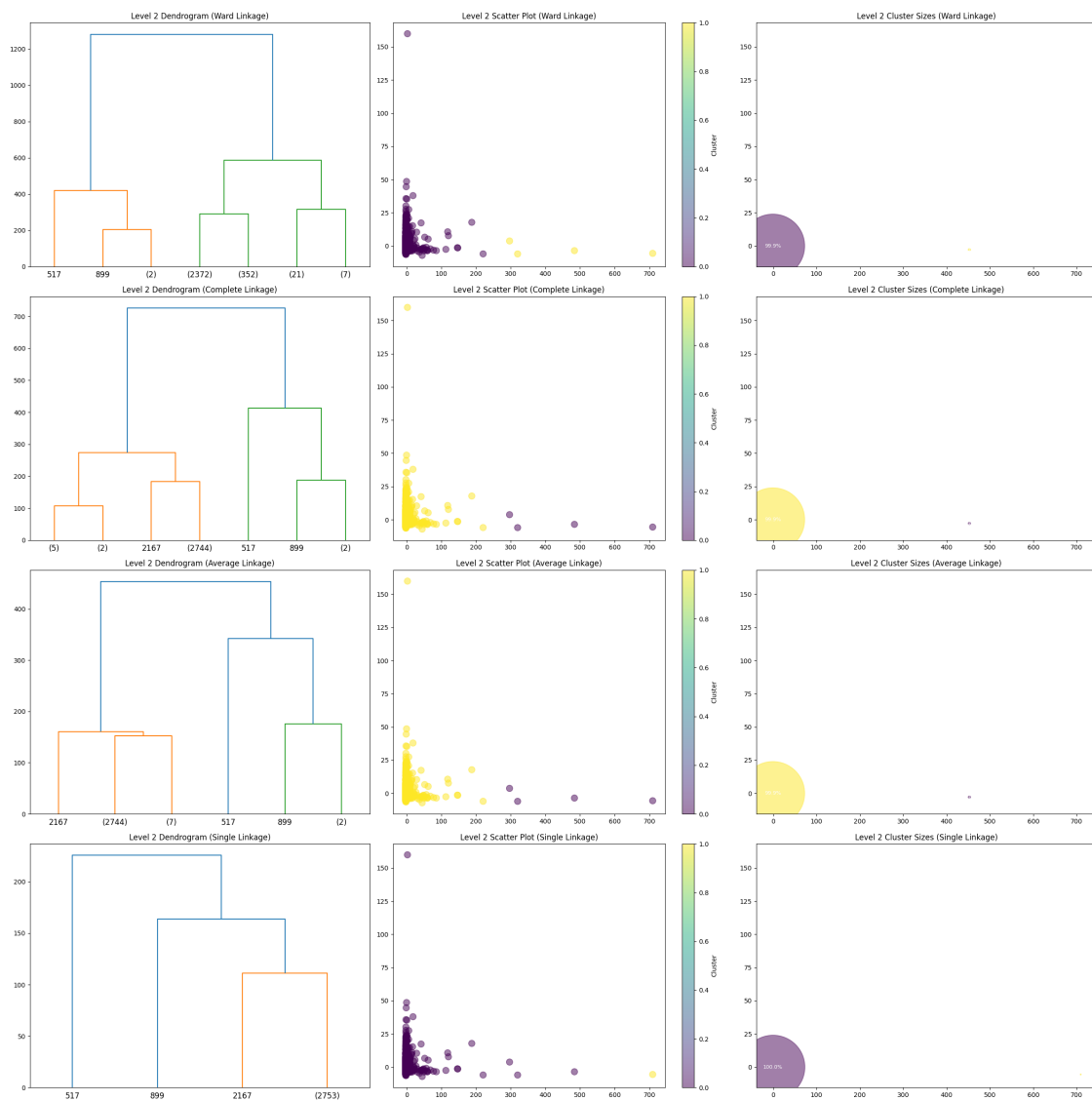
```

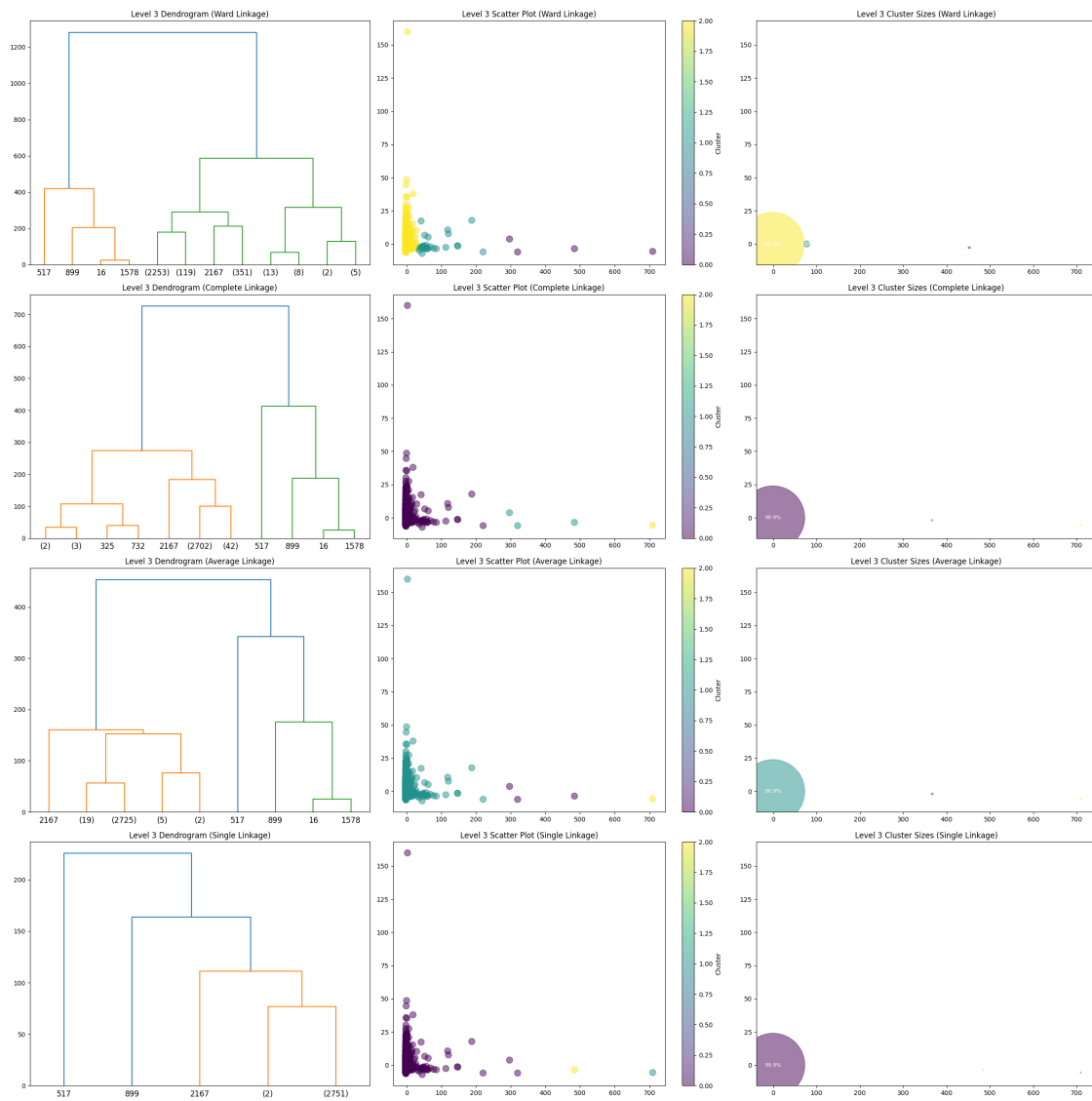
```

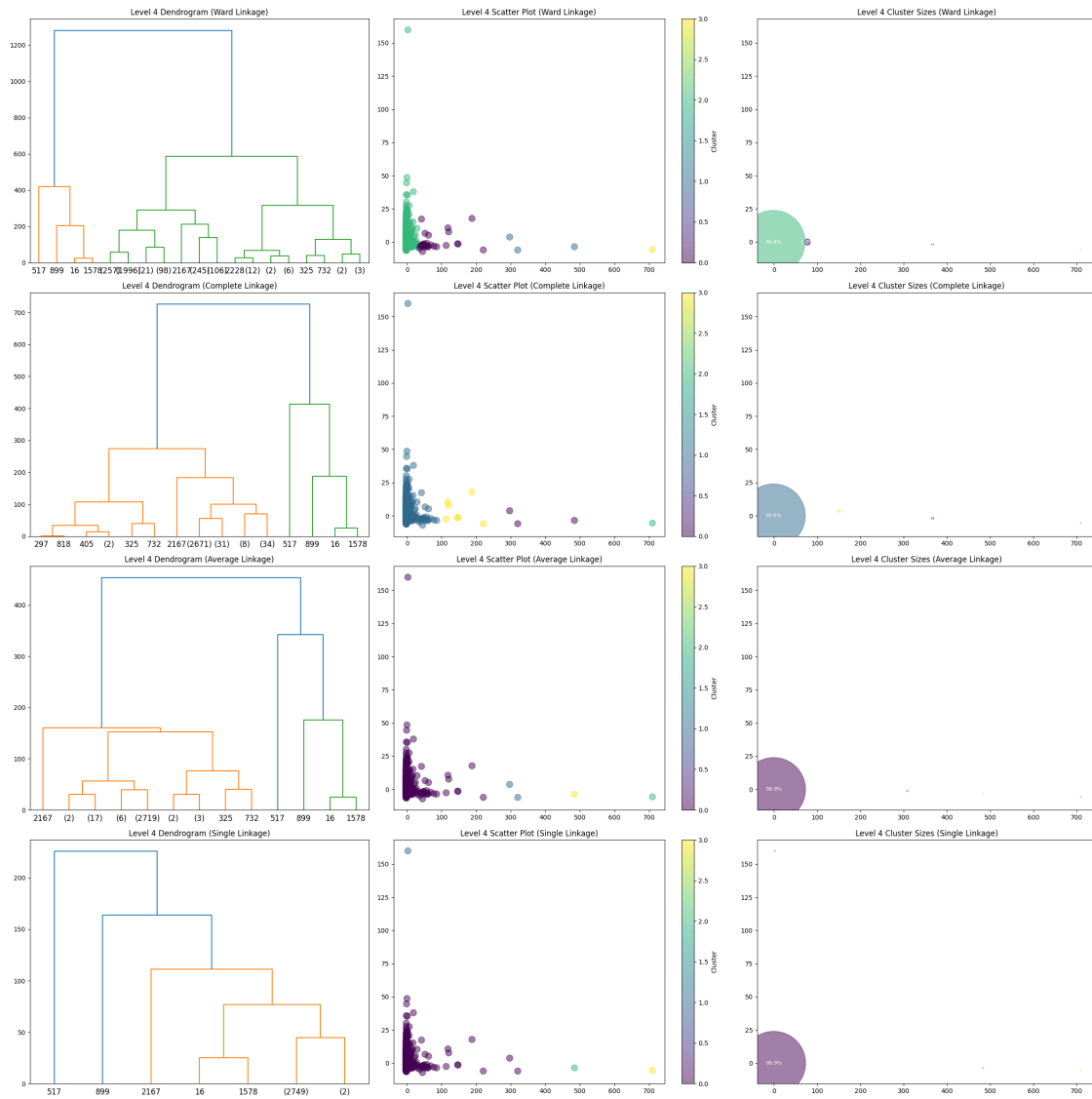
[125]: plot_dendrogram_with_methods(pca_results.values, 'pca',
↪levels=MAX_CLUSTER_LEVEL)

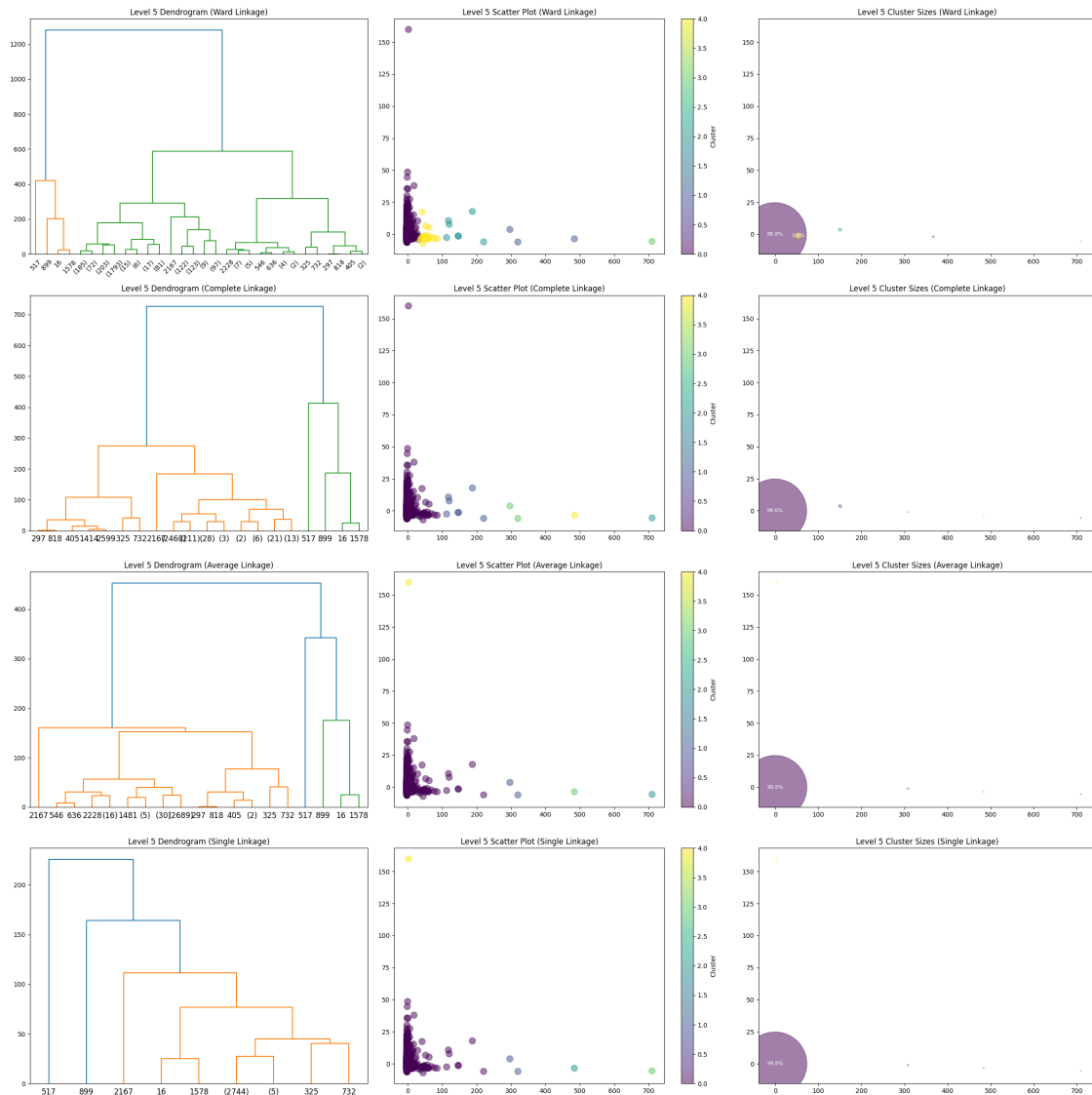
```

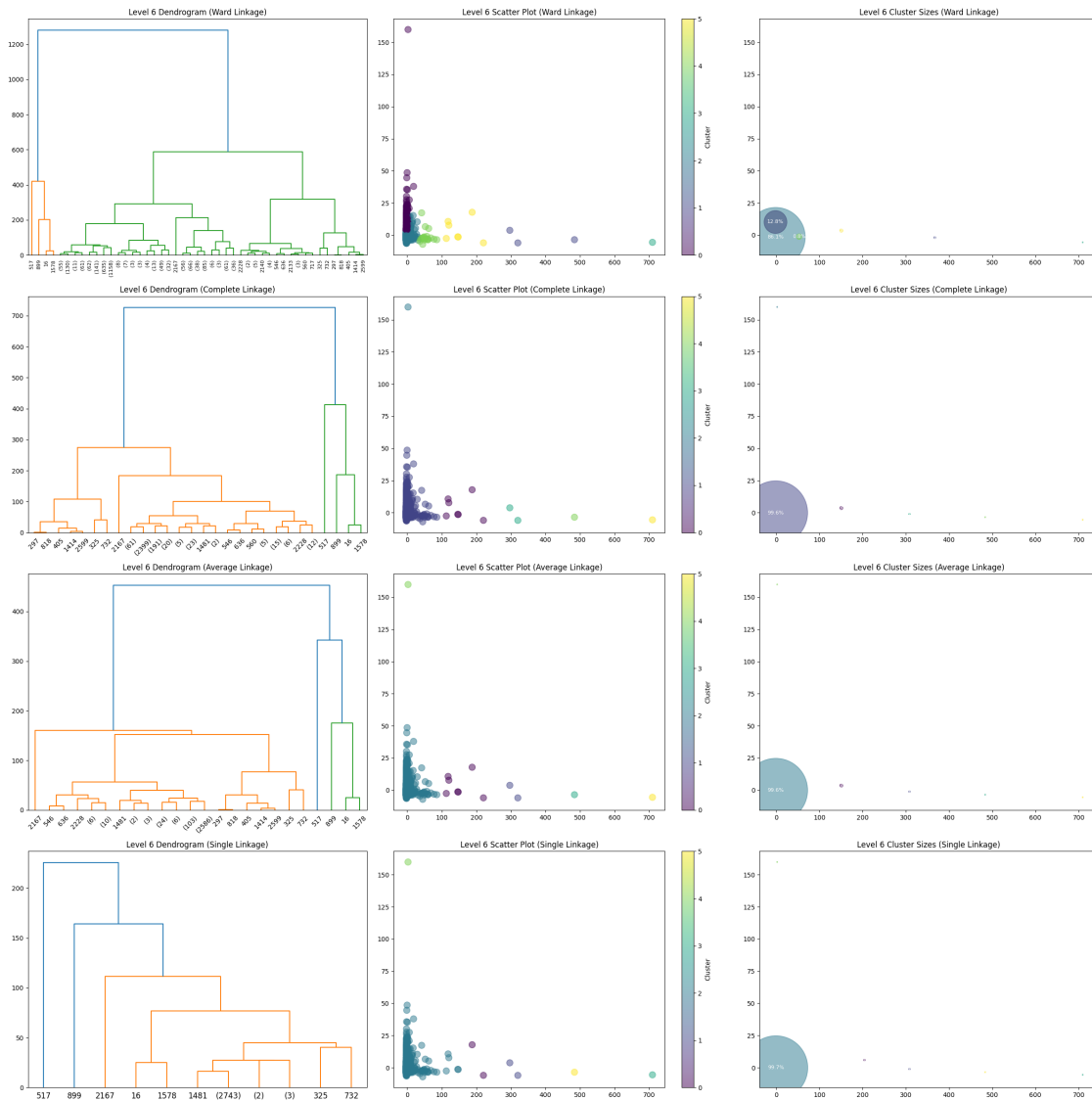








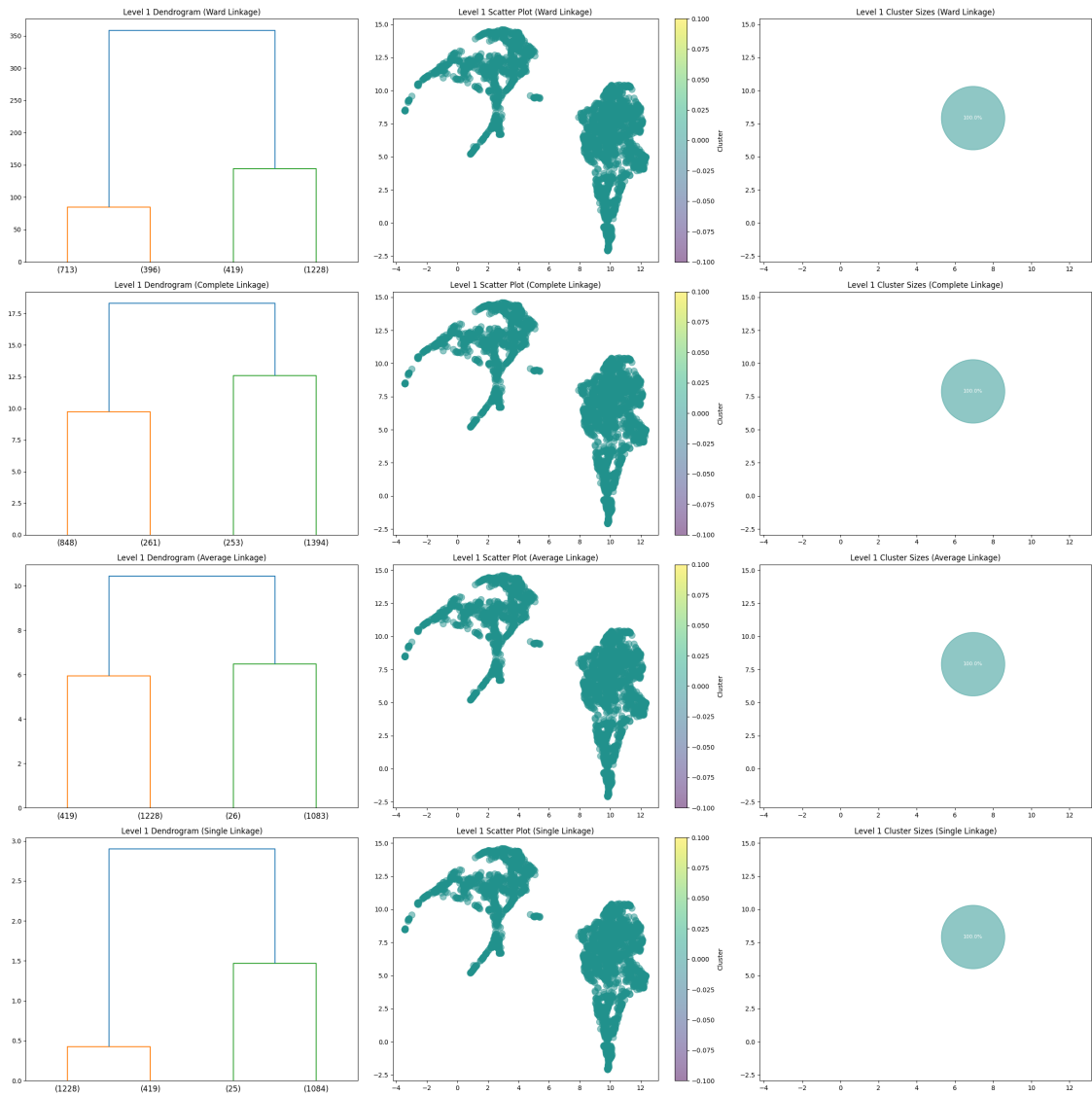


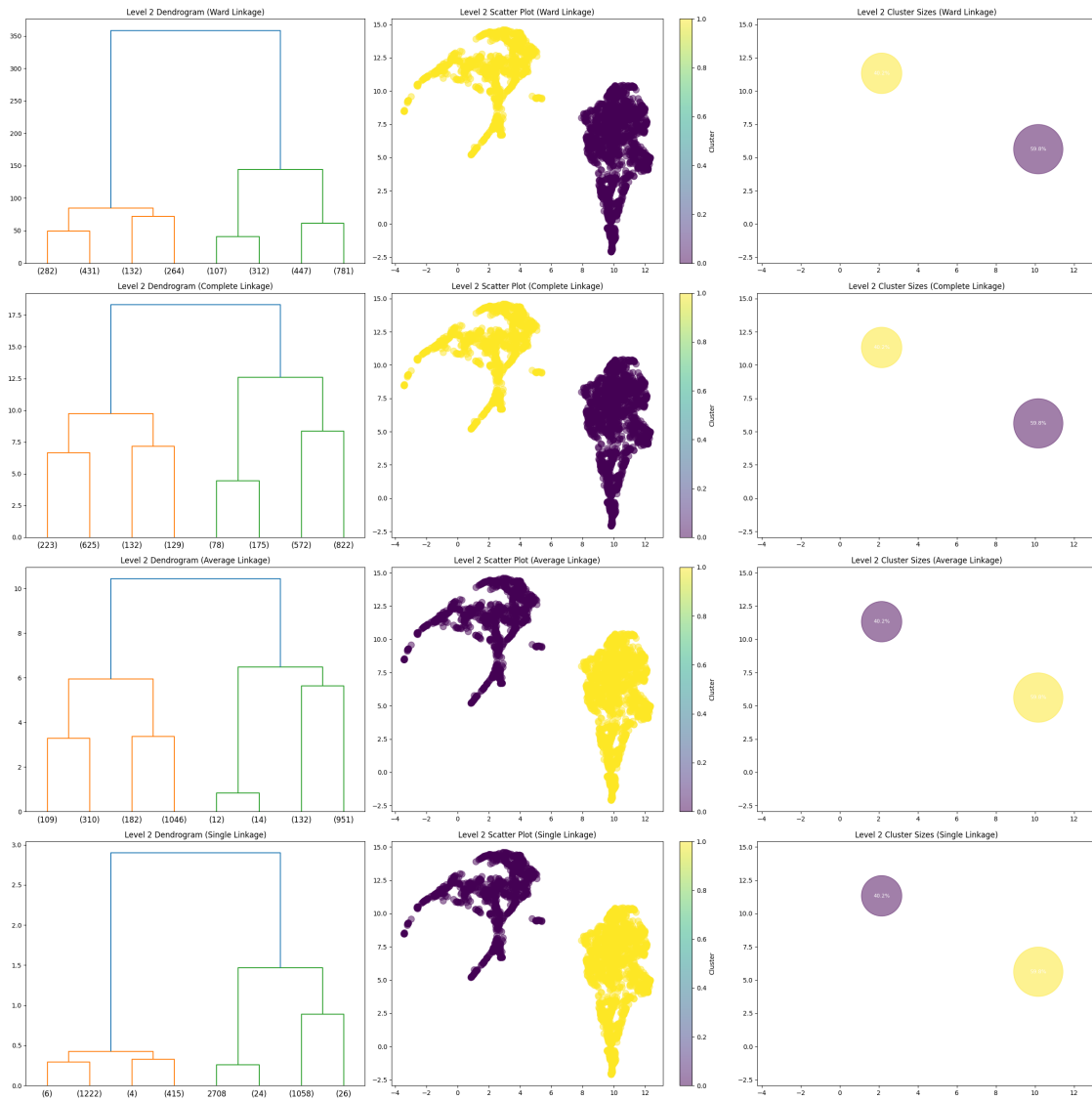


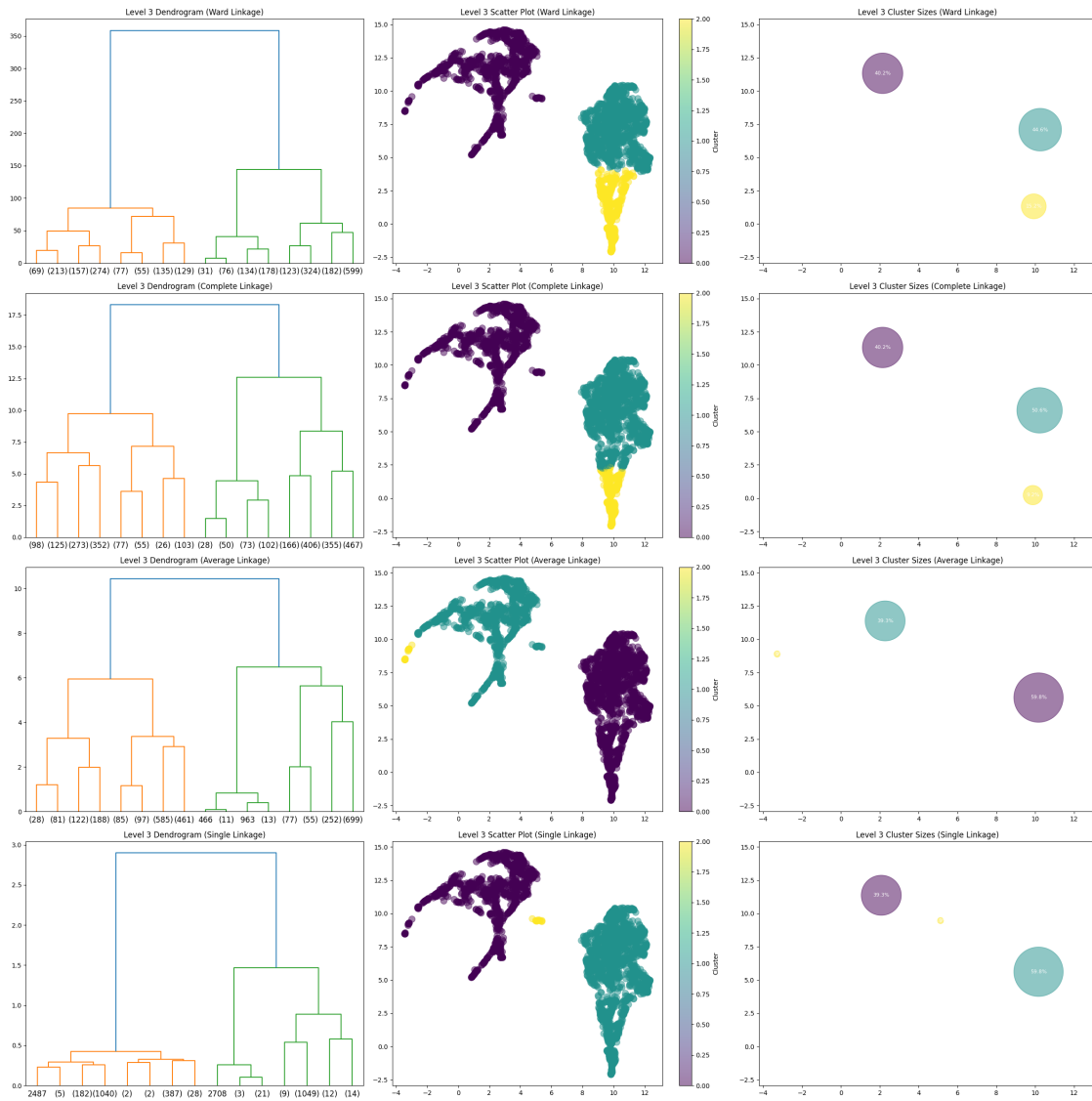
```
[126]: df['dim_red_umap_x'] = umap_transformed[:, 0]
df['dim_red_umap_y'] = umap_transformed[:, 1]
umap_results = df[['dim_red_umap_x', 'dim_red_umap_y']]
```

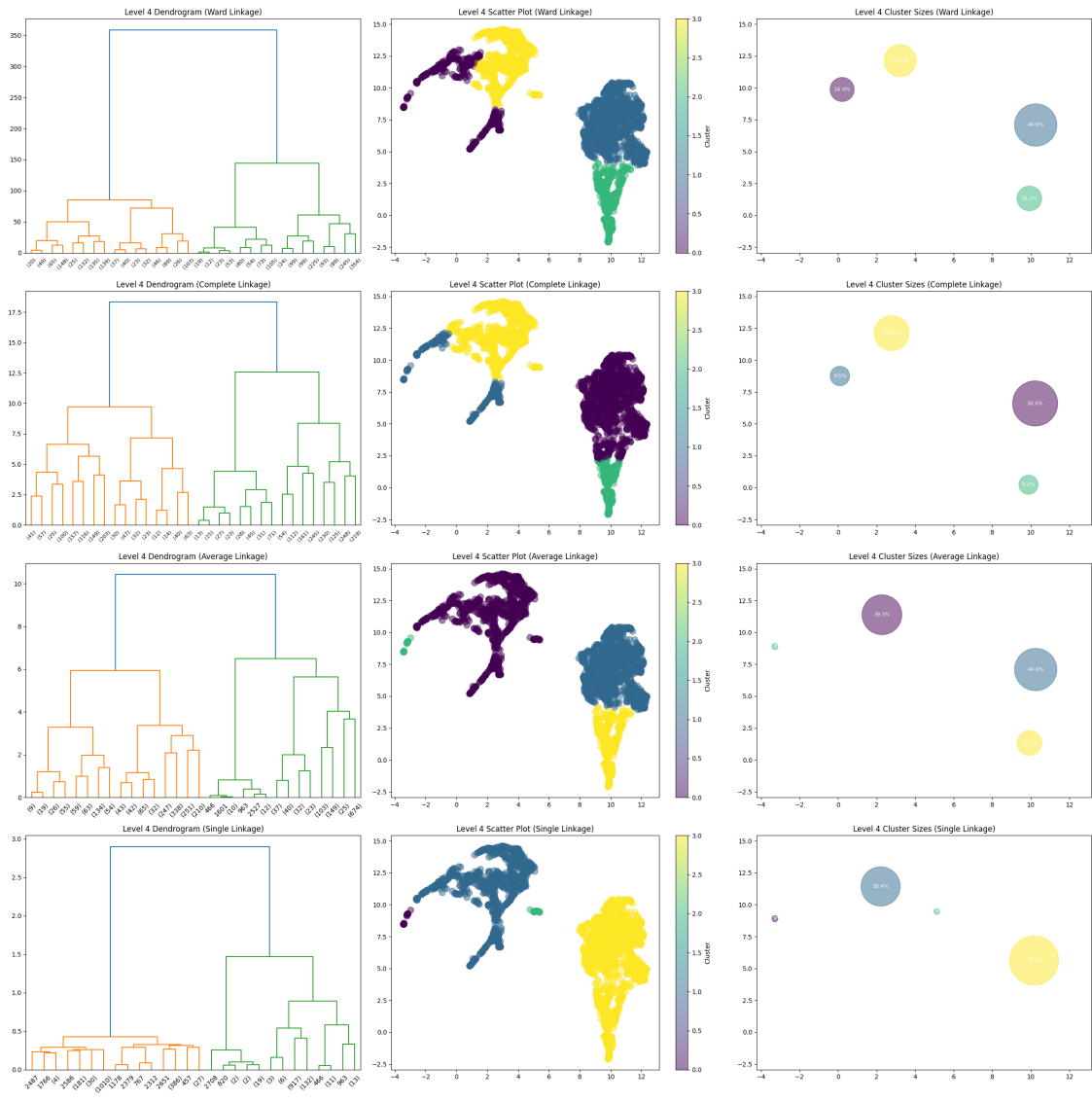
```
[127]: df['dim_red_tsne_x'] = tsne_transformed[:, 0]
df['dim_red_tsne_y'] = tsne_transformed[:, 1]
tsne_results = df[['dim_red_tsne_x', 'dim_red_tsne_y']]
```

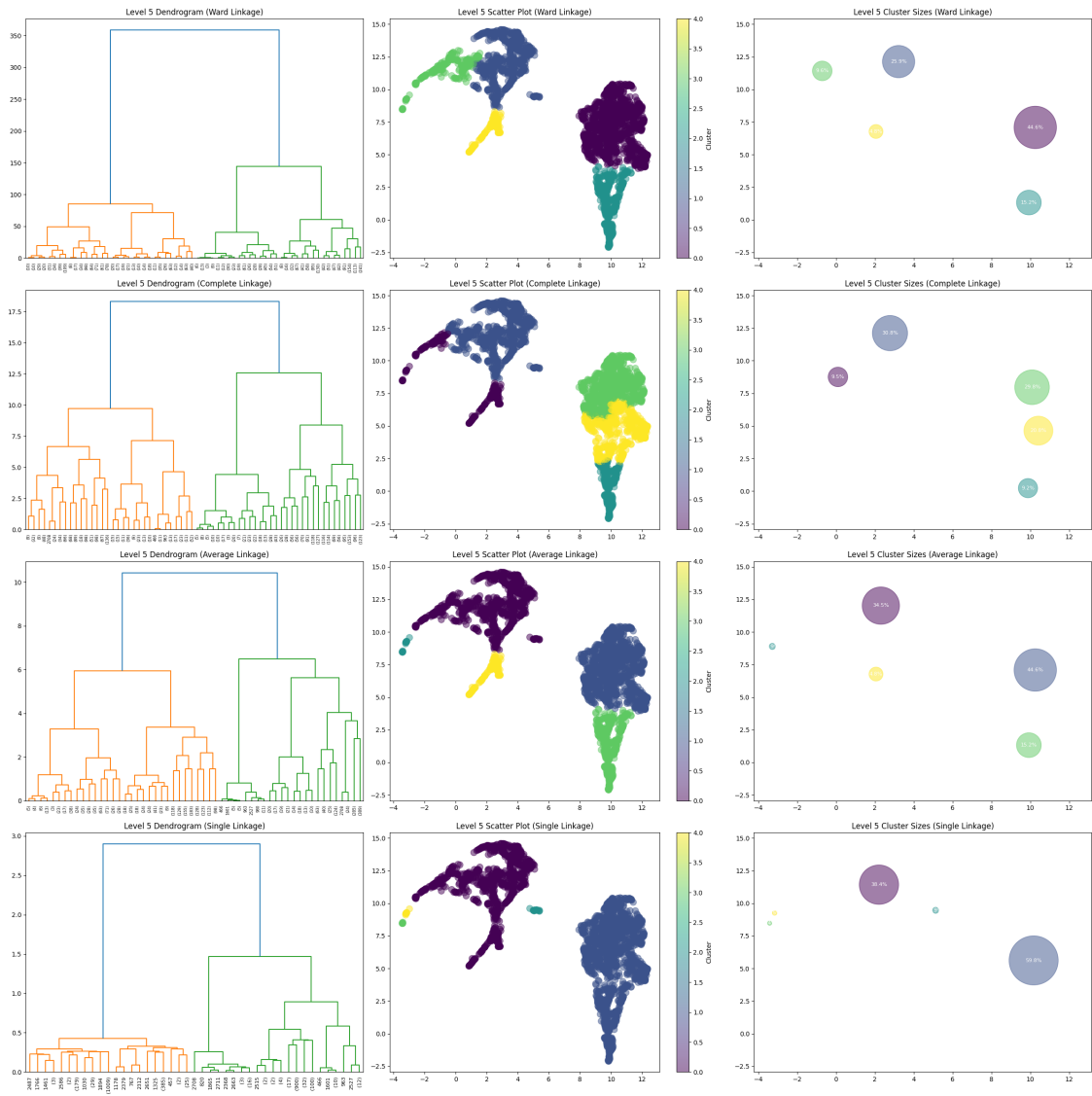
```
[128]: plot_dendrogram_with_methods(umap_results.values, 'umap',
↪ levels=MAX_CLUSTER_LEVEL)
```

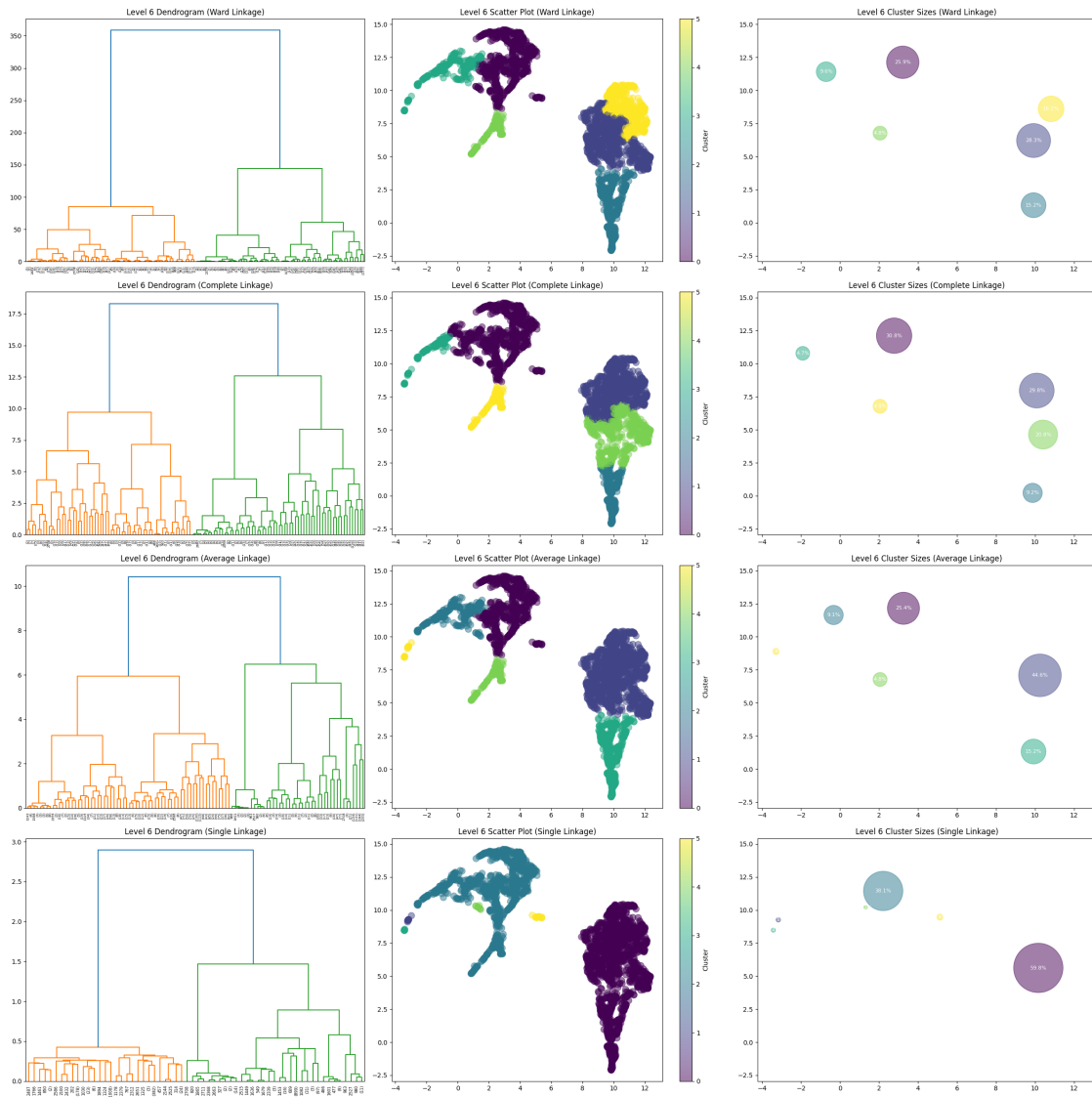




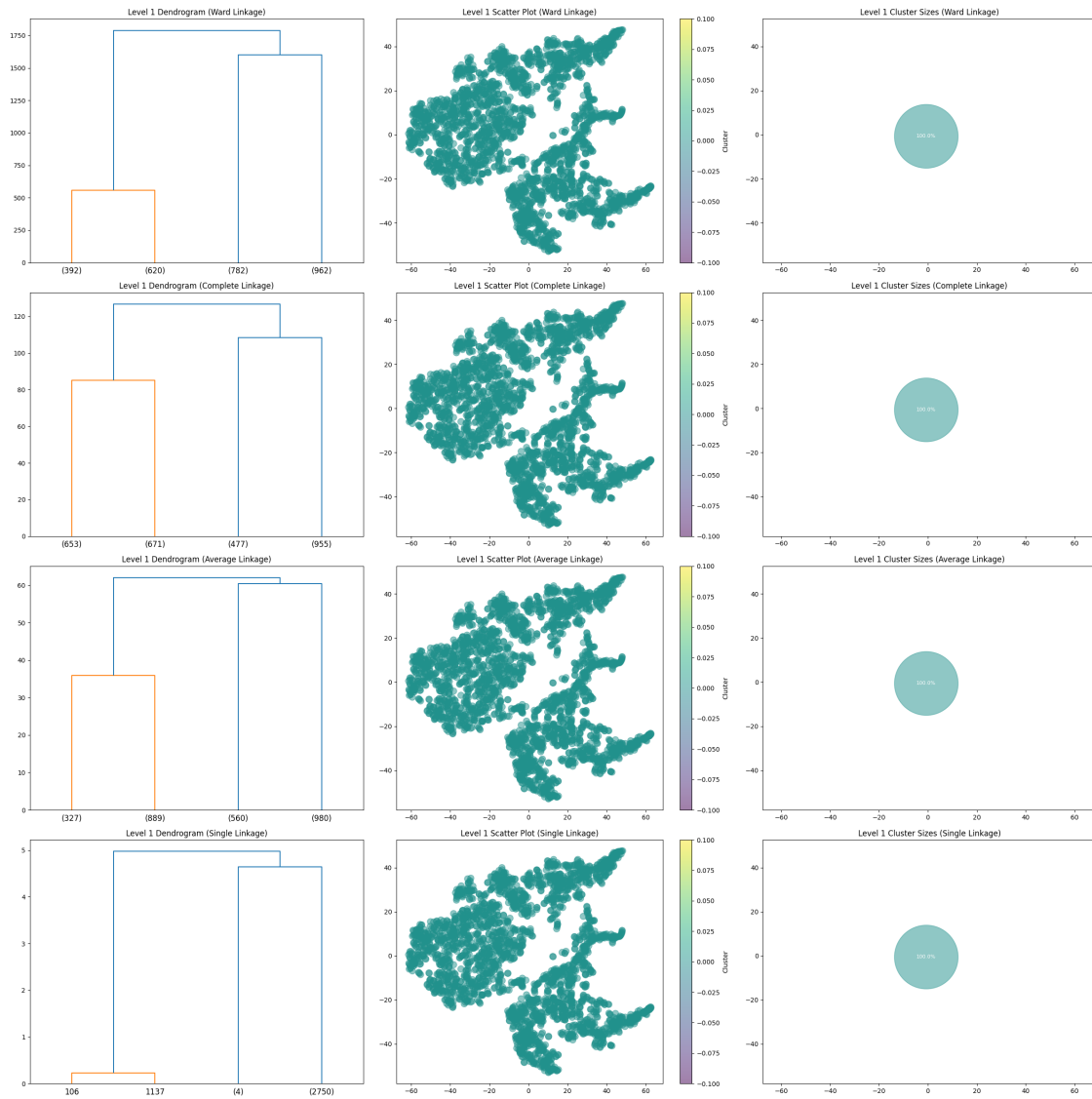


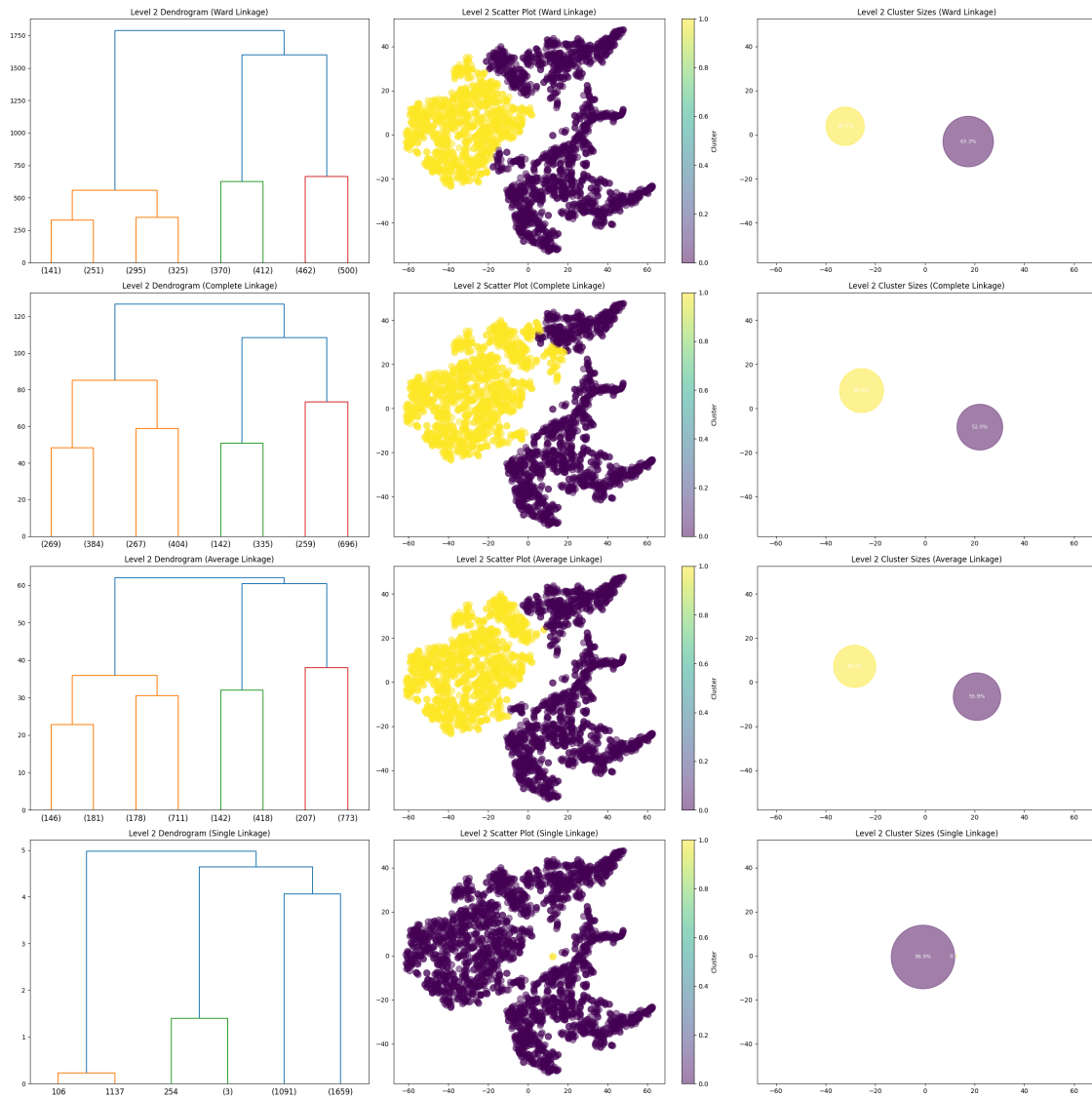


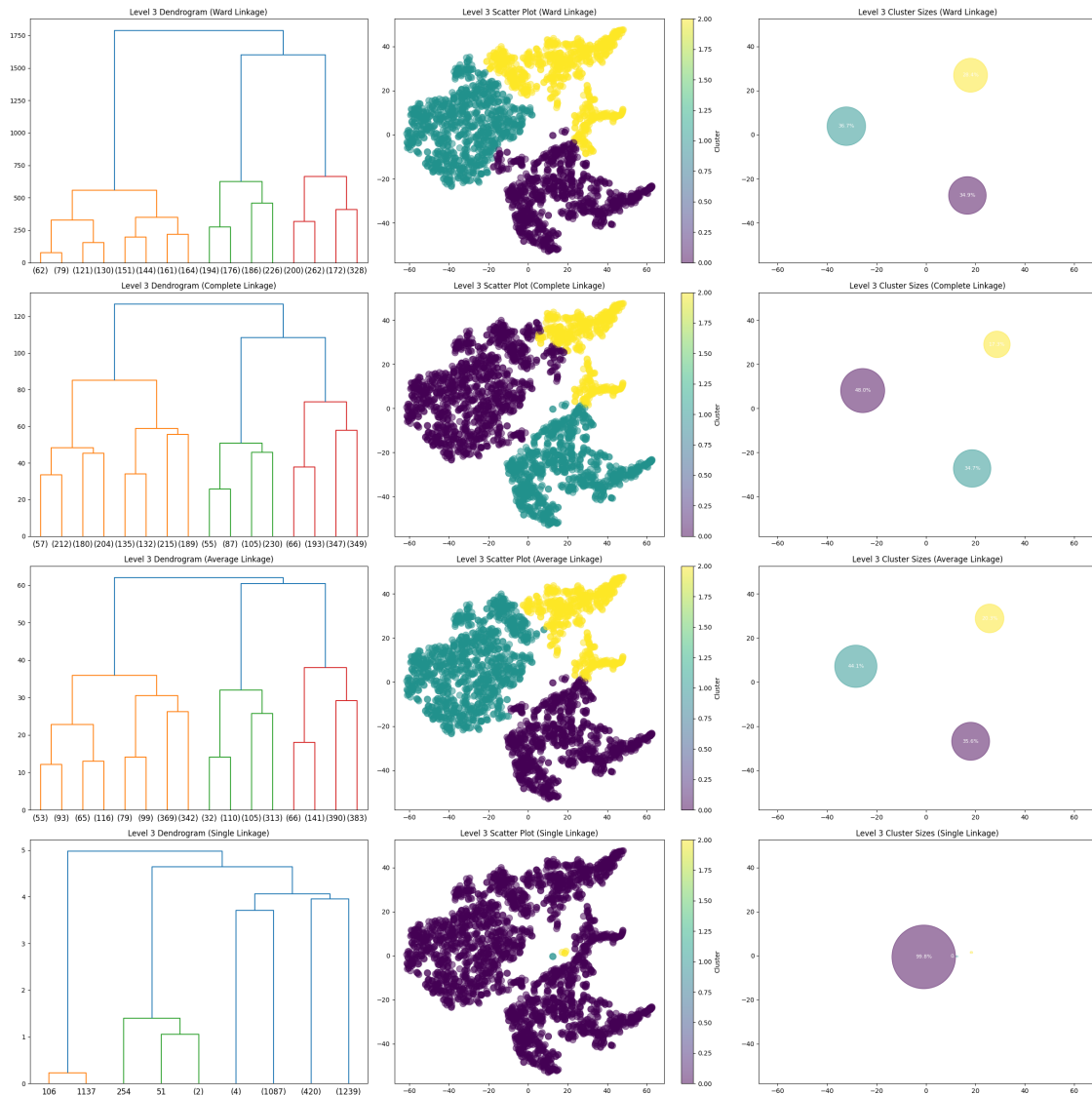


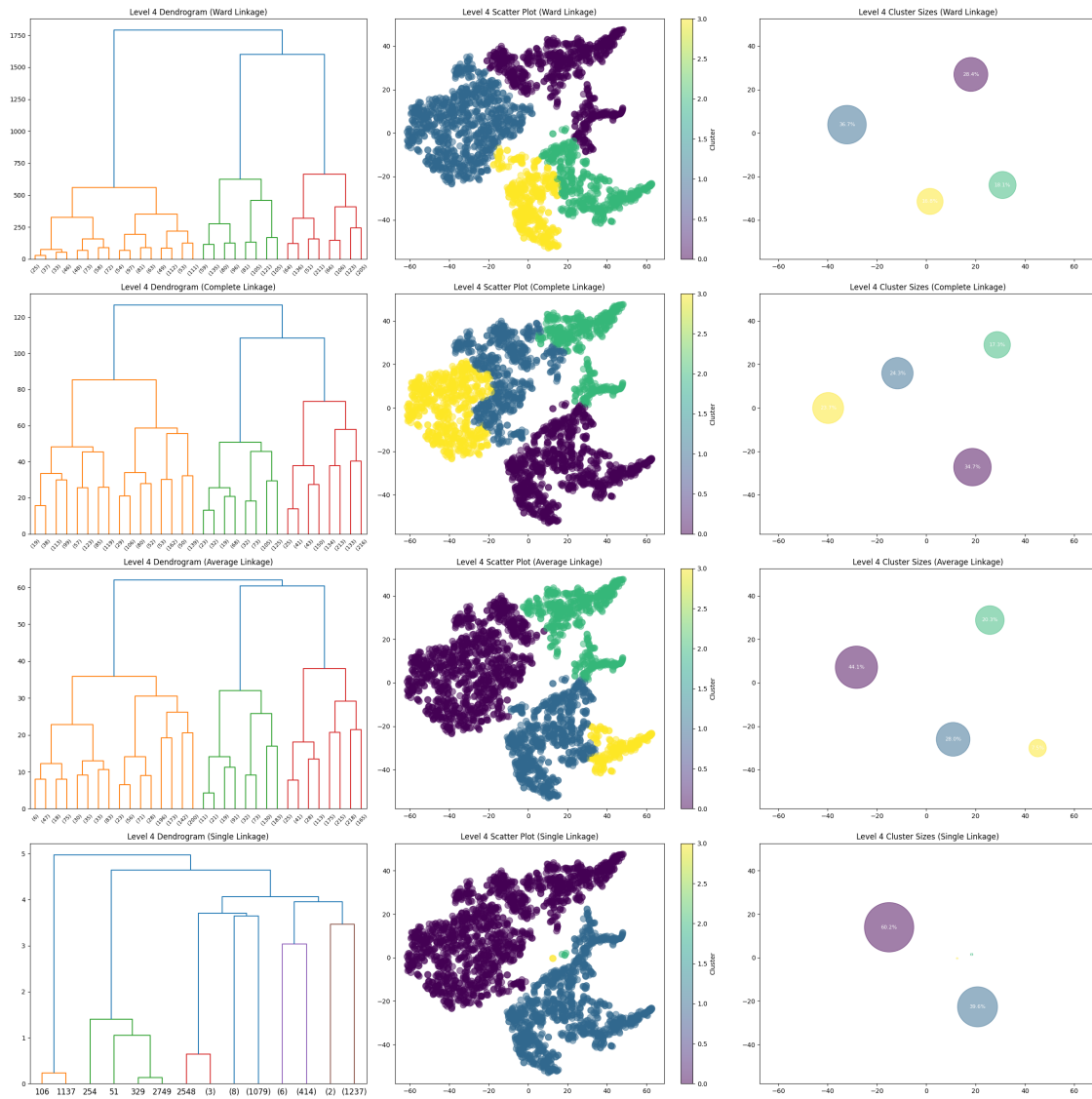


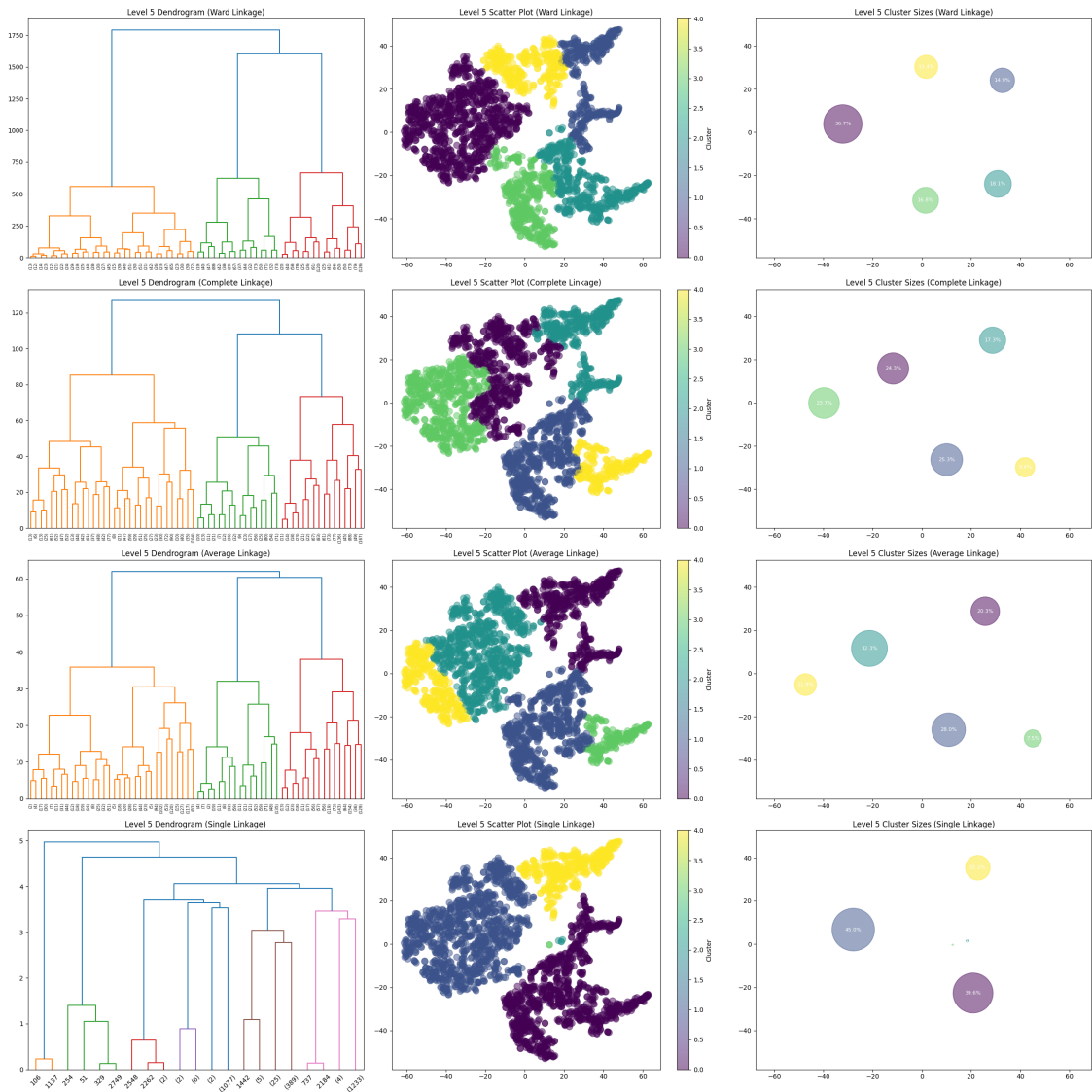
```
[129]: plot_dendrogram_with_methods(tsne_results.values, 'tsne',
    ↪levels=MAX_CLUSTER_LEVEL)
```

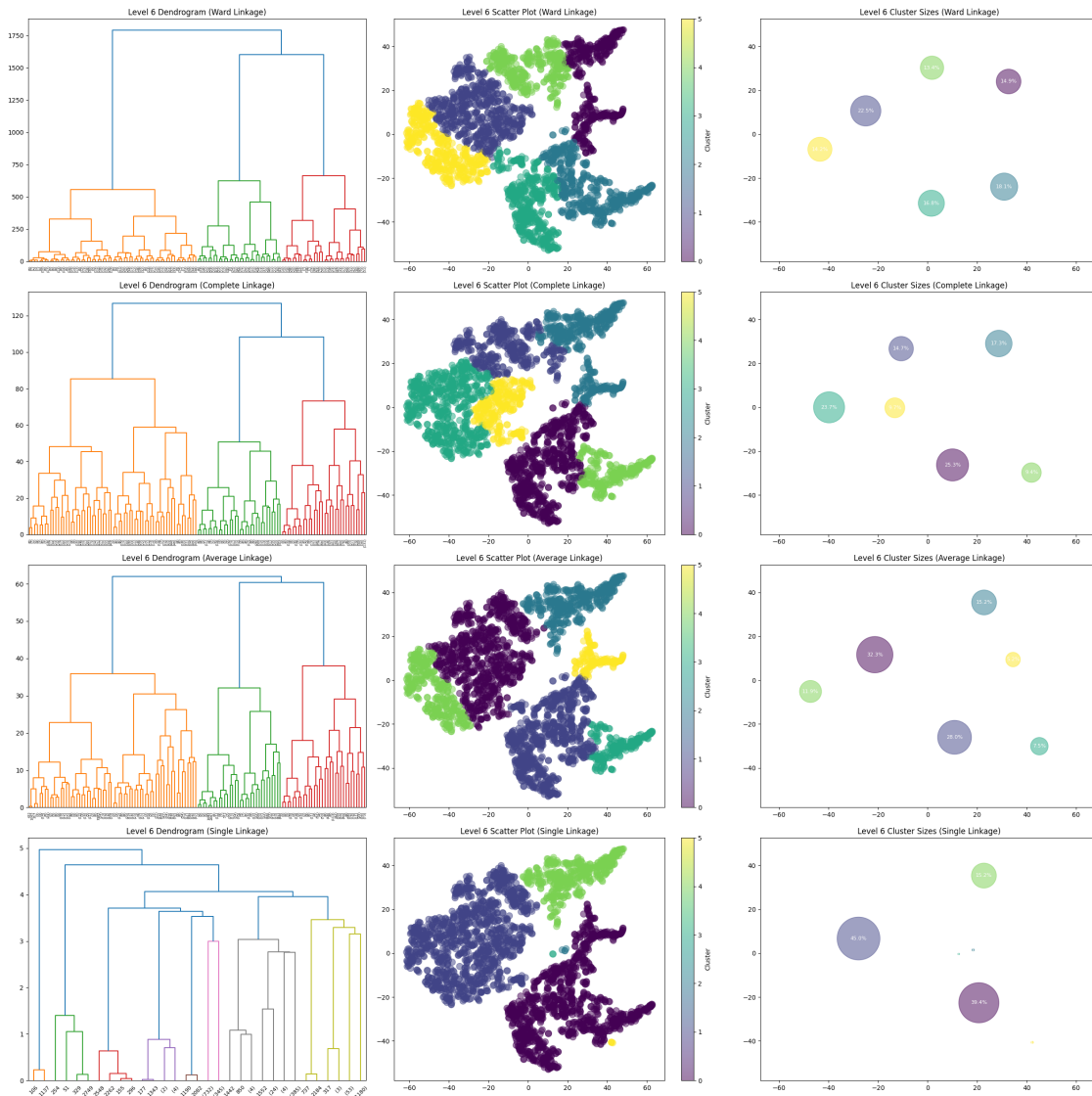












18 1C. Summary

At the end of the report include a summary section in which you summarise your findings. The summary is not a narrative of what you have done, but a condensed informative section of what you have found about the data that you should report to the Head of the Analytics Unit. The summary may include the most important findings (specific characteristics (or values) of some attributes, important information about the distributions, some clusters identified visually that you propose to examine, associations found that should be investigated more rigorously, etc.).

condensed informative section of what you have found about the data that you should report to the Head of the Analytics Unit

19 Deliverables

The deliverables are:

- A report, for which the structure should follow the tasks of the assignment, and
- An Excel workbook file with individual spreadsheets for each task (spreadsheets should be labelled according to the task names, for example, “1A”). Each of the results of parts (a) to (d) in task 1B should be presented in a separate sheet (and respectively table in the assignment report).
- In the report, include a section (starting with a section title) for each of the tasks in the assignment.

20 Notes

- 2-class (A,B) stellar classification
- For Larger Amounts of Missing Data (like 40%) or Complex Patterns: KNN might provide more accurate imputations since it considers the similarity between observations. (GRVSmag and e_GRVSmag) -> KNN but normalize first
- Same for Age-Flame (40%) -> KNN but first normalize
- Mass-Flame (8.1) Median is more robust to outliers and skewed data.
- Delete rows of missing z-flame and Lum-Flame
- Binning:
 - Square-root choice: Use the square root of the number of data points as the number of bins. This is a simple and commonly used rule of thumb.
 - Sturges’ formula: This formula suggests using $1 + \log_2(n)$ is the number of data points. It is based on the assumption that the data is approximately normally distributed and tends to produce a smaller number of bins.

21 Export

```
[130]: !jupyter nbconvert fda_a2_25061944.ipynb --to pdf --out fda_a2_25061944.pdf
```

```
[NbConvertApp] WARNING | Unrecognized alias: 'out', it will have no effect.
[NbConvertApp] Converting notebook fda_a2_25061944.ipynb to pdf
/Users/nicolas/Library/Caches/pypoetry/virtualenvs/ml-hub-tji4fZ52-py3.11/lib/python3.11/site-packages/nbconvert/utils/pandoc.py:51:
RuntimeWarning: You are using an unsupported version of pandoc (3.1.12.3).
Your version must be at least (1.12.1) but less than (3.0.0).
Refer to https://pandoc.org/installing.html.
Continuing with doubts...
  check_pandoc_version()
[NbConvertApp] Support files will be in fda_a2_25061944_files/
[NbConvertApp] Making directory ./fda_a2_25061944_files
[NbConvertApp] Making directory ./fda_a2_25061944_files
[NbConvertApp] Making directory ./fda_a2_25061944_files
```

```
[NbConvertApp] Writing 287506 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 290440 bytes to fda_a2_25061944.pdf
```